경로 추종 성능 분석
문서 모음

2026.10.20 · RB 협동로봇 · rb_servo_gui / jb2_gui

요약에서 상세 실험, 반복성 분석, 기술 부록 순으로 묶었습니다.
기존 보고서의 내용과 실험 조건을 유지한 합본입니다.

문서별 목차

문서	합본 쪽	수록 범위
추종 성능 확인 요약 · 2페이지	2–3	2026-10-20 / 평가 방법과 주요 결과
궤적 추종 성능 평가 전체 보고서	4–34	10/08 방식 비교 및 10/14 servo 프로파일 실험
웨이포인트 분할·속도·경유점 도달 오차	35–46	10/16 jb2·servo 비교
동일 조건 반복 기록·오차 방향 분석	47–49	2026-10-20 / 기존 기록의 예비 분석
평가 지표 해설	50–71	산출식·그래프 / 일부 DTW 설명은 이전 방식
외부 스크립트 인터페이스 정의서	72–79	통신·명령 인터페이스 참고
결과 데이터 명세서	80–85	결과 CSV·궤적 CSV 정의
실험 데이터 명세서	86–97	원시 실험 데이터 구성 정의

읽기 전 확인

- 보고서 간 분석 코드·오차 정의가 다릅니다. 특히 10/08 DTW 매칭쌍 오차와 10/14 최단 경로 거리를 직접 비교하거나 개선 배율로 해석하지 않습니다.
- 반복성 분석은 저장된 실기 2회에서 확인한 예비 결과이며, 통계적 반복 정밀도나 작업공간 오차 방향을 확정하지 않습니다.
- sources에는 PDF·Markdown·HTML·그림·이미 포함된 집계 CSV/JSON 및 생성 스크립트를 보존했습니다. 외부 실행 폴더의 전체 원시 측정 데이터는 포함하지 않았습니다.
- 데이터 명세서의 results/, trajectories/, runs/ 등은 별도 데이터 패키지 설명입니다. 이 모음에 해당 전체 데이터가 포함된다는 뜻은 아닙니다.

쪽 번호는 통합 PDF의 첫 페이지부터 센 번호입니다. 개별 문서에 표시된 자체 페이지 번호와 다를 수 있습니다.

경로 추종 성능 확인 요약

평가 방법과 확인 순서 · 1 / 2 | RB rb20_1900es · rb_servo_gui · 2026.10.20 요약

1. 무엇을 확인하는가

추종 성능은 **경로 정확도, 시간·속도 추종, 정지 정확도**를 나누어 확인한다. 경로를 비슷하게 지나갔더라도 시점이 늦거나 반전점을 건너뛰면 동일한 성능으로 볼 수 없다. 속도가 부드럽다는 이유만으로 경로 정확도가 높다고 판단하지 않는다.

2. 측정과 비교의 원칙

- 기준은 원본 경로와 실제 전송한 지령을 함께 보관한다. 분할·보간 때문에 생긴 경로 차이를 로봇 추종 오차와 구분한다.
- **실기:** 엔코더각 jnt_ang, **시뮬레이션:** 지령각 jnt_ref로 평가한다. 모드·궤적·속도·가속도·명령 주기·분할 수 ·gain/alpha·blending을 기록한다.
- 관절별 위치·부호 있는 속도·가속도를 같은 시간축에 겹쳐 본다. 이상 프로파일과 실측의 시간 차이를 먼저 확인하고, DTW는 시간 차이를 줄여 경로 형상을 비교하는 보조 수단으로 사용한다.

3. 추종 성능 확인표

확인 항목	볼 지표·그래프	판단할 내용
경로 정확도	매칭쌍의 6관절 L2 오차 p95·최대, 시간별 오차	p95는 전반적 수준, 최대는 국소 이탈. 최대 발생 시점의 자세를 확인
경로 누락·반전	커버리지, 진행도, 관절별 극값	DTW가 숨길 수 있는 건너뜀·찍턴 미도달 확인. 목표 극값과 실측 극값을 직접 비교
시간·속도 추종	이상/실측 속도·가속도, 추종 시간, 전송 시각	지연·감속·속도 제한·주기 흔들림 확인. 경로 정확도와 별도 판단
시작·종료 정확도	안정 구간의 관절각과 목표각 차이	최종 자세 오차 및 오버슈트 확인. 경로 오차가 작아도 별도 평가
이동 안정성	속도 리플, 횡방향 흔들림, 정체 비율	가다 서다 반복과 빠른 떨림 구분. 리플은 방식별 비교 지표
실제 공간 영향	링크 표면의 최대 변위 [mm]	작은 관절각 오차도 긴 팔 끝에서 큰 위치 차이로 확대될 수 있음

원문 초기 판정값: 매칭쌍 오차 p95 ≤ 1.0°, 최대 ≤ 2.0°, 횡방향 흔들림 ≤ 0.3°, 정체 비율 ≤ 2%. 응용 요구로 검증된 합격선이 아닌 초기값이다. 원문 10/08 종합 판정에서는 속도 리플을 제외했다.

4. 결과를 읽는 순서

① 전체 위치·시간 비교 → ② 최대 오차 구간 확대 → ③ 반전점·종료점 확인 → ④ 속도·가속도와 전송 간격 확인 → ⑤ 같은 조건끼리 비교. 관절별 값과 6관절 L2 값을 혼동하지 않는다. 최단 경로 거리만 작다고 순서·시간 추종까지 검증된 것은 아니다.

출처: 원문 「RB 협동로봇 외부 스크립트 기반 궤적 추종 성능 평가 보고서」 1.3~1.7, 3.8절. 수치는 기존 보고서의 요약이며 신규 실험 결과가 아니다.

실측 결과와 판단

추종 성능 검증 결과 · 2 / 2 | 2026.10.08 방식 비교 / 2026.10.14 프로파일 타이밍 실험

1. 명령 방식별 정확도와 부드러움 — 10/08

궤적 A·B·C × 시뮬레이션·실기 × 4방식, **총 24회**. 아래 값은 방식별 6회 중앙값이며, 당시 분석 코드의 DTW 매칭쌍 기준이다.

명령 방식	추종 오차 p95 [°]	최대 오차 [°]	속도 리플
jb2 · blend 0	0.31	0.59	48%
jb2 · blend 50	0.35	0.58	23%
servo · gain 1, alpha 1	0.37	0.58	15%
jb2 · blend 100	0.41	0.58	8%

관찰: blending 증가 시 리플은 감소했지만 p95 오차는 증가했다. 최대 오차는 거의 같아 최대값만으로 방식 차이를 구분하기 어렵다. 원문은 0.2초 간격 지령점의 직선 연결과 원본 경로 사이 차이를 주요 오차 요인으로 분석했다. 리플을 제외한 종합 판정은 20/24회 PASS, 4회는 진행도 기준 미달이었다.

2. 속도·가속도 프로파일 추종 — 10/14 실기

설정 속도 **30°/s**, 가속도 **50°/s²**, 전송 주기 **20ms**, gain/alpha=1, t2=0.04초. A·B·C 각 1회. 이 표의 경로 오차는 **측정점에서 원본 경로까지 최단 거리**다.

궤적	이상 / 측정 시간 [s]	거리 p95 / 최대 [°]	최대 속도 [°/s]	최대 가속도 [°/s²]
A	14.84 / 14.72	0.054 / 0.074	31.8	77
B	8.62 / 8.52	0.035 / 0.051	27.8	67
C	7.30 / 7.18	0.029 / 0.040	20.4	59

확인된 성능: 세 실험의 경로 거리 최대는 0.074° 이하, 종료 오차는 약 0.001°. 다만 A의 측정 최대속도는 설정을 약 6% 초과했고, 세 실험 모두 측정 최대가속도가 설정값 50을 초과했다. 지령 최대가속도는 48.9~49.9로 제한 이내였다. 제어기 응답과 엔코더 차분 노이즈 중 무엇이 초과의 원인인지는 미확정이다.

3. 결론과 후속 확인

경로 형상·종료 자세 추종은 해당 실험에서 양호했지만, 실측 속도·가속도 제한 준수까지 확인된 것은 아니다. 반전·최대 오차 구간 확대, 동일 필터 조건의 가속도 재계산, 반복 실험을 통해 확인해야 한다. 원문에서 링크 표면의 최악 순간 변위는 8~16mm였으므로 작업 정밀도는 관절각뿐 아니라 공간 오차로 판단한다.

비교 시 주의: 두 실험은 분석 버전과 오차 정의가 다르므로 위 두 표의 수치로 개선 배율을 계산하지 않는다. 10/14 최단 거리는 시간·순서를 검증하지 않으며, 극값 앵커 매칭의 스파이크 역시 무조건 실제 경로 이탈로 해석하지 않는다.

출처: 원문 3.5, 3.7, 3.8절 및 4~5절. 원본: rb_servo_gui/docs/report/report.md. 실기 프로파일 시험은 궤적당 1회로 반복 재현성은 추가 확인이 필요하다.

RB 협동로봇 외부 스크립트 기반 궤적 추종 성능 평가 보고서

- 대상: Rainbow Robotics rb20_1900es (RT, 제어기 IP 10.0.2.7), 외부 스크립트 API (TCP 5000/5001)
- 실험일: 2026-10-08 16:25 ~ 16:58 (3.1 ~ 3.7절), 2026-10-14 (servo 프로파일 타이밍, 3.8절)
- 비교 대상: 24회 (명령 방식 4종 × 궤적 3종 × 시뮬레이션·실기)
  - 일부 궤적·모드만 수행한 조건은 제외 (3.1절)
- 시험 도구: `rb_servo_gui` (저장소 내), 지표 상세 정의는 `rb_servo_gui/docs/metrics.md`
- 수식 표기: 전부 그림(PNG)으로 삽입, 뷰어의 수식 지원 여부와 무관

목차

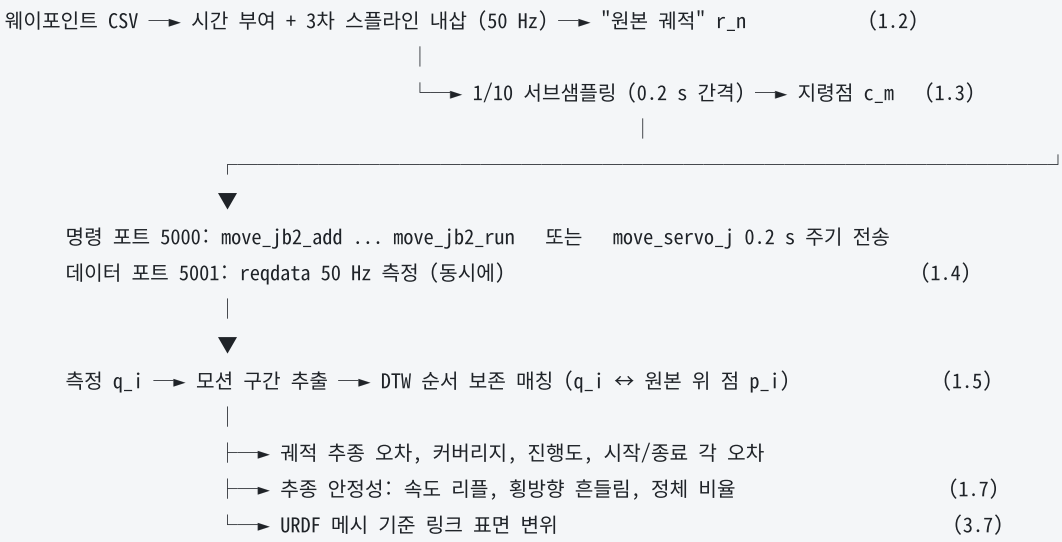
- 요약
- 1. 궤적 추종 성능 평가 방법
  - 1.1 전체 흐름
  - 1.2 입력 궤적의 연속화 (내삽)
  - 1.3 로봇 지령 궤적 생성 (서브샘플링)
  - 1.4 명령 전송과 동시 측정
  - 1.5 DTW 매칭쌍 생성
  - 1.6 실기 / 시뮬레이션 모드 평가 절차
  - 1.7 평가 지표: 추종 오차와 추종 안정성
- 2. 로봇 인터페이스: 외부 스크립트와 UI Script
  - 2.1 구성
  - 2.2 상태 데이터 (reqdata)
  - 2.3 UI Script 문법 예시
  - 2.4 사용 시 주의사항
- 3. 실기 및 시뮬레이션 성능 평가
  - 3.1 실험 조건과 비교 대상
  - 3.2 명령어와 파라미터
  - 3.3 move_jb2 결과
  - 3.4 move_servo_j 결과
  - 3.5 명령 방식별 종합: 추종 오차와 속도 안정성의 트레이드오프
  - 3.6 시뮬레이션과 실기 비교
  - 3.7 링크 메시 기준 변위
  - 3.8 servo 프로파일 타이밍 (10/14 추가 실험, 실기·시뮬레이션)
- 4. 결론과 권장 사항
- 5. 한계
- 부록. 첨부 데이터 (CSV)

요약

항목	결과
비교 대상	jb2 (accel 120, blend 0 / 50 / 100), servo (gain 1, alpha 1) × 궤적 A·B·C × 시뮬레이션·실기 = 24회
명령 방식별 트레이드오프	**추종 오차와 속도 안정성이 반비례.** 추종 오차 p95 중앙값 jb2 blend 0 0.31° < blend 50 0.35° < servo 0.37° < blend 100 0.41°, 속도 리플 중앙값은 반대 순서 48 % > 23 % > 15 % > 8 %
메시 변위 (rb20_1900es_RT URDF)	표면 변위가 추종 오차 순서를 그대로 따름: 최악 점 변위 시간 평균 1.37 / 1.78 / 2.10 / 2.49 mm, 시간 p95 4.6 / 4.9 / 5.1 / 6.1 mm (같은 순서). 최악 순간 값은 방식과 무관하게 8.0 ~ 16.0 mm
오차의 주 원인	**지령점 간격(0.2 s) 자체.** 지령점 직선 연결 경로와 원본의 차이 최대 0.44 / 0.59 / 0.67° (궤적 C / B / A), 측정 최대 오차와 거의 동일
move_servo_j 종료 처리	마지막 점 30회 반복으로 종료 각 오차 0.00° (미반복 시 3.59° 오버슈트)
시뮬레이션 대 실기	동일 조건 12쌍, 측정 경로 차이 p95 0.02 ~ 0.06°, 최대 0.09° 이하. 추종 오차·리플 거의 동일
servo 프로파일 타이밍 (10/14, 3.8절)	속도·가속도 설정으로 만든 프로파일을 20 ms 고정 주기로 전송. 실기 3회 모두 원본 경로까지의 거리 p95 0.03~0.05° (최대 0.04~0.07°), 종료 오차 0.001°. 같은 조건의 실기·시뮬레이션 경로 차이 p95 0.028° (궤적 C). 측정 가속도는 실기에서 한계를 넘음 (59~77 대 한계 50)
판정 (리플 제외 8개 항목)	24회 중 PASS 20회. FAIL 4회는 궤적 C jb2 의 진행도 0.989 ~ 0.990 (기준 0.99)

1. 궤적 추종 성능 평가 방법

1.1 전체 흐름

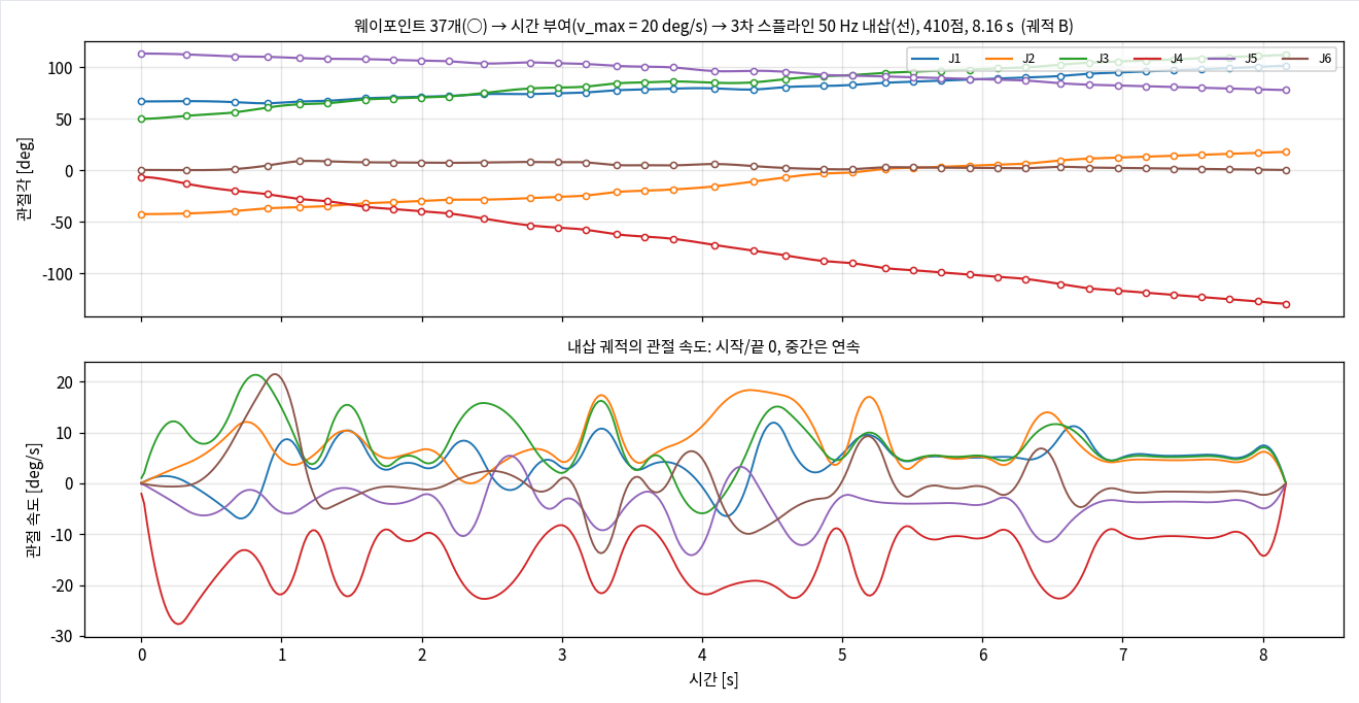


1.2 입력 궤적의 연속화 (내삽)

- 배경
  - 경로 계획기 웨이포인트: 간격 불균일, 시간 정보 없음
  - 그대로 전송 시 점 사이 움직임이 제어기 보간 방식에 좌우됨
  - → **시간이 부여된 연속 궤적**으로 변환 후 평가 기준(원본)으로 사용
- 처리 절차
  1. 연속 중복 웨이포인트 제거
  2. 구간별 시간 부여: 최대 이동 관절이 `v_max` (20 deg/s)로 움직이는 시간, 하한 0.2 s
  3. 3차 스플라인(시작/끝 속도 0) 내삽 후 50 Hz(0.02 s 간격) 샘플링 → 위치·속도 연속

$$\Delta t_k = \max\left(\frac{\max_j |w_{k+1,j} - w_{k,j}|}{v_{max}},\ 0.2\ s\right), \qquad t_k = \sum_{m<k} \Delta t_m$$

$$\mathbf{r}(t) = \text{CubicSpline}(\{t_k, \mathbf{w}_k\}), \quad \dot{\mathbf{r}}(0) = \dot{\mathbf{r}}(T) = \mathbf{0}, \qquad \mathbf{r}_n = \mathbf{r}(n/f_s),\ f_s = 50\ \text{Hz}$$



웨이포인트와 내삽 궤적

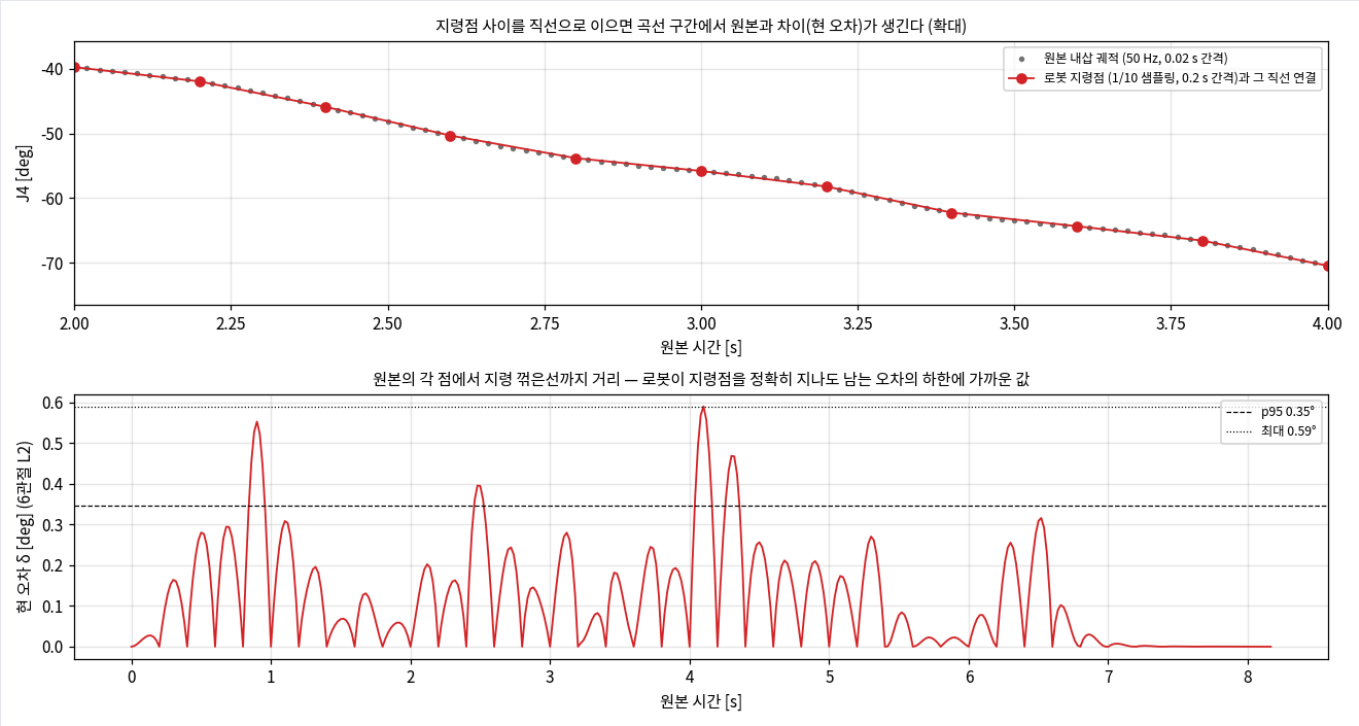
1.3 로봇 지령 궤적 생성 (서브샘플링)

- 50 Hz 원본 전량 전송 시 TCP 명령 과다 → `s` 번째 점마다 추출해 지령점으로 사용
- 이번 실험: `s = 10`, **0.2 s 간격**, 마지막 점은 항상 포함

$$\mathbf{c}_m = \mathbf{r}_{m \cdot s} \quad (m = 0, 1, \ldots), \quad \Delta t_{cmd} = \frac{s}{f_s} = \frac{10}{50} = 0.2\ s \quad \text{(last point always kept)}$$

- 현 오차: 지령점 직선 연결 경로와 원본 곡선의 차이
  - 로봇이 지령점을 정확히 지나도 남는 오차 → 추종 오차의 **하한에 가까운 값**

$$\delta_n = \min_m \operatorname{dist}(\mathbf{r}_n,\ [\mathbf{c}_m, \mathbf{c}_{m+1}]) \qquad \text{(chord error of the command polyline)}$$



지령점과 현 오차

궤적	원본 길이	원본 점 수 (50 Hz)	지령점 수 (0.2 s)	경로 길이 (6관절 L2)	현 오차 p95	현 오차 최대
A (`waypoints_sample`)	14.67 s	735	75	452.4°	0.41°	0.67°
B (`waypoints_robot1`)	8.16 s	410	42	172.0°	0.35°	0.59°
C (`waypoints_robot2`)	5.78 s	291	30	134.6°	0.36°	0.44°

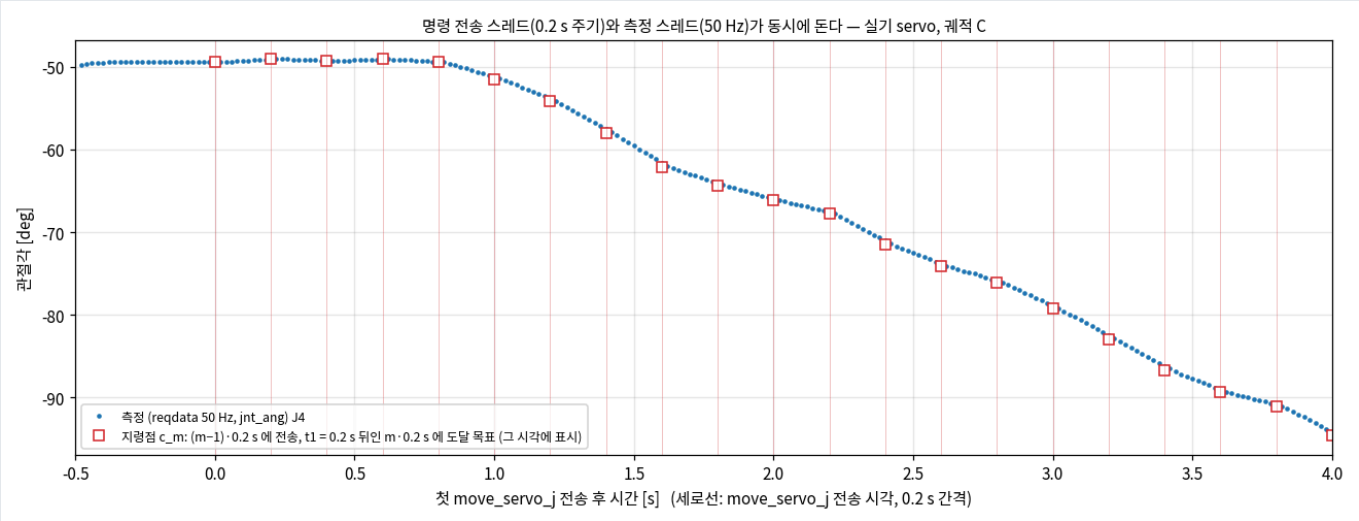
- 3장 측정 결과의 추종 오차 최대값 ≈ 위 표의 현 오차 최대 → **현재 오차의 대부분은 지령 간격에서 발생**

1.4 명령 전송과 동시 측정

- 명령 전송과 측정을 별도 TCP 연결·별도 스레드로 동시 수행

구분	포트	주기	내용
명령	5000	servo: 0.2 s 고정 주기 / jb2: 리스트 구성 후 실행 1회	텍스트 한 줄 = 명령 하나
측정	5001	50 Hz (`reqdata`)	관절각, 로봇 상태, 시뮬레이션/실기 모드

- 측정각 선택
  - 시뮬레이션: 엔코더 값 `jnt_ang` 가 0 으로 고정·미갱신 (실측 확인) → 지령 추종값 `jnt_ref` 사용
  - 실기: `jnt_ang` 사용
- 이전 명령 종료 대기
  - 직전 모션 종료 확인 후에만 다음 모션 명령 전송
  - 종료 기준: 측정각이 목표의 `reach_tol` 이내에 `reach_hold` 동안 유지 + `robot_state` idle(1)
  - 목표 미도달 상태로 정지 시 정체 시간 기준으로 중단
- 시작 자세
  - 추종 전 `move_j` 로 궤적 첫 점까지 이동 후 도달 확인
  - 해당 구간은 추종 평가에서 제외, 시작 각 오차에만 사용
- servo 전송 시점: 지령점 `c_m` 을 `(m−1)·0.2 s` 에 전송, `t1 = 0.2 s` 뒤 도달 목표



명령 전송과 측정

1.5 DTW 매칭쌍 생성

- 같은 시각끼리 비교하지 않는 이유
  - 경로를 정확히 지나가도 늦은 출발·속도 차이가 있으면 같은 시각의 원본 점과 큰 차이 발생
  - move_jb2 는 실행 시간이 원본의 1.2 ~ 2.8배로 변동 (accel 120)
  - 평가 목적은 "경로를 정확히 지나갔는가" → 측정 샘플마다 **원본 경로 위 대응점**을 먼저 결정
- 최근접 점만 찾지 않는 이유
  - 같은 자세를 두 번 지나는 경로에서 다른 시간대 점과 우연히 짝지어질 위험
  - → **시간 순서 보존 DTW (Dynamic Time Warping)** 사용: 지연·선행은 허용, 시간 역행은 불허
- 누적 비용 계산: 측정 샘플 1개 진행 시 원본 점 0 ~ K 개 진행 허용

$$D(i,k) = C(i,k) + \min_{k-K \le k' \le k} D(i-1,k'), \qquad C(i,k) = \|\mathbf{q}_i - \mathbf{r}_k\|_2$$

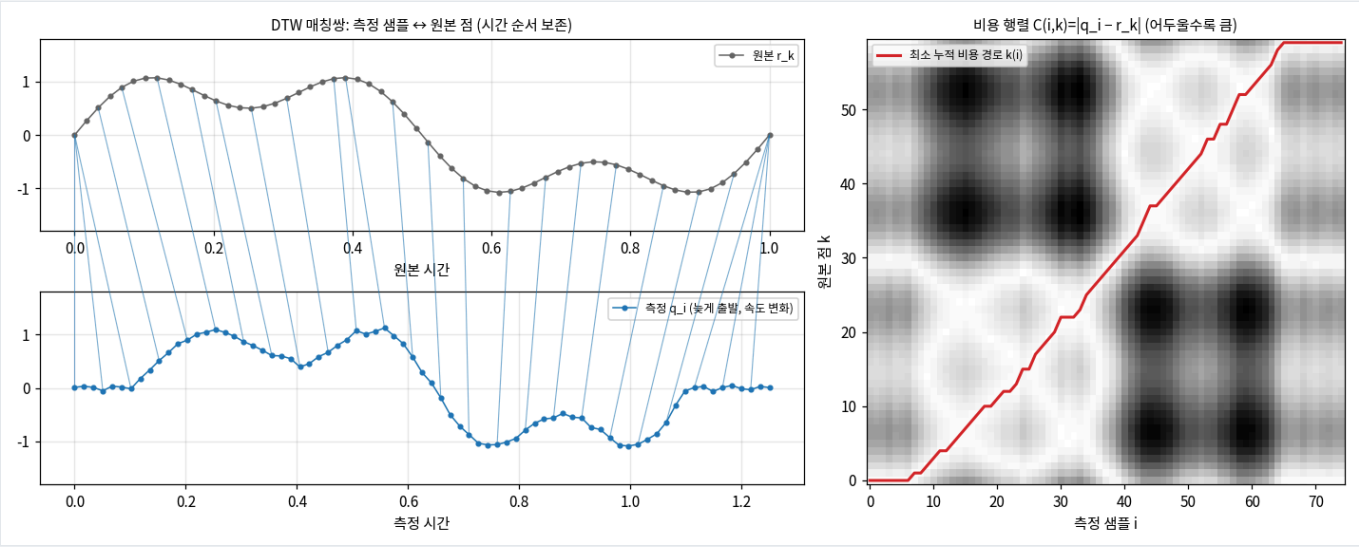
$$k(1) \le k(2) \le \cdots \le k(N), \qquad k(1) < \frac{M}{20}, \quad k(N) > M - \frac{M}{20}, \qquad K = \max\left(30,\ \frac{M}{5}\right)$$

- 제약과 보정
  - 짝 순서 비감소(역행 없음), 시작은 원본 앞쪽 5 %, 끝은 뒤쪽 5 % 안에서 선택
  - DTW 결과(원본 50 Hz 정점)를 양옆 선분 위로 재투영 → 정점 간격만큼의 양자화 오차 제거 (`refine_pairs`)
  - 측정 샘플 수와 원본 점 수가 달라도 적용 가능 (건너뛰기, 다대일 매칭 허용)
- 예시 코드: `analysis.ordered_match` 를 풀어 쓴 형태 (실제 코드는 동일 계산의 벡터화 버전)

```
import numpy as np
from scipy.spatial.distance import cdist

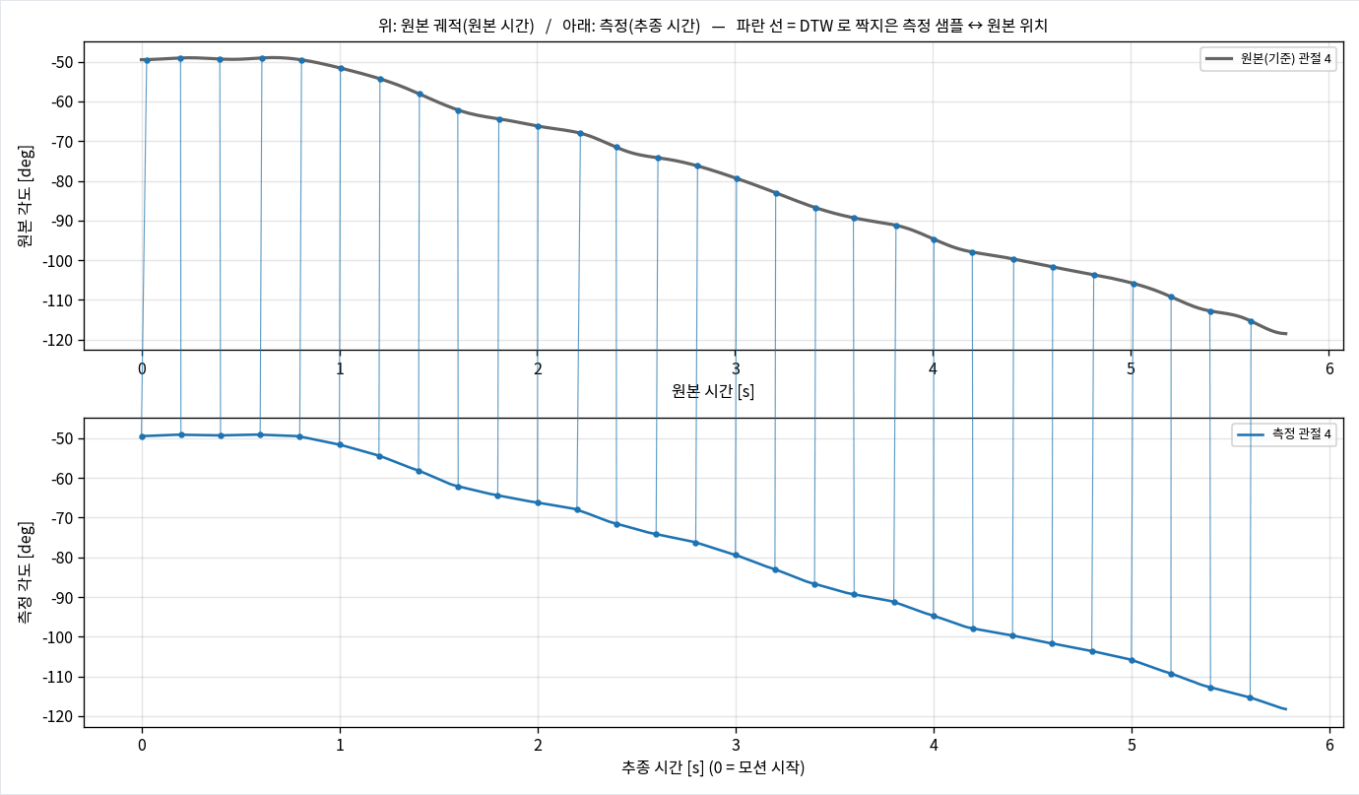
def ordered_match(meas, ref, K=None):
    """meas: (N,6) 측정 관절각, ref: (M,6) 원본 궤적.
    반환: 측정 샘플 i 에 짝지은 원본 점 번호 k(i) (감소하지 않음)."""
    N, M = len(meas), len(ref)
    K = K or max(30, M // 5)              # 측정 1샘플 동안 원본이 진행할 수 있는 최대 점 수
    C = cdist(meas, ref)                  # C[i, k] = ||q_i - r_k||
    D = np.full((N, M), np.inf)
    head = max(5, M // 20)
    D[0, :head] = C[0, :head]             # 시작은 원본 앞쪽에서만
    for i in range(1, N):
        for k in range(M):
            D[i, k] = C[i, k] + D[i - 1, max(0, k - K):k + 1].min()
    tail = max(5, M // 20)
    k = M - tail + int(np.argmin(D[-1, M - tail:]))   # 끝은 원본 뒤쪽에서만
    idx = [k]
    for i in range(N - 1, 0, -1):                      # 역추적
        lo = max(0, k - K)
        k = lo + int(np.argmin(D[i - 1, lo:k + 1]))
        idx.append(k)
    return np.array(idx[::-1])

# 사용 예: 늦게 출발하고 중간 속도가 다른 측정 신호
tr = np.linspace(0, 1, 60);  r = np.sin(2*np.pi*tr) + 0.5*np.sin(6*np.pi*tr)
tm = np.linspace(0, 1.25, 75); q = np.interp(np.clip((tm - 0.1) / 1.0, 0, 1) ** 1.15, tr, r)
k = ordered_match(q[:, None], r[:, None], K=8)
error = q - r[k]                          # 매칭쌍 기준 오차 (시간 어긋남은 포함되지 않음)
```



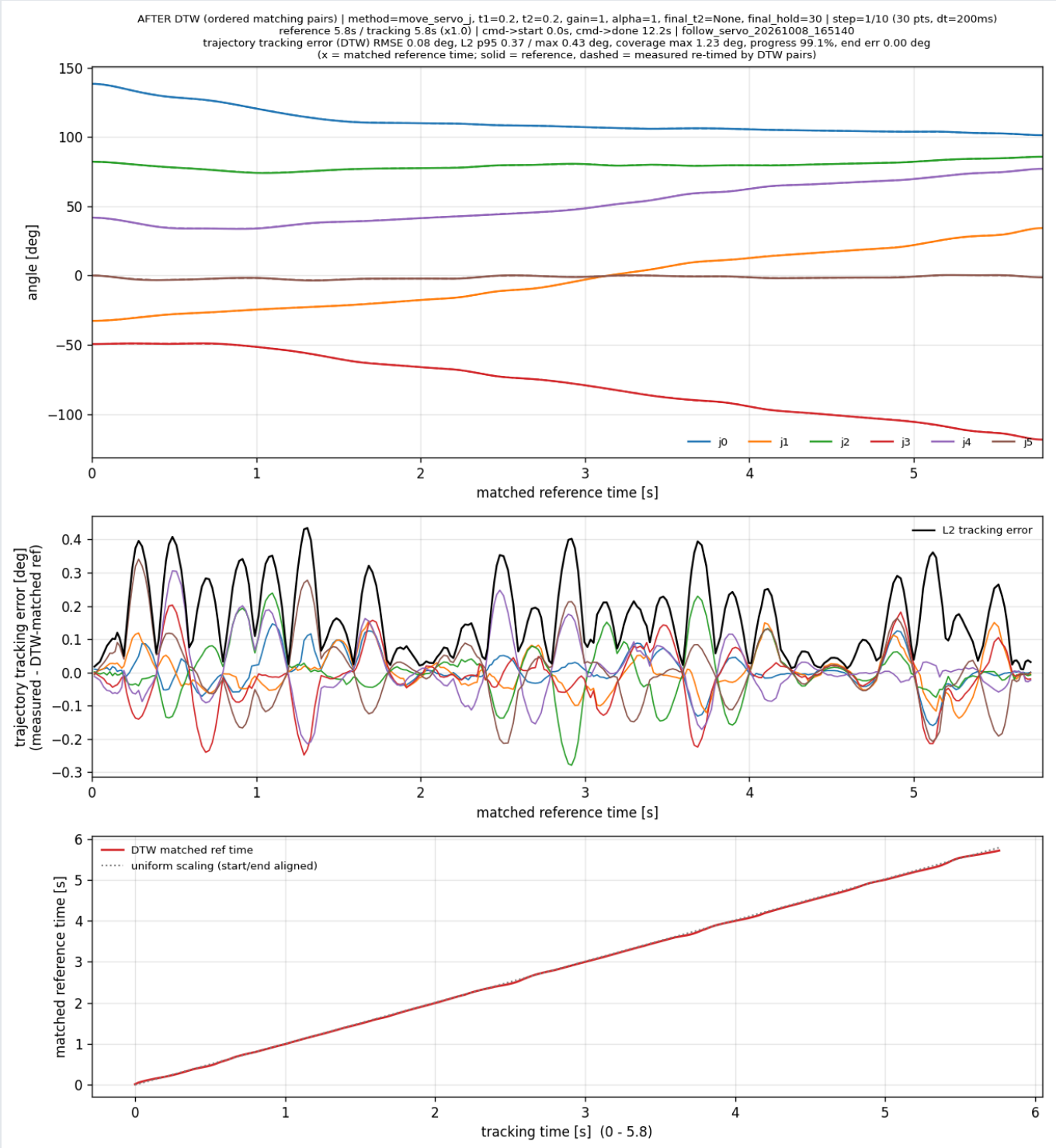
DTW 예시

- 예시 그림 해석
  - 오른쪽 빨간 경로: 누적 비용 최소 짝
  - 늦게 출발한 앞부분: 경로 수평 유지 (원본 첫 점에 계속 매칭)
  - 이후: 원본을 따라 대각선 진행
- 실제 실행(궤적 C, servo) 매칭: 아래 측정 샘플과 위 원본 위치를 파란 선으로 연결



실행 데이터의 DTW 매칭

- 실행마다 저장되는 비교 그림 (실기 servo, 궤적 C)
  - 맨 아래: 측정 시간 대 매칭된 원본 시간
  - 점선(시작·끝만 맞춘 균일 비례)과 거의 겹침 → 시간 왜곡 작음



매칭 후 비교 그림

1.6 실기 / 시뮬레이션 모드 평가 절차

항목	시뮬레이션 모드	실기 모드
모드 전환 명령	`pgmode simulation`	`pgmode real`
로봇 동작	움직이지 않음, 제어기 내부 지령값만 갱신	실제 구동
측정각	`jnt_ref` (엔코더 값은 0 고정)	`jnt_ang` (엔코더)
진입 조건 (시험 도구)	기본값	확인 창에서 E-Stop 준비 확인 + `REAL` 직접 입력, 로봇 idle 상태
실행 중 감시	측정 스레드가 50 Hz 로 모드 값 확인, 변경 시 즉시 중단	동일

항목	시뮬레이션 모드	실기 모드
종료 후	-	시뮬레이션 모드 복귀 (선택), 실기 설정 미저장

- 시험 도구 "중지" 버튼 = `task stop` 전송일 뿐 **비상정지 아님** → 실기 시험 중 물리 E-Stop 을 손 닿는 곳에 배치
- 동일 궤적·파라미터로 시뮬레이션과 실기를 각각 수행해 같은 지표로 비교 (3.6절)

1.7 평가 지표: 추종 오차와 추종 안정성

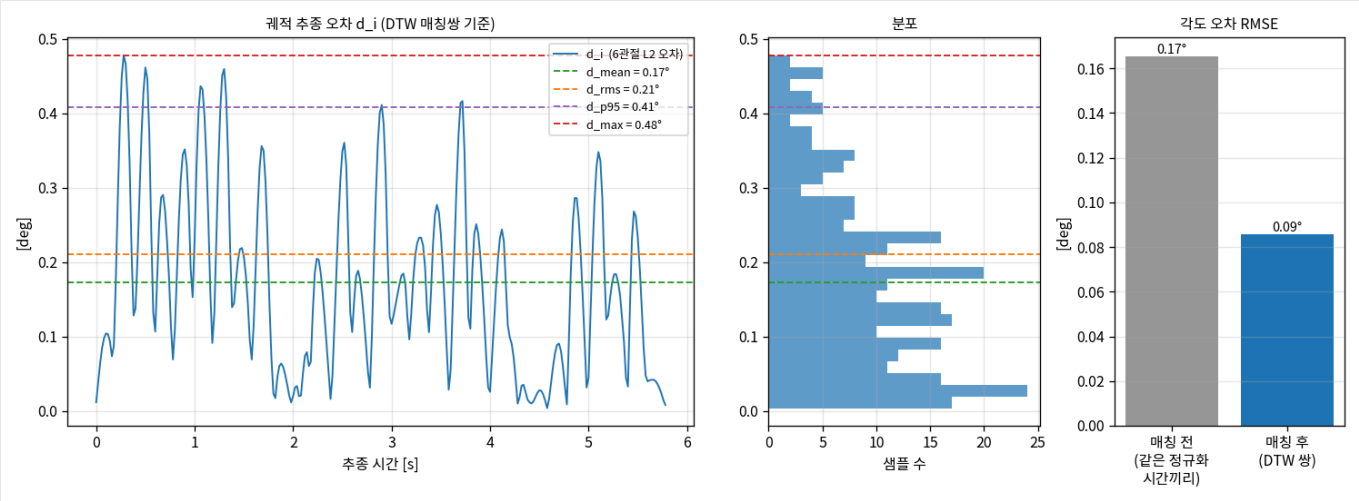
- 보고서에 사용한 지표만 정리 (전체 정의와 설명 그래프는 `docs/metrics.md`)

(1) 궤적 추종 오차

- 정의: 측정 샘플 `q_i` 와 DTW 로 짝지은 원본 위 점 `p_i` 의 차이, 6관절 벡터 L2 크기 (deg)

$$\mathbf{e}_i = \mathbf{q}_i - \mathbf{p}_i, \qquad d_i = \|\mathbf{e}_i\|_2 = \sqrt{\sum_{j=1}^{6} e_{ij}^2}$$

$$d_{mean} = \frac{1}{N}\sum_i d_i, \quad d_{rms} = \sqrt{\frac{1}{N}\sum_i d_i^2}, \quad d_{p95} = P_{95}(d_i), \quad d_{max} = \max_i d_i$$



궤적 추종 오차 그래프

- 판정 기준: `d_p95` $\le$ `1.0°`, `d_max` $\le$ `2.0°`
- 보조 지표
  - 커버리지: 원본 각 점에서 측정 경로까지 거리 → 경로 건너뜀 검출
  - 진행도: 원본 경로를 어디까지 지나갔는지

$$c_k = \min_{\ell \in W(k)} \operatorname{dist}(\mathbf{r}_k,\ [\mathbf{q}_\ell, \mathbf{q}_{\ell+1}]), \quad c_{mean},\ c_{p95},\ c_{max}$$

$$\text{progress} = \max\{\, p_i :\ d_i \le d_{th} \,\}, \qquad p_i = \frac{\text{arc length up to matched point}}{L}$$

(2) 시작 / 종료 각 오차

- 추종 오차와 별도로, 정지 자세와 목표의 차이를 평가
- 정의: 추종 끝 안정 구간 중앙값과 원본 마지막 점의 관절별 차이 중 최댓값

$$e_{end} = \max_j \left| \operatorname{median}_{t \ge t_{last} - \max(t_{hold},\, 0.3)} q_j(t) - r_{N,j} \right|$$

(3) 추종 안정성 (매끄러움)

- 원칙: **경로 자체의 꺾임·비매끄러움은 감점하지 않음** → 경로 모양과 무관한 양만 사용

- 준비: 경로를 따라간 속력(진행도의 시간 변화율)과 그 느린 추세 산출

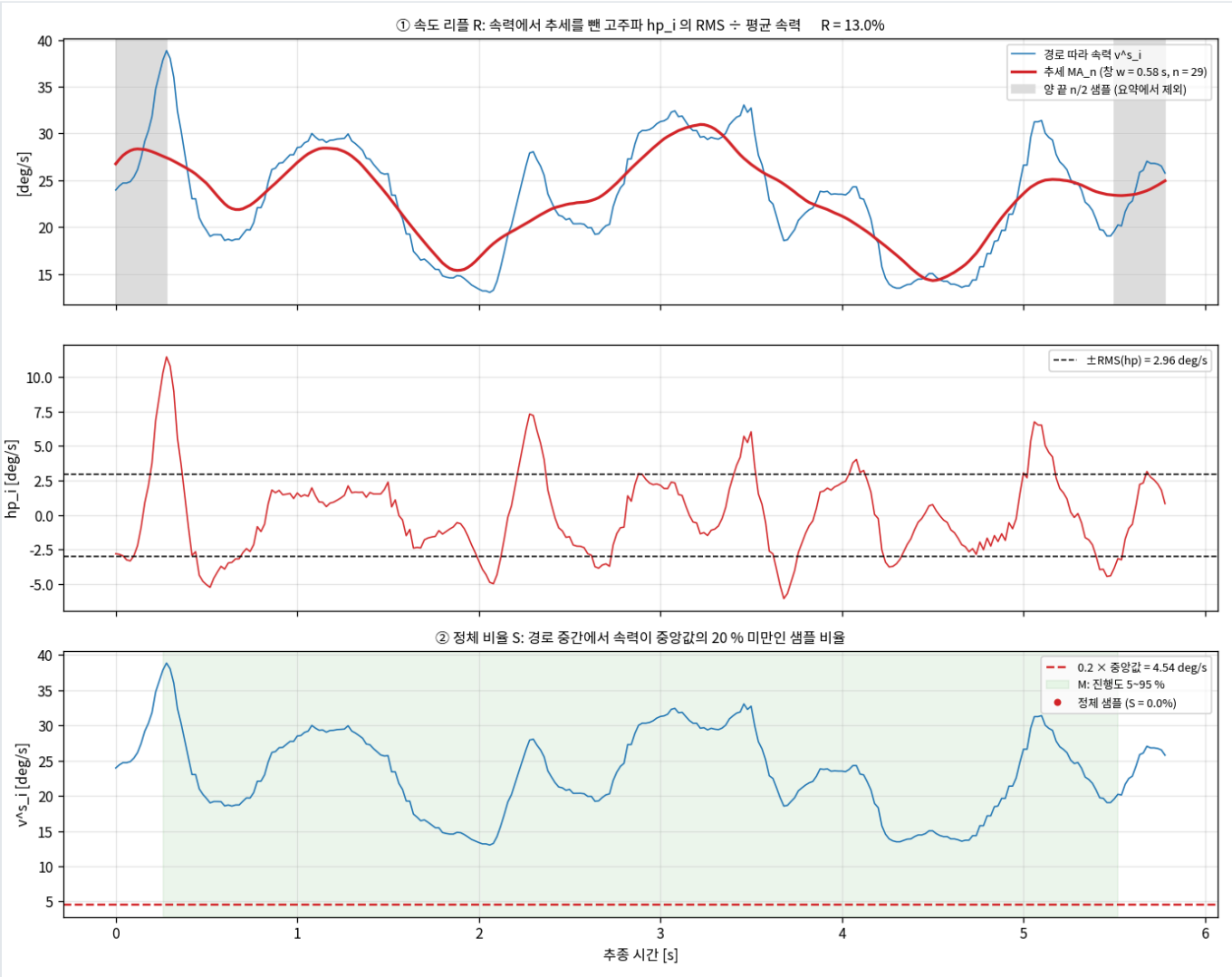
$$s_i = p_i \cdot L, \qquad v_i^s = \frac{s_h - s_l}{t_h - t_l} \quad (\text{same } \pm 0.1\ \text{s window})$$

$$w = \text{clip}\left(\frac{T_{track}}{10},\ 0.3,\ 2.0\right)\ [s], \qquad n = \max\left(\text{round}\left(\frac{w}{\Delta t}\right),\ 3\right)$$

$$hp_i = v_i^s - \text{MA}_n(v^s)_i, \qquad \mathbf{e}_i^{hp} = \mathbf{e}_i - \text{MA}_n(\mathbf{e})_i$$

- **속도 리플** `R` (참고 지표, 판정에 미사용)
  - 경로 속력의 추세 둘레 요동 크기 / 평균 속력
  - 가다 서다 반복 시 증가
  - 명령 방식 간 트레이드오프 비교에 사용 (3.5절)

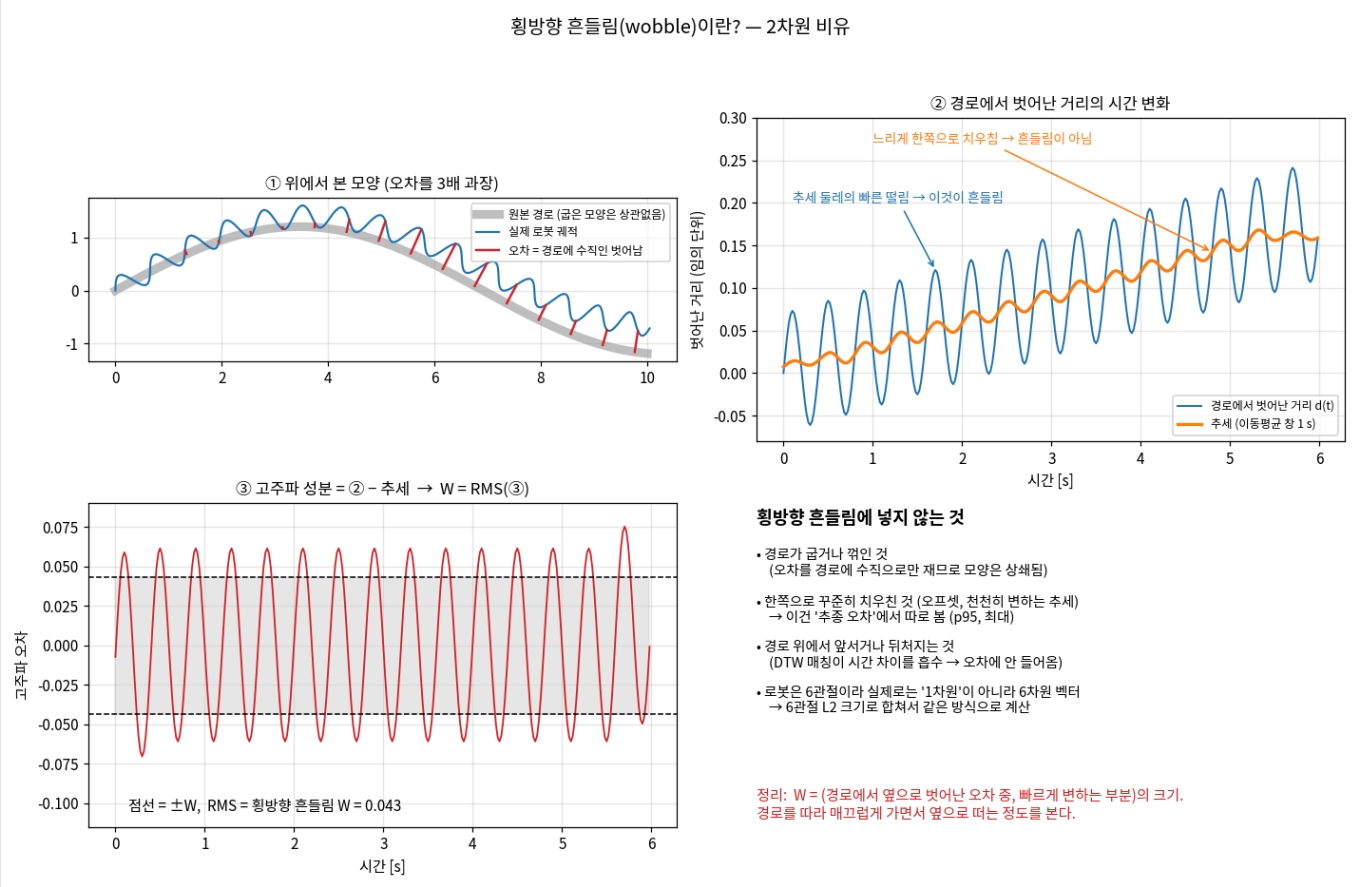
$$R = \frac{\sqrt{\text{mean}_{i \in core}\ hp_i^2}}{\text{mean}_{i \in core}\ v_i^s}$$



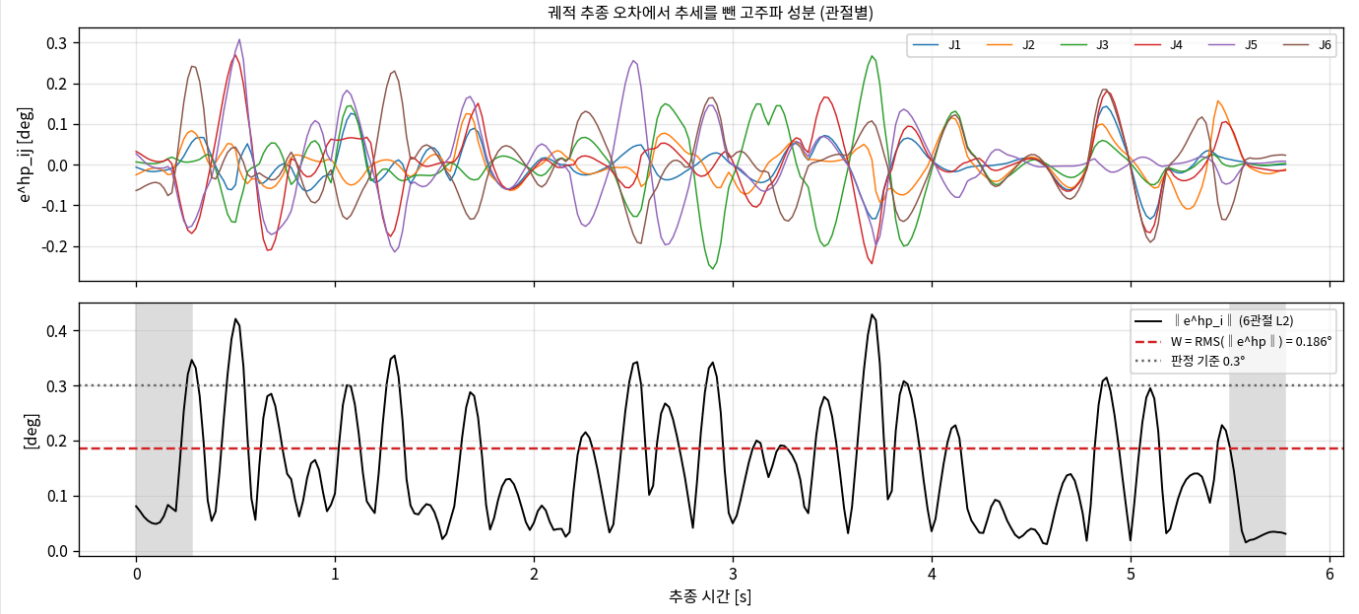
속도 리플 그래프

- **횡방향 흔들림** `W`
  - 경로 수직 방향 벗어남 중 빠르게 떠는 성분의 RMS
  - 한쪽으로 꾸준히 치우친 성분은 추세로 분류 → `W` 가 아닌 추종 오차에 반영
  - 기준 `W ≤ 0.3°`

$$W = \sqrt{\mathrm{mean}_{i \in core} \left\| \mathbf{e}_i^{hp} \right\|_2^2} \quad [\mathrm{deg}]$$



횡방향 흔들림 개념

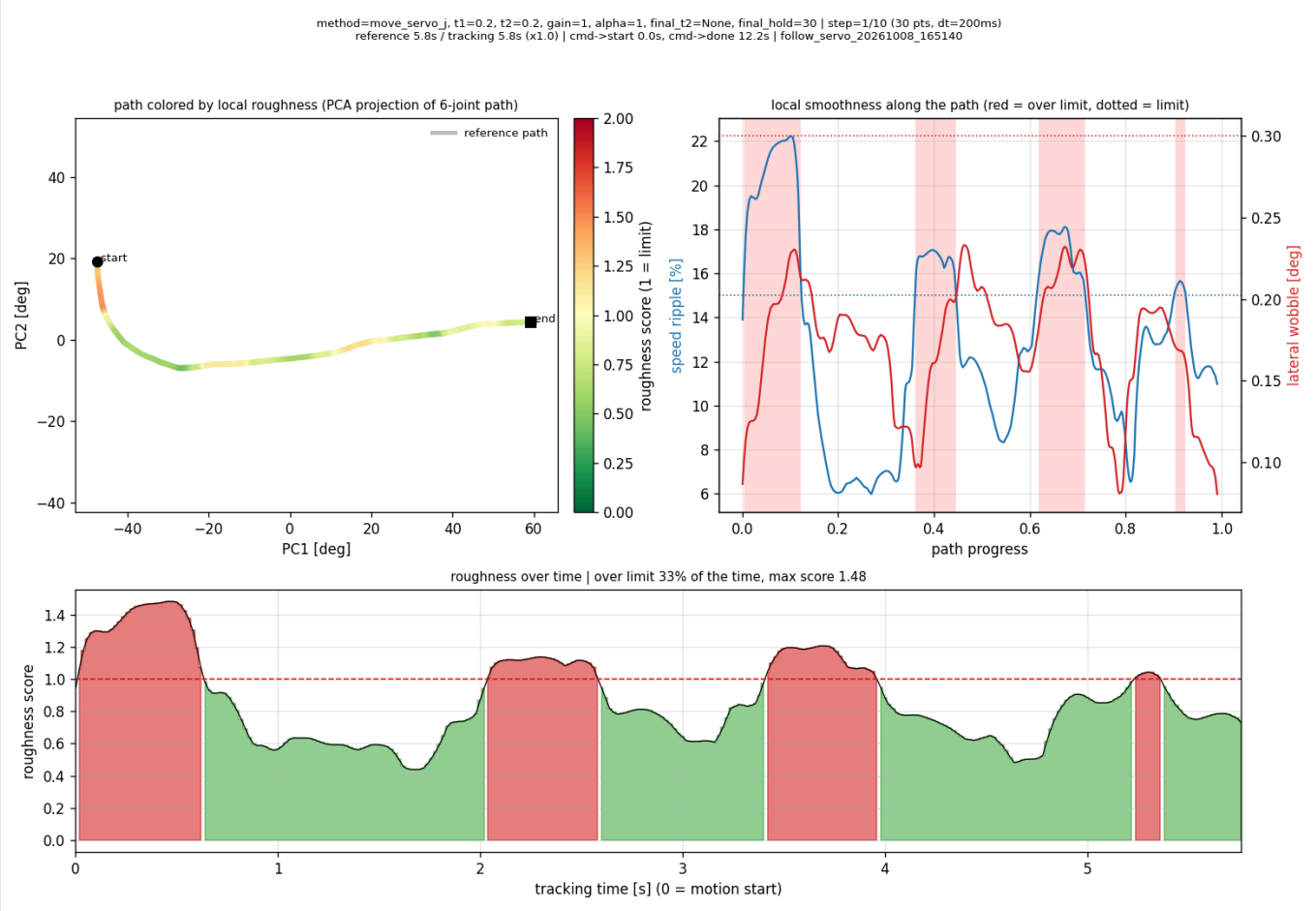


횡방향 흔들림 실측

- **정체 비율** S
  - 경로 중간(진행도 5 ~ 95 %)에서 속력이 중앙값의 20 % 미만인 샘플 비율
  - 멈칫거림 검출
  - 기준 $S \leq 2\ \%$

$$S = \frac{\#\{\, i \in \mathcal{M} :\ v_i^s < 0.2 \cdot \mathrm{median}(v_{\mathcal{M}}^s) \}}{|\mathcal{M}|}, \qquad \mathcal{M} = \{\, i :\ 0.05 < p_i < 0.95 \,\}$$

- 매끄러움 지도: 같은 지표를 샘플별 이동 창으로 계산, 거친 구간을 색으로 표시 (실기 servo, 궤적 C)



매끄러움 지도

(4) 판정 기준

- 8개 항목 전부 통과 시 PASS: 추종 오차 p95, 최대, 커버리지, 진행도, 샘플 간 이동, 시작/종료 각 오차, 흔들림, 정체
- 속도 리플은 판정에서 제외
  - 리플은 명령 방식(blend 등)에 따라 구조적으로 정해지는 값 → 합격/불합격이 아닌 트레이드오프 비교 대상
- **합격선은 응용 요구가 아닌 임의 초기값**

2. 로봇 인터페이스: 외부 스크립트와 UI Script

- 참고 문서: 외부 스크립트 API, UI Script

2.1 구성

포트	역할	형식
5000 (명령)	UI Script 명령, 모드 전환, task 제어	텍스트 한 줄(`\n` 종료) = 명령 하나. 응답 `The command was executed` 또는 `The command is not allowed`
5001 (데이터)	로봇 상태 요청	`reqdata` 전송 시 고정 길이 이진 패킷 응답

- 명령은 수신 즉시 실행, 별도 확인 절차 없음 → 전송 전 모드·목표값 확인 필요
- 연속 전송 시 구문 오류 사례
  - 응답 대기 없이 여러 줄을 몰아 보내면 TCP 가 줄을 합치거나 분할
  - 태블릿 표시: `syntax error at ' ' in this line: move_jb2_add(j`
  - 대응: `TCP_NODELAY` 설정, 리스트 구성 명령은 한 줄마다 응답 대기 후 다음 줄 전송

2.2 상태 데이터 (reqdata)

- 형식: 리틀엔디안 580바이트, 오프셋은 바이트 단위
- 헤더 확인: 첫 바이트 `0x24`, 네 번째 바이트 `0x03`
- 본 프로젝트 사용 필드

오프셋	형식	필드	사용
4	float	제어기 시간	기록
8	float × 6	`jnt_ref` 관절 지령값 [deg]	시뮬레이션 측정각
32	float × 6	`jnt_ang` 관절 엔코더값 [deg]	실기 측정각 (시뮬레이션에서는 0)
104	float × 6	`tcp_pos`	미사용 (URDF 검증에 활용 가능, 미확인)
340	int	`robot_state` (1 = idle, 3 = 실행 중)	모션 종료 확인
404	int	`real_vs_simulation_mode` (1 = 시뮬레이션)	모드 감시

2.3 UI Script 문법 예시

- 관절값 전달: 임시 변수 `jnt[j0, j1, j2, j3, j4, j5]` (deg), 별도 선언 불필요
- 예시 관절값: 궤적 B 의 첫 지령점들

모드 전환

```
pgmode simulation
pgmode real
```

관절 이동 `move_j(J, 속도 deg/s, 가속도 deg/s², 대기 0/1)` — 대기 기본값 1

```
move_j(jnt[0,0,0,0,0,0], 20, 40)
move_j(jnt[66.7570,-42.4990,49.9020,-6.5370,113.2200,0.3630], 50, 120)
```

블렌딩 경로 `move_jb2_*` — 리스트에 여러 점 적재 후 일괄 실행 (시간 기반 블렌딩)

```
move_jb2_clear()
move_jb2_add(jnt[66.9632,-42.2424,51.4177,-9.6759,112.9050,0.2800], 50, 120, 0, 100)
move_jb2_add(jnt[67.0438,-41.5110,53.5442,-14.9398,111.9343,0.1912], 50, 120, 0, 100)
...
move_jb2_run(0)
```

- `move_jb2_add(J, 속도, 가속도, 0, 블렌드 비율 %)` : 네 번째 인자는 더미
- `move_jb2_run(대기)` : 0 이면 실행 시작 후 즉시 반환

서보 `move_servo_j(J, t1, t2, gain, alpha)` — 목표 관절각 실시간 갱신

```
move_servo_j(jnt[66.9632,-42.2424,51.4177,-9.6759,112.9050,0.2800], 0.2, 0.2, 1, 1)
move_servo_j(jnt[67.0438,-41.5110,53.5442,-14.9398,111.9343,0.1912], 0.2, 0.2, 1, 1)
...   (0.2 s 마다 다음 점 전송)
```

작업(task) 제어

```
task load my_task
task repeat 1
task play once
task pause
task resume_a
task stop
```

터미널 직접 전송 (시뮬레이션 모드에서 시험)

```
# 명령 포트: 한 줄씩 입력하면 응답이 돌아온다
nc 10.0.2.7 5000
pgmode simulation
move_j(jnt[0,0,0,0,0,0], 20, 40)

# 데이터 포트: 580바이트 이진 응답 (16진수로 확인)
printf 'reqdata\n' | nc -q 1 10.0.2.7 5001 | xxd | head
```

Python 최소 예시

```
import socket, struct
cmd = socket.create_connection(("10.0.2.7", 5000))
dat = socket.create_connection(("10.0.2.7", 5001))
cmd.sendall(b"pgmode simulation\n"); print(cmd.recv(256))
cmd.sendall(b"move_j(jnt[0,0,0,0,0,0], 20, 40)\n"); print(cmd.recv(256))

dat.sendall(b"reqdata\n")
buf = b""
while len(buf) < 580:
    buf += dat.recv(580 - len(buf))
jnt_ref = struct.unpack_from("<6f", buf, 8)
jnt_ang = struct.unpack_from("<6f", buf, 32)
robot_state, = struct.unpack_from("<i", buf, 340)
is_sim, = struct.unpack_from("<i", buf, 404)
```

2.4 사용 시 주의사항

- 명령은 확인 없이 즉시 실행, `pgmode real` 상태의 모션 명령은 실제 로봇 구동
- `task stop` 은 작업 정지 요청이며 비상정지 아님
- 시뮬레이션에서 `jnt_ang` = 0 → 도달 판단에 사용 금지 (미도달 상태를 도달로 판단해 다음 명령이 나간 사례 있음)
- `move_servo_j` 문서상 범위: `t1` ≥ `0.002 s`, `0.02 < t2 < 0.2 s`, `gain > 0`, `0 < alpha < 1`
  - 이번 실험 `t2 = 0.2`, `alpha = 1.0` 은 **문서 범위의 경계값**
  - 제어기 거부는 없었으나 문서상 열린 구간 → 범위 내 값(예: `t2 = 0.1`, `alpha = 0.9`)으로 재확인 필요
- `t1` 보다 짧은 간격의 서보 명령 중첩 동작은 문서에 없음 → 이번 실험은 명령 주기 = `t1` = 0.2 s

3. 실기 및 시뮬레이션 성능 평가

3.1 실험 조건과 비교 대상

항목	값
로봇 / URDF	rb20_1900es (RT) / `urdf/rb20_1900es_RT.urdf` (링크 표면 변위 계산에 사용)
궤적	A `waypoints_sample` (14.67 s), B `waypoints_robot1` (8.16 s), C `waypoints_robot2` (5.78 s)
내삽	시간 부여 `v_max = 20 deg/s` , 3차 스플라인 50 Hz
지령 간격	1/10 서브샘플링 → 0.2 s
측정	`reqdata` 50 Hz

- 비교 대상: **궤적 3종 모두를 시뮬레이션과 실기로 수행한 조건만** 사용 (총 24회)

명령 방식	파라미터	궤적	모드	횟수
jb2 blend 0	speed 50, accel 120, blend 0	A, B, C	시뮬 / 실기	6
jb2 blend 50	speed 50, accel 120, blend 50	A, B, C	시뮬 / 실기	6
jb2 blend 100	speed 50, accel 120, blend 100	A, B, C	시뮬 / 실기	6
servo	t1 = t2 = 0.2, gain 1, alpha 1, 마지막 점 30회 반복	A, B, C	시뮬 / 실기	6

- 제외 조건 (궤적·모드 일부만 수행 → 비교 불가)
  - jb2 accel 500 (시뮬레이션 위주), accel 180 / 200 (실기 궤적 B 만)
  - servo gain 0.2, alpha 0.5
  - 분석 결과가 없는 실기 1회 (`164030`)
- 원본 궤적은 시간 간격을 요구하지 않고 **순서만 보존**하면 됨 → 실행 시간은 평가하지 않음 (표의 추종 시간은 참고값)

3.2 명령어와 파라미터

- 시작 자세 이동: 두 방식 모두 `move_j(c_0, speed, accel)` 로 동일

move_jb2 (블렌딩 관절 이동)

```
move_jb2_clear()
move_jb2_add(c_1, 50, 120, 0, blend)   # c_1 ~ c_M 까지 반복, 한 줄마다 응답 대기
move_jb2_run(0)
```

파라미터	의미	값
`speed`	관절 속도 [deg/s] (상한)	50
`accel`	관절 가속도 [deg/s²]	120
`blend`	이웃 점과의 블렌딩 비율 [%]. 0 이면 점마다 정지	0, 50, 100

- 시간 지정 없음 → 실행 속도는 속도·가속도·블렌드로 결정
- 지령점 간격이 촘촘해 `speed` 도달 전 감속 필요
  - 실제 속력 상한 ≈ $\sqrt{(\text{accel} \times \text{점 사이 거리})}$
  - 궤적 B (점 사이 평균 4.2°), accel 120 → 약 22 deg/s, 측정 최고 속력(약 20 deg/s)과 유사 (3.3절 그림)
- 리스트 구성(`move_jb2_add` 반복): 점당 약 10 ms (응답 대기 포함, 궤적 A 75점 기준)

move_servo_j (실시간 서보)

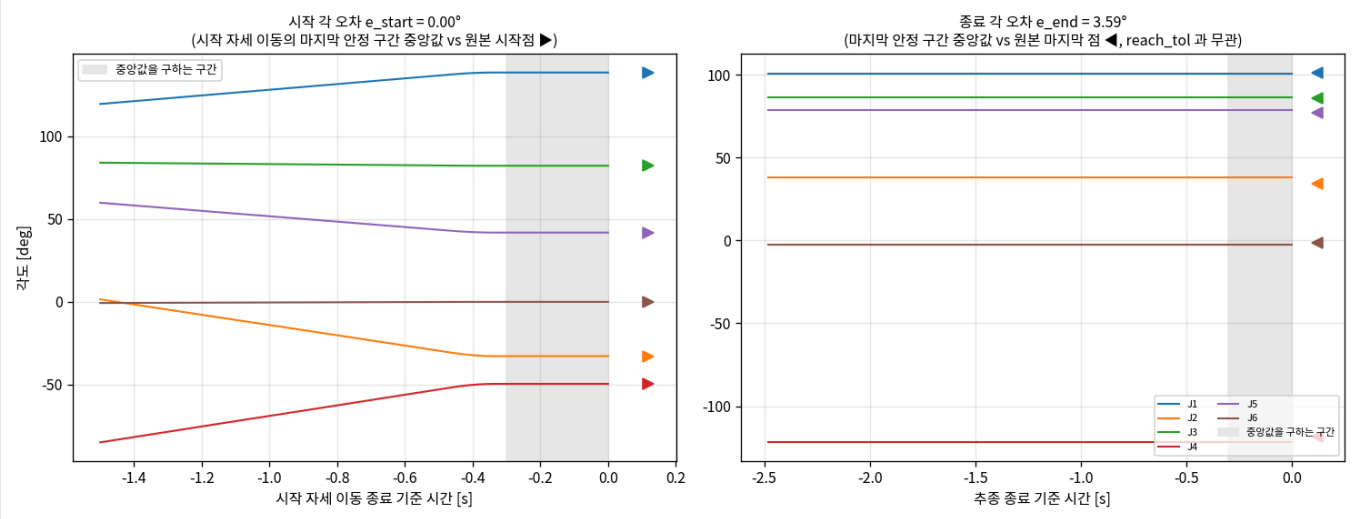
```
move_servo_j(c_m, 0.2, 0.2, 1, 1)      # m = 1 .. M, 0.2 s 마다 전송
move_servo_j(c_M, 0.2, 0.2, 1, 1)      # 마지막 점을 30 번 더 반복
```

파라미터	문서상 의미	값
`t1`	도달 시간 [s] (≥ 0.002)	0.2 (= 명령 주기)
`t2`	유지 시간 [s] (0.02 < t2 < 0.2)	0.2
`gain`	속도 추종 비율 (> 0)	1.0
`alpha`	저역 통과 필터 (0 < alpha < 1)	1.0
`final_hold`	(시험 도구) 마지막 점 반복 횟수	30

- 지령마다 도달 시간 지정 → 명령 주기(0.2 s)에 따라 진행
- 이 절과 3.3 ~ 3.7절의 servo 는 **원본 시간 간격(0.2 s)으로 지령점을 보내는 방식**(`original`)이다. 속도·가속도로 프로파일을 만들어 20 ms 주기로 보내는 방식은 3.8절
- 종료 오버슈트 (문서에 없는 동작, 관측 결과)
  - 마지막 지령 후 마지막 지령 속도 × `t2` 만큼 더 진행 후 정지
  - 대응: 마지막 점 반복 전송으로 마지막 지령 속도를 0 으로 만듦

$$\Delta q_{over} \approx \dot{q}_{last} \cdot t_2 \qquad \text{(observed; not in the manual)}$$

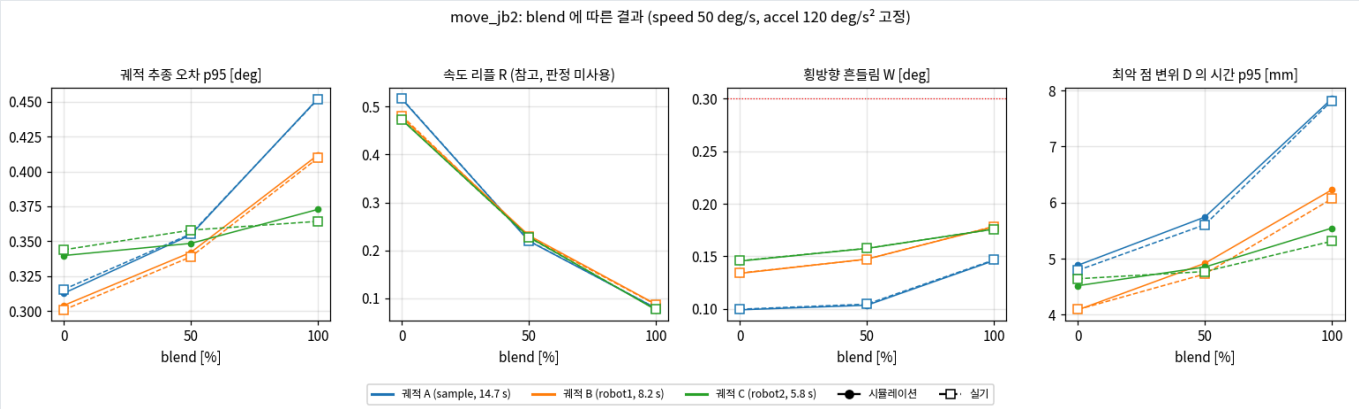
- 효과
  - 마지막 점 미반복 이전 실행(`20261008_132802`): 종료 각 오차 3.59°
  - `final_hold = 30` 적용 이번 실험: 전부 0.00°



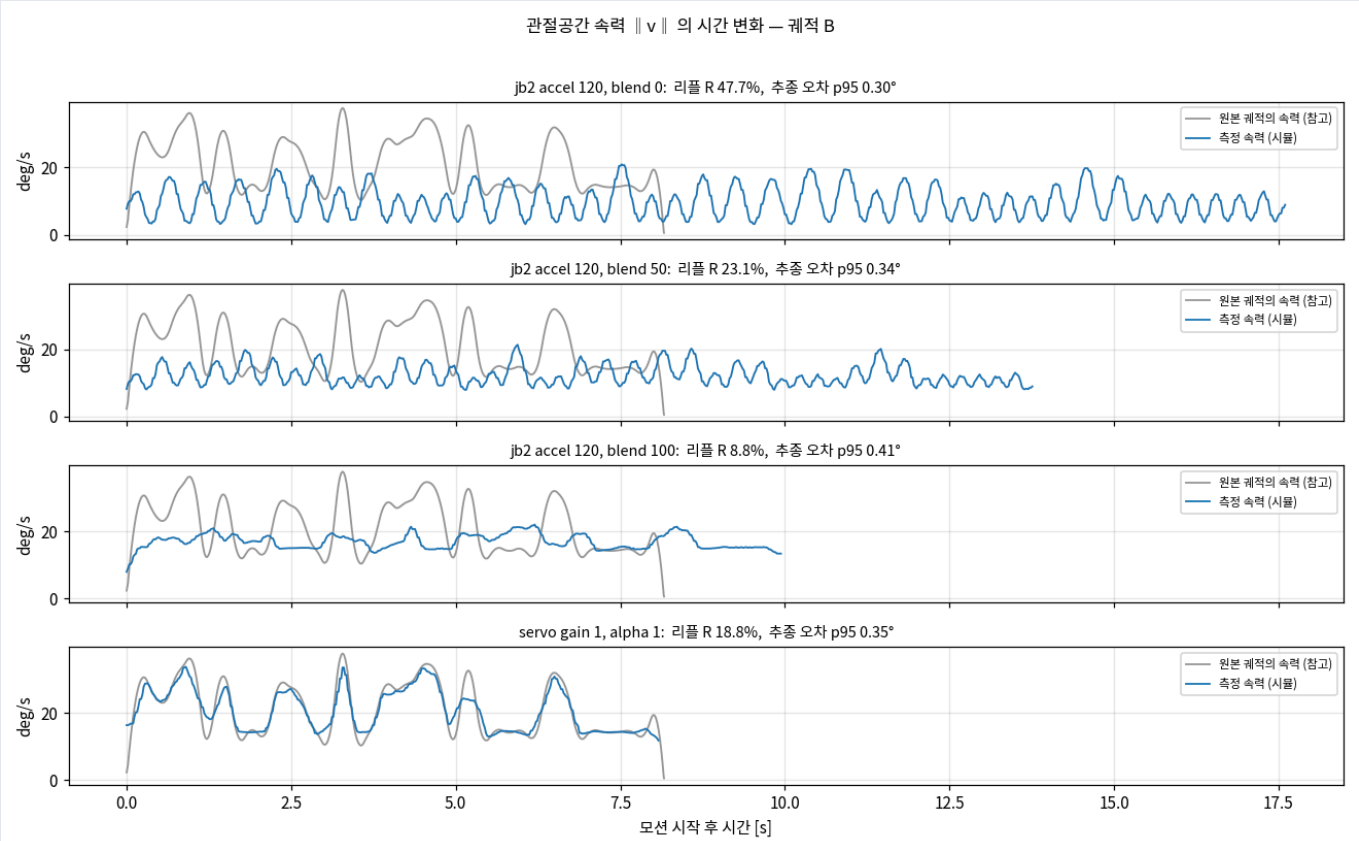
종료 오버슈트 사례

3.3 move_jb2 결과

- 조건: speed 50 deg/s, accel 120 deg/s² 고정, blend 0 / 50 / 100



jb2 결과



속력 프로파일

- 표: 시뮬 / 실기 (같으면 한 값), `D` 는 최악 점 변위 (rb20_1900es_RT)

궤적	blend	추종 [s] (참고)	d_p95 [°]	d_max [°]	리플 R	흔들림 W [°]	D 시간 평균 [mm]	D 시간 p95 [mm]	D 시간 최대 [mm]
A	0	41.5	0.31 / 0.32	0.66	51.5 / 51.6 %	0.10	1.36 / 1.39	4.9 / 4.8	13.7 / 13.8
A	50	32.1	0.35 / 0.36	0.66	22.0 %	0.10	1.76 / 1.79	5.7 / 5.6	14.8 / 14.3
A	100	22.7	0.45	0.67 / 0.66	8.0 %	0.15	2.58 / 2.61	7.9 / 7.8	15.5 / 13.7
B	0	17.6	0.30	0.59	47.7 / 48.1 %	0.13	1.15 / 1.19	4.1	10.0 / 10.3
B	50	13.8	0.34	0.58	23.1 / 22.8 %	0.15	1.46 / 1.49	4.9 / 4.7	10.0 / 9.7
B	100	9.9	0.41	0.58	8.8 / 8.7 %	0.18	1.99 / 2.06	6.2 / 6.1	9.8 / 10.4

궤적	blend	추종 [s] (참고)	d_p95 [°]	d_max [°]	리플 R	흔들림 W [°]	D 시간 평균 [mm]	D 시간 p95 [mm]	D 시간 최대 [mm]
C	0	14.6	0.34	0.44	47.2 / 47.3 %	0.15	1.47 / 1.50	4.5 / 4.6	8.6 / 8.0
C	50	11.5	0.35 / 0.36	0.43 / 0.44	22.8 / 22.7 %	0.16	1.85 / 1.87	4.9 / 4.8	8.6
C	100	8.4	0.37 / 0.36	0.43 / 0.44	7.6 / 7.7 %	0.18	2.49	5.5 / 5.3	8.5 / 8.4

해석

- blend 0
  - 지령점마다 감속·정지 → 속력 톱니 형태, 리플 47 ~ 52 %
  - 지령점 직선 연결 경로를 따라감 → 추종 오차 p95 최소 (0.30 ~ 0.34°), 표면 변위 평균·p95 도 최소
- blend 100
  - 속력 균일 유지 → 리플 8 ~ 9 %
  - 지령점 근처 모서리 깎임 → 추종 오차 p95 가 blend 0 대비 0.03 ~ 0.14° 증가 (궤적 A 에서 최대), 표면 변위 평균 약 1.7 ~ 1.9배
- blend 50: 오차·리플 모두 중간
- 추종 오차 최대 `d_max`
  - blend 와 무관하게 궤적별로 거의 일정 (A 0.66, B 0.58, C 0.43 ~ 0.44°)
  - 1.3절 현 오차 최대(0.67 / 0.59 / 0.44°)와 동일 → 지령 간격에서 기인
- 횡방향 흔들림 `W`: 0.10 ~ 0.18°, 전부 기준(0.3°) 이내, blend 가 클수록 약간 증가

3.4 move_servo_j 결과

- 조건: `t1 = t2 = 0.2 s`, gain 1, alpha 1, 마지막 점 30회 반복
- 표: 시뮬 / 실기 (같으면 한 값)

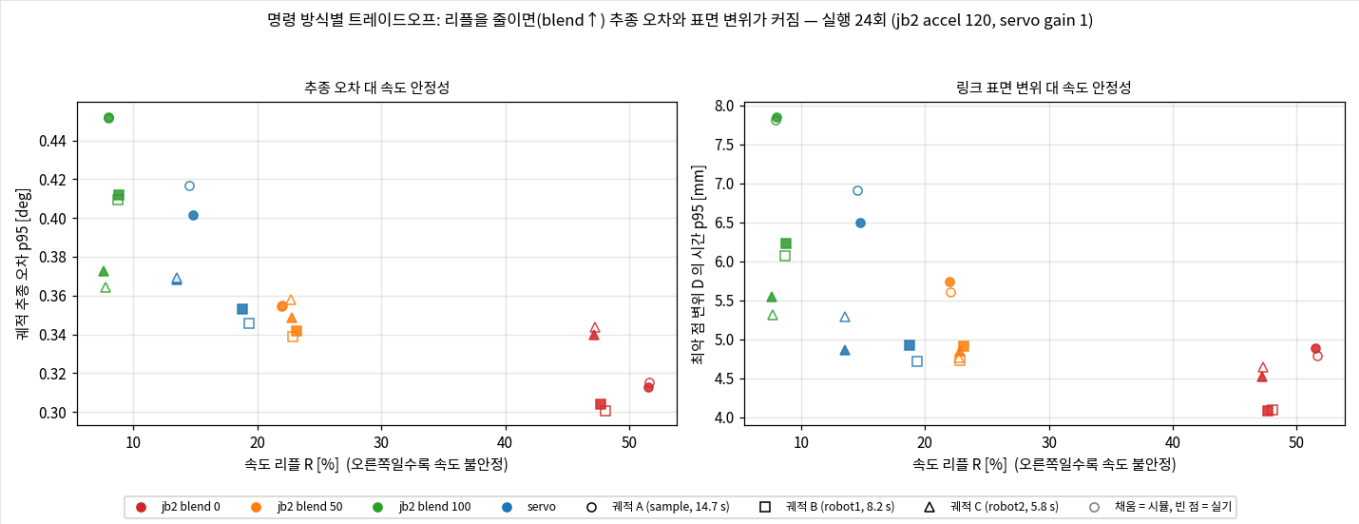
궤적	추종 [s] (참고)	d_p95 [°]	d_max [°]	종료 오차 [°]	리플 R	흔들림 W [°]	D 시간 평균 [mm]	D 시간 p95 [mm]	D 시간 최대 [mm]
A	14.5	0.40 / 0.42	0.66 / 0.70	0.000	14.8 / 14.5 %	0.14 / 0.15	2.11 / 2.28	6.5 / 6.9	16.0 / 14.8
B	8.1	0.35	0.59 / 0.58	0.000	18.8 / 19.3 %	0.17	1.55 / 1.63	4.9 / 4.7	10.0 / 10.3
C	5.8	0.37	0.44 / 0.43	0.000	13.5 %	0.18	2.09 / 2.19	4.9 / 5.3	8.5 / 8.6

해석

- 추종 오차 p95 0.35 ~ 0.42°, 리플 13.5 ~ 19.3 % → jb2 blend 50 과 100 사이
- 측정 속력이 원본 속력을 잘 추종 (3.3절 속력 그림 맨 아래)
  - 리플의 상당 부분은 0.2 s 지령 주기에 따른 속력 요동으로 판단
- 종료 각 오차: 마지막 점 반복으로 전부 0.000° (실기 0.0003°)
- 지령 지연(`t_latency`): 0.02 ~ 0.06 s

3.5 명령 방식별 종합: 추종 오차와 속도 안정성의 트레이드오프

- 비교 축: 추종 오차(경로 정확도) ↔ 속도 안정성(리플·흔들림) → 그 결과로 메시에서 발생한 변위
- 4개 방식 모두 같은 구성 (궤적 A·B·C × 시뮬·실기, 각 6회)



트레이드오프

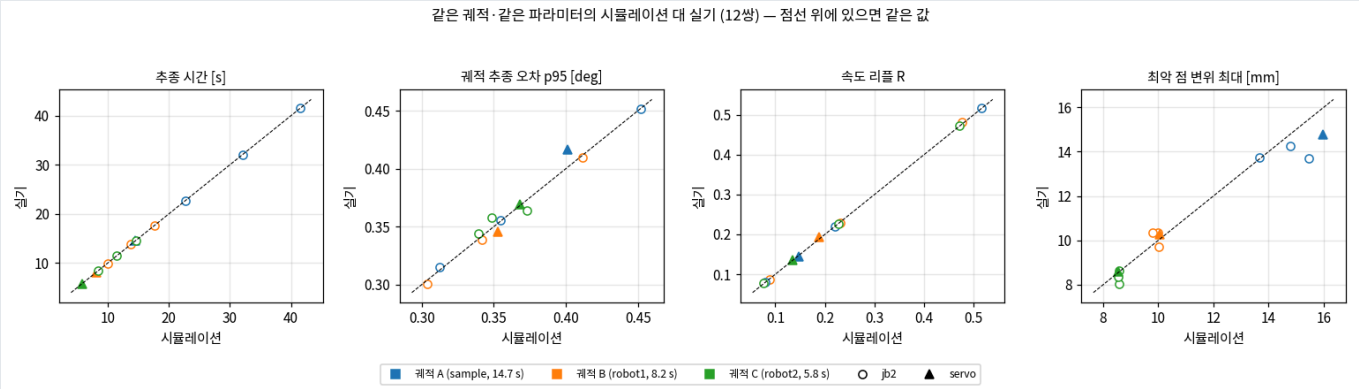
종합 표 (값은 중앙값, 괄호는 최소 ~ 최대, `D` 는 rb20_1900es_RT 기준)

명령 방식	추종 오차 p95 [°]	추종 오차 최대 [°]	속도 리플 R	흔들림 W [°]	D 시간 평균 [mm]	D 시간 p95 [mm]	D 시간 최대 [mm]
jb2 blend 0	**0.31** (0.30 ~ 0.34)	0.59 (0.44 ~ 0.66)	48 % (47 ~ 52 %)	**0.13** (0.10 ~ 0.15)	**1.37** (1.15 ~ 1.50)	**4.6** (4.1 ~ 4.9)	10.2 (8.0 ~ 13.8)
jb2 blend 50	0.35 (0.34 ~ 0.36)	0.58 (0.43 ~ 0.66)	23 % (22 ~ 23 %)	0.15 (0.10 ~ 0.16)	1.78 (1.46 ~ 1.87)	4.9 (4.7 ~ 5.7)	9.9 (8.6 ~ 14.8)
servo	0.37 (0.35 ~ 0.42)	0.58 (0.44 ~ 0.70)	15 % (14 ~ 19 %)	0.17 (0.14 ~ 0.18)	2.10 (1.55 ~ 2.28)	5.1 (4.7 ~ 6.9)	10.2 (8.5 ~ 16.0)
jb2 blend 100	0.41 (0.36 ~ 0.45)	0.58 (0.43 ~ 0.67)	**8 %** (8 ~ 9 %)	0.18 (0.15 ~ 0.18)	2.49 (1.99 ~ 2.61)	6.1 (5.3 ~ 7.9)	10.1 (8.4 ~ 15.5)

- 트레이드오프
  - 추종 오차 p95: blend 0 < blend 50 < servo < blend 100
  - 속도 리플: 같은 순서로 반대 (48 → 23 → 15 → 8 %)
  - blend ↑ → 지령점 모서리를 둥글게 깎아 속도는 고르지만 경로에서 더 벗어남
  - blend 0 → 지령점 직선 경로를 그대로 따라가지만 점마다 감속·정지
  - servo → 오차·리플 모두 jb2 blend 50 과 100 사이
- 메시에서 발생한 오차
  - 표면 변위 시간 평균·p95 가 추종 오차 순서를 그대로 따름 (평균 1.37 → 2.49 mm, p95 4.6 → 6.1 mm)
  - 최악 순간 값(시간 최대)은 방식 간 차이 작음 (중앙값 9.9 ~ 10.2 mm), 궤적에 따라 8.0 ~ 16.0 mm
  - → 최악값은 방식보다 궤적과 지령 간격(현 오차)이 결정
- 흔들림 W: 전 방식 기준(0.3°) 이내, blend 0 이 가장 작음

3.6 시뮬레이션과 실기 비교

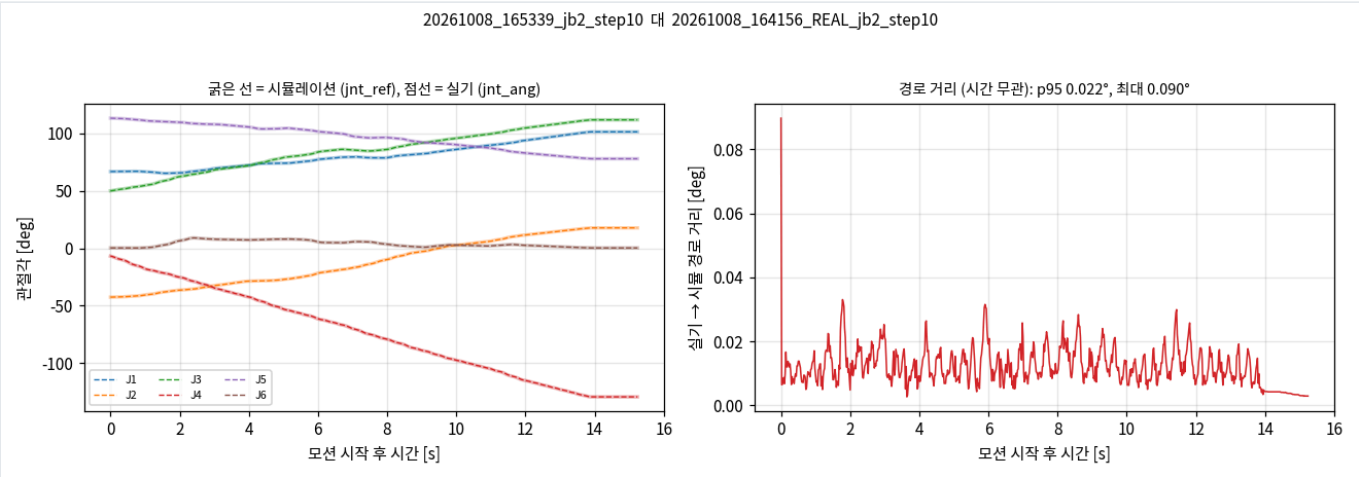
- 대상: 비교 집합의 시뮬레이션·실기 12쌍
- 측정각: 시뮬레이션 `jnt_ref`, 실기 `jnt_ang` (엔코더)



시뮬레이션 대 실기

- 경로 기준 비교 (시간 무관): 실기 측정의 각 점에서 시뮬레이션 측정 경로까지 거리

방식	궤적	파라미터	경로 차이 p95 [°]	경로 차이 최대 [°]
jb2	A	blend 0 / 50 / 100	0.023 / 0.025 / 0.030	0.047 / 0.039 / 0.046
jb2	B	blend 0 / 50 / 100	0.021 / 0.022 / 0.025	0.033 / 0.090 / 0.034
jb2	C	blend 0 / 50 / 100	0.019 / 0.020 / 0.022	0.055 / 0.025 / 0.047
servo	A / B / C	gain 1	0.056 / 0.037 / 0.036	0.085 / 0.065 / 0.051



시뮬레이션과 실기 겹쳐 보기

- 측정 경로 차이 최대 0.09° 이하, 추종 오차(0.3 ~ 0.7°)보다 한 자릿수 작음
  - servo 가 jb2 보다 약간 큼 (0.04 ~ 0.06° 대 0.02 ~ 0.03°)
- 추종 시간 차이 0.02 s 이내, 리플 차이 0.6 %p 이내
- 표면 변위
  - 시간 p95 차이 0.4 mm 이내
  - 시간 최대 차이는 최대 1.8 mm (궤적 A blend 100: 15.5 / 13.7 mm) → 최악 한 순간에 민감한 지표
- 결론: **이 속도 범위·무부하 조건에서는 시뮬레이션으로 실기 관절 경로 예측 가능**
  - 단, 실기 측정도 관절 엔코더 값 → 링크 처짐, 감속기 백래시, 공구 끝 위치 오차 미포함

3.7 링크 메시 기준 변위

- 목적: 관절 오차(deg)를 로봇 표면의 실제 벗어남(mm)으로 환산
- 관절별 1° 오차가 만드는 최악 점 변위 (rb20_1900es_RT, 궤적 A 시작 자세)

관절	J1	J2	J3	J4	J5	J6
1° 당 변위 [mm]	25.6	31.8	18.3	6.8	6.9	2.4

- 방법
  - URDF 링크 충돌 메시에서 링크당 표면 점 600개 추출
  - 추종 구간의 **매칭쌍 전체**에 대해 측정 자세 `q_i` 와 짝지은 원본 자세 `p_i` 를 각각 순기구학으로 배치
  - **같은 메시 점끼리의 거리** 측정
  - 대상: 팔 관절과 함께 움직이는 링크 (`rt_link1` ~ `rt_link_end`), 레일·캐리지 구조물 제외, 외부축은 두 자세 모두 0

$$D_i = \max_v \| \mathbf{x}_v(\mathbf{q}_i) - \mathbf{x}_v(\mathbf{p}_i) \|_2, \qquad D_{max} = \max_i D_i\,, \quad D_{p95} = P_{95}(D_i), \quad D_{mean} = \frac{1}{N}\sum_i D_i$$

$$\mathrm{CD}_i = \frac{1}{2}\left(\frac{1}{|A|}\sum_{a\in A}\min_{b\in B}\|a-b\| + \frac{1}{|B|}\sum_{b\in B}\min_{a\in A}\|a-b\|\right)$$

$$\mathrm{HD}_i = \max\left(\max_{a\in A}\min_{b\in B}\|a-b\|,\ \max_{b\in B}\min_{a\in A}\|a-b\|\right)$$

- 주 지표: **최악 점 변위** `D`
  - Chamfer 는 평균값이라 최악 위치를 과소평가 (비교 집합에서 `D` 최대가 Chamfer 최대의 2.1 ~ 2.7배)
- 정밀도 확인 (실기 servo 궤적 A)
  - 공간 샘플링: 링크당 600점 최댓값과 메시 전체 정점 최댓값 차이 시계열 최대 기준 0.2 %
  - 시간 샘플링: 매칭쌍 일부(200개)만 쓰면 순간 최댓값을 놓침 (14.8 → 13.3 mm) → 본 보고서는 매칭쌍 전체 사용

방식별 결과



경우별 표면 변위

- `D` 시간 평균·p95: blend 0 < blend 50 < servo < blend 100 (3.5절 종합 표)
- `D` 시간 최대: 궤적 C 8.0 ~ 8.6 mm, B 9.7 ~ 10.4 mm, A 13.7 ~ 16.0 mm → 자세 변화가 큰 궤적일수록 큼, 방식 간 차이 작음

궤적별 링크 변위

- 값: 링크마다 "그 순간 해당 링크의 최악 점 변위"를 매칭쌍 전체로 구한 시계열의 최댓값 / P95 [mm]
- 링크: `rt_link1` (어깨 회전부) ~ `rt_link_end` (손끝), rt_link0 (베이스)은 변위 0 이라 생략
- 원본 수치(평균 포함): `link_displacement_rb20.csv`

궤적 A (waypoints_sample) — 링크별 변위 최댓값 / P95 [mm]

방식	모드	link1	link2	link3	link4	link5	link6	link_end
jb2 blend 0	시뮬	0.7 / 0.3	8.1 / 3.2	7.8 / 3.1	11.6 / 4.0	11.8 / 4.1	11.9 / 4.2	13.7 / 4.9
jb2 blend 0	실기	0.7 / 0.3	8.0 / 3.1	7.6 / 3.0	11.2 / 3.9	11.4 / 4.0	11.7 / 4.1	13.8 / 4.8
jb2 blend 50	시뮬	0.8 / 0.3	8.2 / 3.5	7.9 / 3.4	12.0 / 4.8	12.2 / 4.9	12.6 / 5.0	14.8 / 5.7
jb2 blend 50	실기	0.8 / 0.3	8.6 / 3.5	8.2 / 3.4	12.7 / 4.7	12.9 / 4.8	13.1 / 4.9	14.3 / 5.6
servo	시뮬	0.7 / 0.3	8.2 / 3.9	7.9 / 3.9	12.7 / 5.5	13.1 / 5.6	13.6 / 5.8	16.0 / 6.5
servo	실기	0.8 / 0.4	8.7 / 4.1	8.4 / 4.0	13.0 / 5.7	13.2 / 5.9	13.4 / 6.0	14.8 / 6.9

방식	모드	link1	link2	link3	link4	link5	link6	link_end
jb2 blend 100	시뮬	0.8 / 0.4	8.1 / 4.4	7.8 / 4.4	12.3 / 6.7	12.7 / 6.8	13.2 / 7.0	15.5 / 7.9
jb2 blend 100	실기	0.8 / 0.4	8.5 / 4.4	8.1 / 4.4	12.3 / 6.7	12.5 / 6.8	12.7 / 7.0	13.7 / 7.8

궤적 B (waypoints_robot1) — 링크별 변위 최댓값 / P95 [mm]

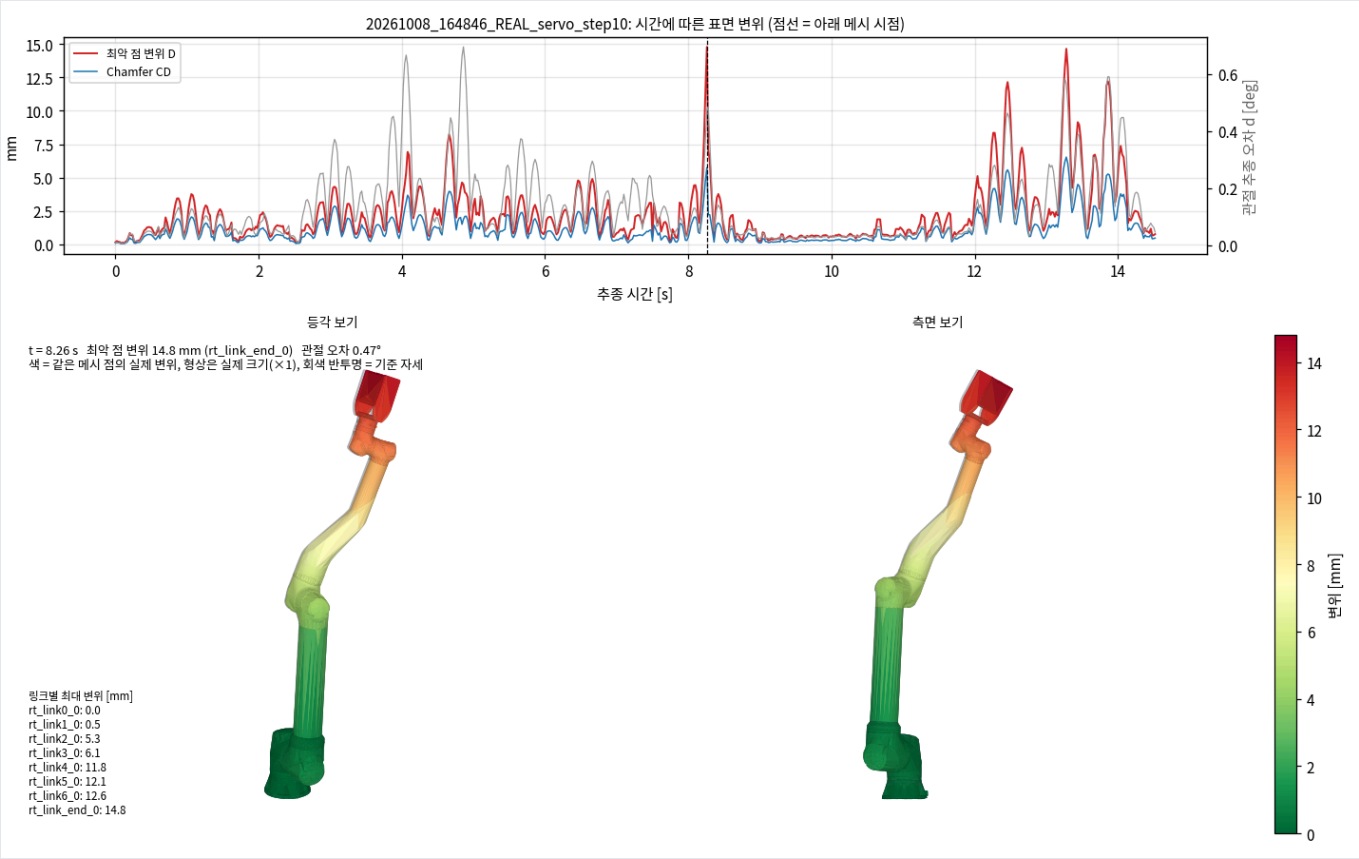
방식	모드	link1	link2	link3	link4	link5	link6	link_end
jb2 blend 0	시뮬	1.0 / 0.4	4.9 / 2.1	5.5 / 2.1	9.6 / 3.5	10.0 / 3.5	9.1 / 3.5	9.4 / 4.0
jb2 blend 0	실기	1.0 / 0.4	5.0 / 2.1	5.7 / 2.1	9.9 / 3.6	10.3 / 3.6	9.5 / 3.5	9.8 / 4.0
jb2 blend 50	시뮬	1.0 / 0.4	4.9 / 2.3	5.5 / 2.3	9.6 / 4.5	10.0 / 4.6	9.1 / 4.3	9.5 / 4.8
jb2 blend 50	실기	1.0 / 0.5	4.9 / 2.2	5.5 / 2.3	9.3 / 4.2	9.7 / 4.3	8.9 / 4.1	9.2 / 4.5
servo	시뮬	1.0 / 0.5	4.9 / 2.2	5.6 / 2.3	9.6 / 4.5	10.0 / 4.6	9.2 / 4.1	9.5 / 4.8
servo	실기	1.0 / 0.4	5.0 / 2.2	5.7 / 2.3	9.9 / 4.3	10.3 / 4.3	9.4 / 4.2	9.8 / 4.5
jb2 blend 100	시뮬	1.0 / 0.5	4.9 / 2.5	5.5 / 2.5	9.4 / 5.2	9.8 / 5.4	9.0 / 5.1	9.3 / 6.1
jb2 blend 100	실기	1.0 / 0.5	4.8 / 2.4	5.5 / 2.6	9.9 / 5.4	10.4 / 5.4	9.5 / 5.2	9.8 / 6.1

궤적 C (waypoints_robot2) — 링크별 변위 최댓값 / P95 [mm]

방식	모드	link1	link2	link3	link4	link5	link6	link_end
jb2 blend 0	시뮬	0.4 / 0.3	3.2 / 2.0	2.9 / 1.9	6.7 / 3.6	6.5 / 3.7	6.9 / 3.8	8.6 / 4.5
jb2 blend 0	실기	0.5 / 0.3	3.3 / 2.0	3.0 / 1.9	6.3 / 3.7	6.2 / 3.7	6.5 / 3.8	8.0 / 4.6
jb2 blend 50	시뮬	0.4 / 0.3	3.2 / 2.1	2.9 / 2.0	6.6 / 3.9	6.5 / 3.9	6.9 / 4.0	8.6 / 4.9
jb2 blend 50	실기	0.5 / 0.3	3.1 / 2.1	2.8 / 2.0	6.7 / 3.9	6.6 / 4.0	7.0 / 4.0	8.6 / 4.8
servo	시뮬	0.4 / 0.3	3.1 / 2.2	2.8 / 2.1	6.6 / 4.0	6.5 / 4.1	6.9 / 4.2	8.5 / 4.9
servo	실기	0.4 / 0.3	3.3 / 2.4	3.0 / 2.3	6.6 / 4.2	6.5 / 4.2	6.9 / 4.4	8.6 / 5.3
jb2 blend 100	시뮬	0.4 / 0.3	3.2 / 2.2	2.9 / 2.2	6.6 / 4.5	6.5 / 4.7	6.9 / 4.8	8.5 / 5.5
jb2 blend 100	실기	0.4 / 0.3	3.3 / 2.3	2.9 / 2.2	6.5 / 4.3	6.3 / 4.4	6.7 / 4.6	8.4 / 5.3

- 링크 위치에 따른 경향
  - 손끝으로 갈수록 변위 증가 (link1 1 mm 이하 → link_end 8 ~ 16 mm), 어깨·팔꿈치 관절 오차가 바깥 링크 전체를 이동시키기 때문
  - 궤적 B 는 link4 ~ link5 가 link6 ~ link_end 와 비슷하거나 큼 → 자세에 따라 최대 변위 링크가 달라짐
- 방식에 따른 경향
  - 손목 쪽 링크(link4 ~ link_end) P95: blend 0 < blend 50 ≈ servo < blend 100 (예: link_end P95 중앙값 4.6 / 4.8 / 5.1 / 6.1 mm) → 3.5절 트레이드오프와 같은 순서
  - 어깨 쪽 링크(link1 ~ link3) P95: 방식 간 차이가 손목 쪽보다 작음 (궤적 A 최대 1.4 mm, B·C 0.5 mm 이내)
  - 링크별 최댓값: 방식보다 궤적이 결정 (같은 궤적 안에서 방식 간 차이 최대 2.3 mm)
- 시뮬레이션 대 실기: 같은 조건에서 링크별 P95 차이 0.5 mm 이내

실기 최대 사례: servo, 궤적 A (`20261008_164846_REAL_servo_step10`)



메시 변위

- t = 8.26 s, 관절 추종 오차 0.47°, 최악 점 변위 **14.8 mm** (`rt_link_end`)
- 표시: 형상은 실제 크기(확대 없음), 색은 같은 메시 점의 변위
- 해당 순간 링크별 최대 변위: link1 0.5, link2 5.3, link3 6.1, link4 11.8, link5 12.1, link6 12.6, link_end 14.8 mm
  - 어깨 쪽 관절 오차가 팔 끝 쪽 링크 전체를 이동시킴 → 바깥 링크일수록 큼
- 표면 변위는 관절 추종 오차(회색)와 함께 증감

모델에 따른 차이

- 같은 측정값이라도 팔이 클수록 mm 값 증가
- 같은 데이터를 rb10_1300e URDF 로 계산하면 시간 평균·p95 가 rb20_1900es_RT 의 약 0.6 ~ 0.7배 → URDF 선택이 결과에 직접 영향

3.8 servo 프로파일 타이밍 (10/14 추가 실험, 실기·시뮬레이션)

방식 변경 (3.2절의 servo 와 다른 점)

- 지령점을 원본 시간 간격으로 보내지 않고, **설정한 속도·가속도로 만든 프로파일**을 **20 ms 고정 주기**로 샘플링해 보냄
  - 프로파일: 원본 궤적 위에서 관절마다 \|속도\| ≤ speed, \|가속도\| ≤ accel 을 지키는 최단 시간 프로파일 (시작·끝에서만 정지, 굽은 곳에서는 감속)
  - 경로는 3차 스플라인으로 보간
  - 실제로 보낼 지령(20 ms 샘플)의 차분 속도·가속도를 계산해, 설정을 넘으면 계획용 한계를 낮춰 다시 만듦 → **지령은 항상 한계 이하** (이 절의 모든 실행에서 확인, 실기 3회의 지령 최대 가속도 48.9 ~ 49.9, 한계 50)
  - `t1` = 주기(20 ms), `t2` = 주기의 2배(0.04 s), gain 1, alpha 1, 마지막 점 반복
- 그래프의 점선 = 이 이상 프로파일
- **분석 코드 버전이 3.1 ~ 3.7절과 다름**: 이 절은 극값 앵커 기반 매칭(`matching.py`)과 모션 구간 끝 수정(마지막 샘플 = 원본 끝점 자세)이 반영된 현재 코드로 계산. 같은 코드로 10/08 데이터를 다시 계산하면 `d_max` 가 1.2 ~ 3.6° 로 달라지므

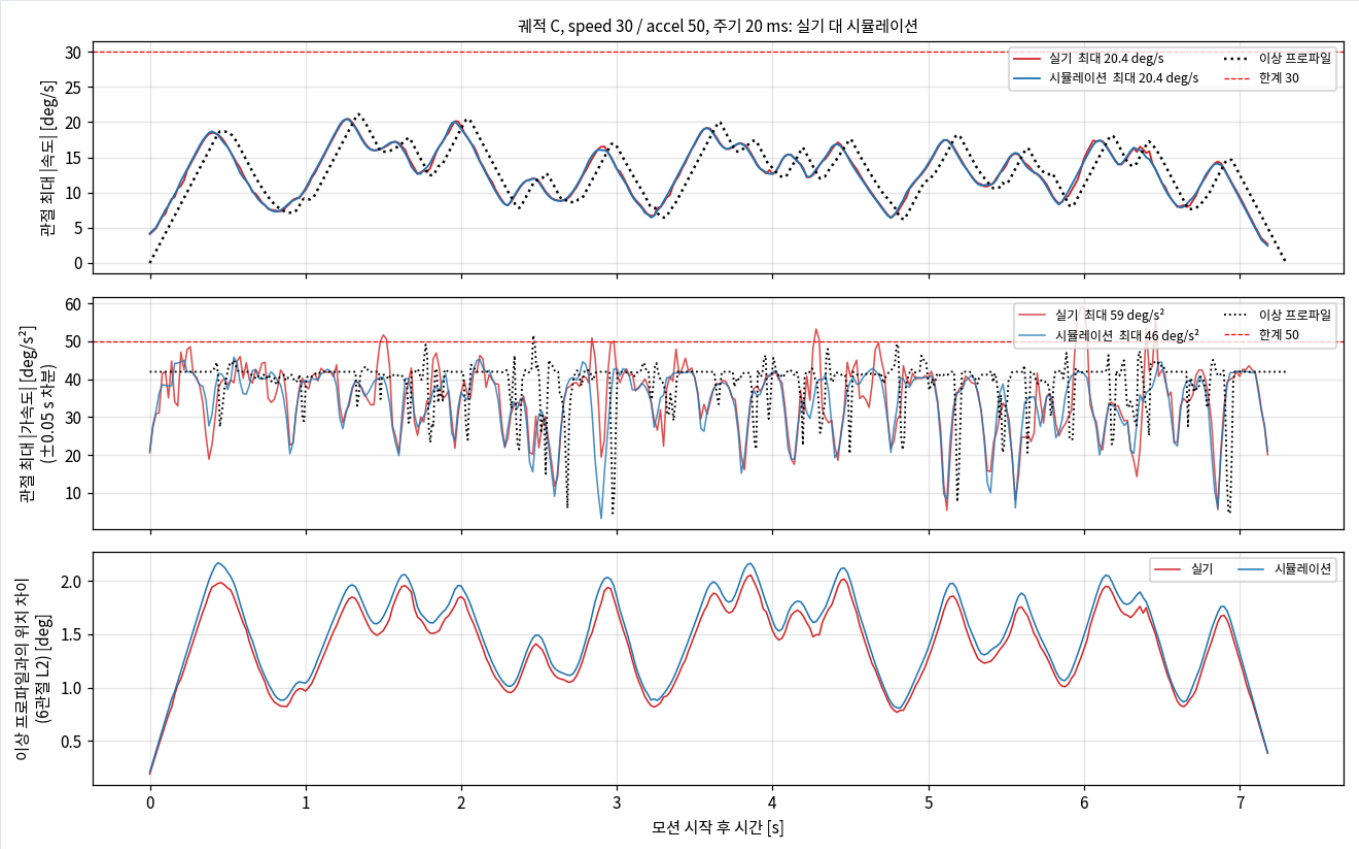
로(3.1 ~ 3.7절은 이전 코드의 값) **두 절의 오차 값을 직접 비교하지 말 것**

- **이 절의 오차 정의**: 측정 샘플에서 **원본 경로(꺾은선)까지의 최단 거리** (시간 매칭 없음)
  - 현재 코드의 매칭쌍 거리(`path_dev_*`)는 실행마다 약 20개 샘플(전체의 3~6 %)이 극값 앵커에 강제로 짝지어져 **단발 스파이크**가 생김 (최단 거리보다 0.05° 이상 큰 샘플이 실행마다 19~25개)
  - 그 결과 매칭쌍 p95·최대가 0.35~0.55° 로 부풀어, 실제 경로 이탈(최대 0.07° 이하)을 반영하지 못함 → 이 절은 최단 거리를 사용하고, 매칭쌍 값은 CSV 에만 남김
  - 최단 거리는 시간 순서를 보지 않으므로 경로를 건너뛰거나 되돌아가는 것은 잡지 못함 (이 실행들은 종료 오차 0.001° 이하, 진행도 정상)

실험 조건

항목	값
실기	3회: 궤적 A·B·C 각 1회, speed 30 deg/s, accel 50 deg/s²
시뮬레이션	지령 한계 수정 이후 12회 (궤적 A·B·C, speed 20 ~ 50, accel 40 ~ 100)
제외	지령 한계 수정 이전 실행 6회 (지령 가속도가 한계 초과), alpha 0.5 실행

실기 결과 (궤적 A·B·C, 30 / 50)



실기 대 시뮬레이션: 속도·가속도

궤적	이상 시간 [s]	측정 시간 [s]	경로까지 거리 p95 [°]	경로까지 거리 최대 [°]	종료 오차 [°]	리플 R	흔들림 W [°]	측정 최대 속도	측정 최대 가속도
A	14.84	14.72	0.054	0.074	0.001	21.6 %	0.06	31.8 (한계 30)	77 (한계 50)
B	8.62	8.52	0.035	0.051	0.001	19.1 %	0.05	27.8	67
C	7.30	7.18	0.029	0.040	0.001	17.2 %	0.05	20.4	59

- 경로 추종은 양호: 원본 경로까지의 거리 p95 0.03 ~ 0.05°, 최대 0.04 ~ 0.07°, 종료 오차 0.001° 수준 (모든 샘플이 0.1° 이내)

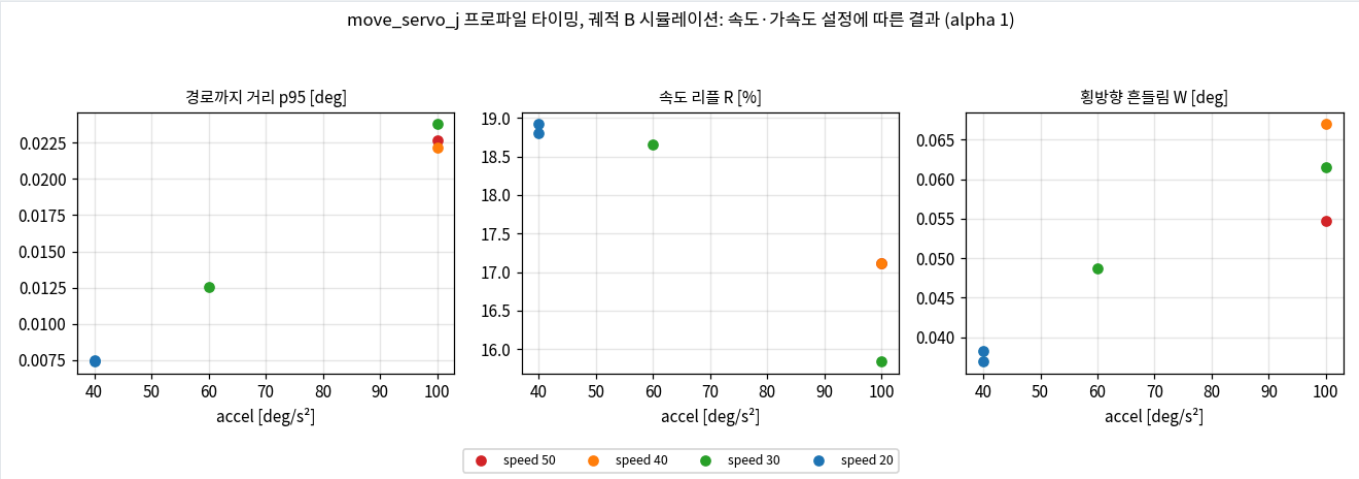
- 측정 시간은 이상 시간보다 0.1 s 정도 짧음
- 측정 속도는 한계 이내 (A 의 31.8 은 한계 30 의 6 % 초과, 100 ms 차분 오차 범위)
- **측정 가속도는 한계 50 을 넘음** (A 77, B 67, C 59)
  - 지령 자체는 한계 이하 (지령 최대 가속도 48.9 ~ 49.9) → 초과는 제어기 응답 또는 엔코더 차분 노이즈에서 발생
  - 둘 중 어느 쪽인지 구분하지 못함 (엔코더 값을 먼저 평활한 뒤 다시 재야 확인 가능)
- 속도 리플 17 ~ 22 %

실기 대 시뮬레이션 (같은 조건은 궤적 C 한 쌍뿐)

항목	시뮬레이션	실기
경로까지 거리 p95 / 최대	0.010° / 0.013°	0.029° / 0.040°
속도 리플	17.3 %	17.2 %
측정 최대 속도	20.4	20.4
측정 최대 가속도	46	59
추종 시간	7.18 s	7.18 s

- 두 경로의 거리: p95 0.028°, 최대 0.038° (3.6절의 servo 쌍 0.036 ~ 0.056° 와 같은 수준)
- 경로까지의 거리는 실기가 시뮬레이션의 약 3배 (0.029 대 0.010°)이나 모두 0.05° 이하
- 속도·가속도에서도 차이: 가속도 46 대 59
- 궤적 A·B 는 같은 조건(30 / 50)의 시뮬레이션이 없어 비교하지 못함

시뮬레이션: 속도·가속도 설정의 영향 (alpha 1, 시뮬레이션 평균, 추종 오차 최대는 최댓값)



속도·가속도 설정에 따른 결과

궤적	speed / accel	횟수	경로까지 거리 p95 [°]	경로까지 거리 최대 [°]	리플 R	흔들림 W [°]	측정 최대 속도	측정 최대 가속도	종료 오차 최대 [°]
A	20 / 40	3	0.007	0.011	19.4 %	0.03	20.3	40	0.446
B	20 / 40	2	0.007	0.011	18.9 %	0.04	20.2	39	0.000
B	30 / 60	1	0.013	0.014	18.6 %	0.05	29.6	61	0.000
B	30 / 100	1	0.024	0.033	15.8 %	0.06	35.9	282	0.000
B	40 / 100	1	0.022	0.025	17.1 %	0.07	38.3	105	0.000
B	50 / 100	1	0.023	0.026	17.1 %	0.05	38.2	100	0.000

궤적	speed / accel	횟수	경로까지 거리 p95 [°]	경로까지 거리 최대 [°]	리플 R	흔들림 W [°]	측정 최대 속도	측정 최대 가속도	종료 오차 최대 [°]
C	20 / 40	2	0.007	0.010	17.6 %	0.05	18.4	37	0.000
C	30 / 50	1	0.010	0.013	17.3 %	0.05	20.4	46	0.000

- 속도·가속도를 낮출수록 경로까지 거리가 줄어듦 (궤적 B: 50 / 100 → 0.023°, 30 / 60 → 0.013°, 20 / 40 → 0.007°), 단 전체가 0.04° 이하라 차이는 작음
- 리플은 15 ~ 19 % 로 설정에 따른 뚜렷한 경향이 없음
- 측정 속도는 설정 한계의 1.02배 이내 (20 / 40 의 20.3 이 최대)
- 측정 가속도는 대체로 한계 부근 (B 20 / 40 의 39, 30 / 60 의 61, 50 / 100 의 100)
  - 이상값: B 30 / 100 의 282 (한 번뿐, 원인 미확인)
- 종료 오차는 한 실행(궤적 A, 20 / 40 첫 번째)을 빼면 0.000°
  - 0.446° 는 마지막 점에 수렴하지 못한 것으로 보임, 같은 조건 재실행 2회는 0.000°
- 실기 3회와 같은 한계(30 / 50)의 시뮬레이션 데이터는 궤적 C 한 건뿐이므로, 시뮬레이션 결과로 실기 값을 예측한다고 일반화할 수는 없음

이전 servo (3.4절) 대비

- 이전: 원본 시간 간격(0.2 s)으로 지령점 전송, 지령 간격이 길어 속도가 계단 모양으로 바뀜
- 이번: 20 ms 고정 주기, 속도·가속도 한계를 지키는 지령
- 분석 코드 버전이 달라 두 방식의 오차는 수치로 직접 비교하지 않음

4. 결론과 권장 사항

1. **명령 방식 선택은 추종 오차와 속도 안정성 사이의 트레이드오프**
   - 오차 우선: jb2 blend 0 (오차 p95 중앙 0.31°, 리플 48 %)
   - 속도 안정성 우선: jb2 blend 100 (리플 8 %, 오차 p95 중앙 0.41°)
   - 중간: jb2 blend 50 (0.35°, 23 %), servo (0.37°, 15 %)
2. **메시 변위는 추종 오차를 그대로 반영 (rb20_1900es_RT)**
   - 표면 변위 시간 평균 1.37 mm (blend 0) ~ 2.49 mm (blend 100), 시간 p95 4.6 ~ 6.1 mm
   - 최악 순간 변위는 방식과 무관하게 8.0 ~ 16.0 mm, 궤적에 따라 결정
   - 응용(간섭 여유 등) 허용치 확정 시 판정 기준으로 추가 권장
3. **추종 오차 최대는 0.2 s 지령 간격이 결정**
   - 방식·blend 와 무관하게 궤적별 현 오차 최대(0.44 ~ 0.67°)와 동일
   - 오차 하한을 낮추려면 지령 간격 축소 또는 곡률이 큰 구간에 지령점 추가가 우선
   - 단, servo 명령 주기 축소 시 `t1` 동반 축소 필요, 0.2 s 미만 주기는 미시험
4. **servo 사용 시 마지막 점 반복 필수** (미반복 시 종료 오버슈트 3.59°)
5. **시뮬레이션 기반 파라미터 사전 선정 유효**: 관절 경로 기준 실기와 0.09° 이내 일치

1. **servo 프로파일 타이밍 (3.8절)**
   - 속도·가속도 설정으로 프로파일을 만들어 20 ms 로 보내면 실기에서도 경로 추종 양호 (원본 경로까지 거리 p95 0.03 ~ 0.05°, 최대 0.07° 이하)
   - 지령은 한계 이하이나 **실기 측정 가속도는 한계를 넘음** (59 ~ 77 대 50) → 가속도 한계가 중요한 응용이면 설정을 낮춰야 함
   - 실기·시뮬레이션이 같은 조건인 비교는 궤적 C 한 쌍뿐이라, 궤적 A·B 의 짝 실험이 필요

다음 시험 제안

- 서브샘플링 비율 2, 5 (0.04 s, 0.1 s) 에서 servo·jb2 비교 → 오차 하한 실제 감소 여부 확인
- 다른 accel, servo gain·alpha 조건을 궤적 3종 × 시뮬·실기 전체로 수행 (이번에는 일부만 수행되어 제외)
- 문서 범위 내 servo 파라미터 (`t2 < 0.2` , `alpha < 1`) 재시험
- `reqdata` 의 `tcp_pos` 와 URDF 순기구학 비교 → URDF 와 실제 로봇 일치 확인
- 공구 장착·부하 조건 실기 시험
- servo 프로파일 타이밍: 궤적 A·B 의 30 / 50 시뮬레이션 짝, 시뮬레이션과 같은 조건의 실기 추가, 엔코더 평활 후 가속도 재측정

5. 한계

- 합격선(흔들림 0.3° 등): 응용 요구가 아닌 임의 초기값
- 진행도 FAIL 4회 (궤적 C jb2, 0.989 ~ 0.990)
  - 종료 각 오차 0.00° 로 끝점 도달 확인
  - 실제 미도달보다 진행도 계산(끝점 근처 매칭)의 경계 효과로 추정
- 실기 측정: 관절 엔코더(`jnt_ang`) 기준 → 링크 처짐, 감속기 백래시, 공구 끝 위치 미포함
- 링크 표면 변위: URDF 와 실제 로봇이 같다는 가정
  - URDF 일치 여부 미확인
  - 공구, 케이블 등 URDF 에 없는 형상 미포함
- 매끄러움 지표의 창 길이가 추종 시간에 비례 → 실행 시간이 크게 다른 실행 간 리플 직접 비교 시 주의
- jb2 실행 시간의 날짜별 변동 사례 있음
  - 태블릿 속도 배율 설정 영향으로 추정, 기록 없음
  - 이번 실험은 한 시간대 내 수행, 반복 간 차이 0.02 s 이내
- 매칭쌍 거리의 스파이크: 현재 코드의 극값 앵커 매칭은 실행마다 약 20개 샘플을 최단 거리보다 크게 짝지어, 매칭쌍 p95·최대(0.35 ~ 0.55°)가 실제 경로 이탈(0.07° 이하)보다 크게 나옴. 3.8절은 최단 거리로 대체했고, 분석 코드(GUI 표의 추종 오차 포함)는 수정하지 않음
- 분석 코드 버전 차이: 3.1 ~ 3.7절은 이전 매칭으로, 3.8절은 극값 앵커 기반 매칭으로 계산. 같은 10/08 데이터를 현재 코드로 다시 계산하면 `d_max` 가 서보·jb2 모두 1.2 ~ 3.6° 로 커지는데, 매칭 앵커의 영향으로 보이나 원인을 확인하지 못함 → 3.1 ~ 3.7절 수치는 이전 코드 기준
- 비교 대상이 jb2 accel 120, servo gain 1 / alpha 1 한 조건씩 → 파라미터 최적값 판단에는 추가 시험 필요

부록. 첨부 데이터 (CSV)

- 첨부: 보고서 표의 원본 수치 (`results/`), 실험에 쓴 궤적 (`trajectories/`)
- 원시 측정 데이터는 미첨부 (별도 보관: `~/rb_servo_gui_runs/report_dataset_20261008/`)
- 열 정의는 첨부 명세서 `RESULTS_SPEC.pdf`

`results/`

파일	내용	보고서 위치
`runs_index.csv`	실행 목록과 명령 파라미터 (비교 대상 24회, 보조 2회)	3.1절
`run_metrics.csv`	실행별 추종 오차, 안정성, 표면 편차, 판정	3.3 ~ 3.4절
`method_summary.csv`	방식별 요약 (중앙값, 최소, 최대)	3.5절
`link_displacement_rb20.csv`	궤적·방식·모드별 링크 변위 (최댓값, P95, 평균)	3.7절
`sim_real_pairs.csv`	시뮬레이션 대 실기 경로 거리	3.6절
`trajectory_chord_error.csv`	궤적별 지령 간격 현 오차	1.3절
`joint_sensitivity_rb20.csv`	관절별 1° 당 표면 변위	3.7절
`servo_profile_runs_1014.csv`	servo 프로파일 타이밍 실행별 결과 (실기 3, 시뮬레이션 18, 경로까지 거리와 매칭쌍 거리 모두 수록)	3.8절

`trajectories/`

파일	내용	보고서 위치
`waypoints_sample.csv` , `waypoints_robot1.csv` , `waypoints_robot2.csv`	궤적 A / B / C 입력 웨이포인트	1.2절
`reference_A_50Hz.csv` , `reference_B_50Hz.csv` , `reference_C_50Hz.csv`	50 Hz 내삽 원본 궤적 (평가 기준)	1.2 ~ 1.3절

웨이포인트 분할에 따른 속도·경유점 도달 오차 (jb2 / servo)

- 목적: 같은 경로를 jb2 의 분할 수·blend 와 servo 의 전송 주기를 바꿔 실행하고, **속도**와 **경유점에 얼마나 가깝게 도달했는지** 비교
- 실험 시각: 2026-10-16 15:22 ~ 15:37, 도구: `rb_servo_test/jb2_gui`, 로봇 IP 10.0.2.7
- 대상: 분석 28건 (jb2 시뮬레이션 12, jb2 실기 13, servo 실기 3), 제외 1건
- 수치는 모두 `runs_summary.csv` 에 있음 (실행 폴더별 원시 값은 `out/<폴더>/waypoint_proximity_0.csv`)

요약

명령	경유점 도달 오차 (최대, 실기)	모션 시간 (이상 12 s)	측정 최고 속도 (설정 10)
jb2 blend 0	0.008 ~ 0.010° (분할 수와 무관)	18.9 ~ 29.7 s (분할 수↑ → 느림)	6.2 ~ 6.8
jb2 blend 50	8.15° (N=4) → 0.60° (N=24)	15.6 → 22.9 s	6.2 ~ 6.8
jb2 blend 100	28.1° (N=4) → 1.18° (N=48)	12.4 → 22.1 s	5.0 ~ **12.1**
servo (gain 1)	0.012 ~ 0.060° (주기 20 → 200 ms)	11.9 ~ 12.0 s	10.5 ~ 10.7

- **blend 0**: 모든 경유점에 정확히 도달 (시뮬레이션 0.000°, 실기 0.01° 이하). 대신 점마다 정지하므로 분할 수가 늘수록 느려짐
- **blend 50·100**: 코너를 깎아 경유점에 못 미침. **오차는 구간 길이에 거의 비례** (blend 100 은 구간 길이의 약 0.25배, blend 50 은 약 0.06배)
- 분할 수를 늘리면 오차는 줄지만 **모션 시간은 늘고 속도는 설정에 못 미침** (blend 100: N=4 12.4 s → N=48 22.1 s)
- **servo**: 경유점에 0.06° 이내로 도달하면서 시간은 이상값(12 s)과 거의 같음. 전송 주기가 길수록 오차 증가 (20 → 200 ms 에서 5배)
- 시뮬레이션과 실기의 도달 오차 차이는 최대 0.007° → 경유점 도달은 제어기 계획이 결정함

1. 실험 조건

- 경로: 6관절이 같은 값으로 움직이는 왕복 `0 → 20 → 0 → 30 → 0` (deg), 시작 자세 `0 0 0 0 0 0`
  - 웨이포인트 5개: start, via1 (20°), via2 (0°, 반전), via3 (30°), end (0°)
  - 구간(leg) 4개: 최대 이동 20°, 20°, 30°, 30°
- 공통 설정: speed 10 deg/s, accel 20 deg/s², speed bar 1.0, 기록 100 Hz, 속도 차분 창 0.1 s
- 이상 모션 시간 12 s: 경유점마다 정지하는 사다리꼴 프로파일 (`vp = min(speed, √(accel × 구간 길이))`)의 합

명령	변경한 항목	값
jb2 (`move_jb2_add` → `move_jb2_run`)	전체 분할 수 N	4, 8, 12, 24 (실기 blend 100 은 48 추가)
	blend	0, 50, 100
servo (`move_servo_j`)	전송 주기	20, 100, 200 ms (gain 1, alpha 1, t2 0.1 s)

- 분할 수 N 은 **경로 전체**의 jb2 구간 수이며 leg 길이에 비례해 나눔 (각 leg 최소 1개)

N	leg 별 분할 수 (20°, 20°, 30°, 30°)	jb2 구간 길이 [deg] (선두 관절)
4	[1, 1, 1, 1]	20.00, 20.00, 30.00, 30.00
8	[2, 2, 2, 2]	10.00, 10.00, 15.00, 15.00
12	[3, 3, 3, 3]	6.67, 6.67, 10.00, 10.00
24	[5, 5, 7, 7]	4.00, 4.00, 4.29, 4.29
48	[10, 10, 14, 14]	2.00, 2.00, 2.14, 2.14

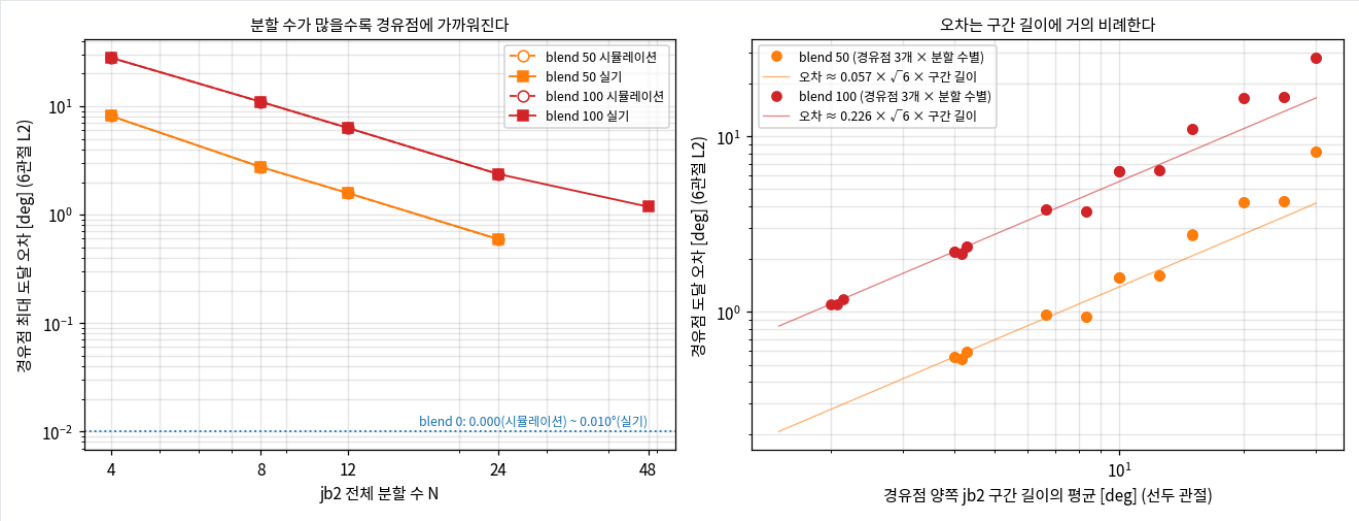
- N=4 는 jb2 에 보낸 점이 경유점·목표 4개뿐임 (중간 점 없음)
- 시뮬레이션 jb2: 12건 (N 4/8/12/24 × blend 0/50/100), 실기 jb2: 13건 (같은 조합 + N=48 blend 100), 실기 servo: 3건
- 제외: servo gain 0.1 (사용자 중지로 미완료) 1건

2. 평가 지표

- **경유점 도달 오차**: 실측 경로(100 Hz 샘플 사이를 선분으로 이은 꺾은선)와 웨이포인트의 **6관절 공간 최단 거리** (deg, L2)
  - 시간은 보지 않음 (timing 이 달라도 경로가 그 점을 지났는지만 평가)
  - 웨이포인트를 지정한 순서대로 지나는 것으로 배정 (시작=목표인 왕복도 구분)
  - 본 보고서의 값은 "경유점 3개 중 최댓값"이며, 개별 값은 CSV 의 via1/via2/via3
- **측정 최고 속도**: 선두 관절의 최대 |속도| (0.1 s 중심 차분)
- **모션 소요 시간**: 시작 자세에서 벗어난 시각 ~ 최종 자세의 0.05° 이내에 계속 머무는 시각
- 같은 조건 1회씩 실행 (반복 없음)

3. 결과

3.1 jb2: 분할 수와 경유점 도달 오차



경유점 도달 오차

경유점 최대 도달 오차 [deg] (시뮬레이션 / 실기)

분할 수 N	blend 0	blend 50	blend 100
4	0.000 / 0.010	8.150 / 8.153	28.124 / 28.130
8	0.000 / 0.010	2.753 / 2.759	11.013 / 11.017
12	0.000 / 0.010	1.574 / 1.579	6.295 / 6.300
24	0.000 / 0.008	0.593 / 0.595	2.371 / 2.373
48	- / -	- / -	- / 1.185

경유점별 오차 (실기, blend 100) [deg]

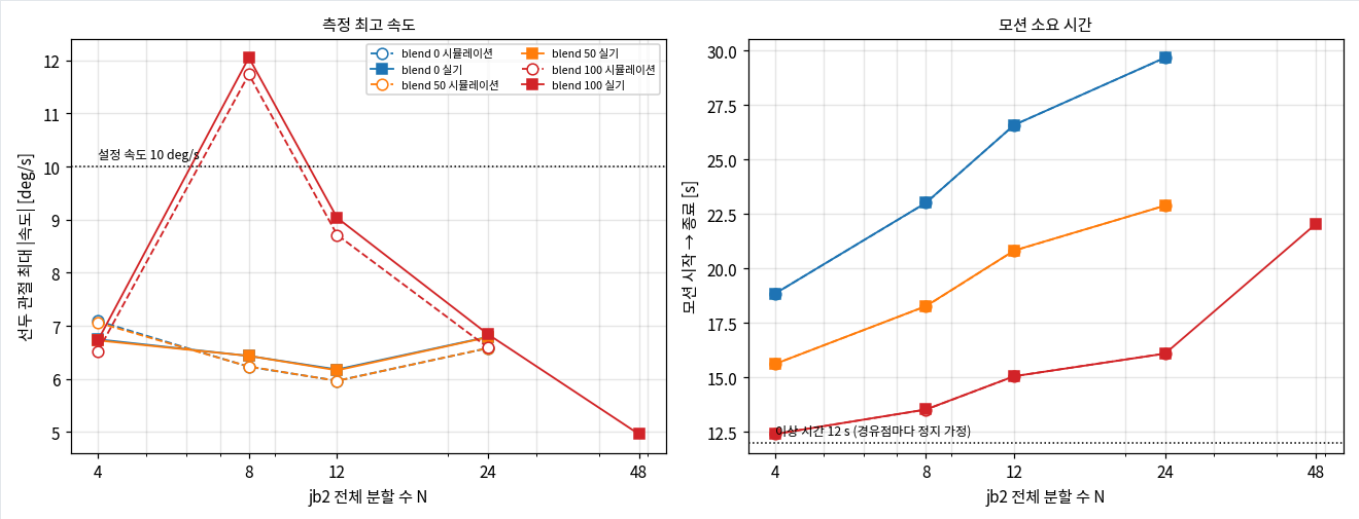
N	via1 (20°)	via2 (0°, 반전)	via3 (30°)	보낸 분할점 평균 / 최대
4	16.65	16.79	28.13	15.40 / 28.13
8	6.30	6.44	11.02	2.98 / 11.02
12	3.84	3.76	6.30	1.17 / 6.30
24	2.21	2.15	2.37	0.29 / 2.37
48	1.11	1.11	1.18	0.08 / 1.18
48	1.11	1.11	1.18	0.08 / 1.18

- **blend 0**: 분할 수와 무관하게 모든 경유점에 정확히 도달 (시뮬레이션 0.000°, 실기 0.008 ~ 0.010° = 엔코더 측정 수준)
- **blend 50**: N=4 8.15° → N=24 0.60°
- **blend 100**: N=4 28.1° → N=24 2.37° → N=48 1.18°
- 오차가 가장 큰 경유점은 via3 (30° 정점). 이동량이 큰 구간(30°)에서 코너 절삭이 큼
- **오차와 구간 길이의 관계**: 경유점 양쪽 jb2 구간 길이의 평균 ℓ 에 대해 오차 / ($\ell \times \sqrt{6}$) 비율 ($\sqrt{6}$ = 6관절이 같은 값으로 움직일 때 L2 환산)

구간 길이 ℓ	blend 50 비율 (평균, 범위, n)	blend 100 비율 (평균, 범위, n)
0 ~ 5°	0.055 (0.053 ~ 0.057, 6)	0.222 (0.212 ~ 0.226, 9)
5 ~ 12°	0.058 (0.046 ~ 0.065, 8)	0.233 (0.184 ~ 0.257, 8)
12 ~ 40°	0.079 (0.053 ~ 0.111, 10)	0.301 (0.210 ~ 0.383, 10)

- 구간이 12° 이하이면 비율이 거의 일정: blend 50 은 약 0.05 ~ 0.07, blend 100 은 약 0.18 ~ 0.26
  - 관절당 환산: 경유점에서 **각 관절이 목표에서 벗어나는 양 ≈ ℓ × (blend 50: 0.06, blend 100: 0.23)** (6관절 같은 이동량, speed 10 기준)
- 구간이 12° 를 넘으면 비율이 더 큼 (평균 blend 50 0.079, blend 100 0.301; 표 참고)
- 같은 경로에서 얻은 경험식이므로 속도·자세·이동량 비율이 다른 경로에는 그대로 적용하지 말고 확인해야 함

3.2 jb2: 속도와 소요 시간



속도와 소요 시간

측정 최고 속도 [deg/s] (시뮬레이션 / 실기, 설정 10)

분할 수 N	blend 0	blend 50	blend 100
4	7.1 / 6.7	7.1 / 6.7	6.5 / 6.7
8	6.2 / 6.4	6.2 / 6.4	11.7 / 12.1
12	6.0 / 6.2	6.0 / 6.2	8.7 / 9.0
24	6.6 / 6.8	6.6 / 6.8	6.6 / 6.8
48	- / -	- / -	- / 5.0
48	-	-	- / 5.0

모션 소요 시간 [s] (시뮬레이션 / 실기, 이상 12.0 s)

분할 수 N	blend 0	blend 50	blend 100
4	18.9 / 18.9	15.6 / 15.6	12.4 / 12.4
8	23.0 / 23.0	18.3 / 18.3	13.5 / 13.5
12	26.6 / 26.6	20.8 / 20.8	15.1 / 15.1
24	29.7 / 29.7	22.9 / 22.9	16.1 / 16.1
48	- / -	- / -	- / 22.1
48	-	-	- / 22.1

- 대부분 **설정 속도 10 에 못 미침** (5.0 ~ 7.1). 구간이 짧으면 속도를 올릴 거리가 부족하기 때문 ($v_p \approx \sqrt{(\text{accel} \times \text{구간 길이})}$)
- **blend 100, N=8 에서만 설정 속도 초과**: 시뮬레이션 11.7, 실기 12.1 (설정의 1.21배), N=12 는 9.0. 코너를 가로지르며 속도가 올라간 것으로 보이며 원인은 확인하지 못함
- **blend 0**: 점마다 정지해 시간이 길고 분할 수가 늘수록 더 길어짐 (N=4 18.9 s → N=24 29.7 s, N이 6배일 때 시간은 약 1.6배)

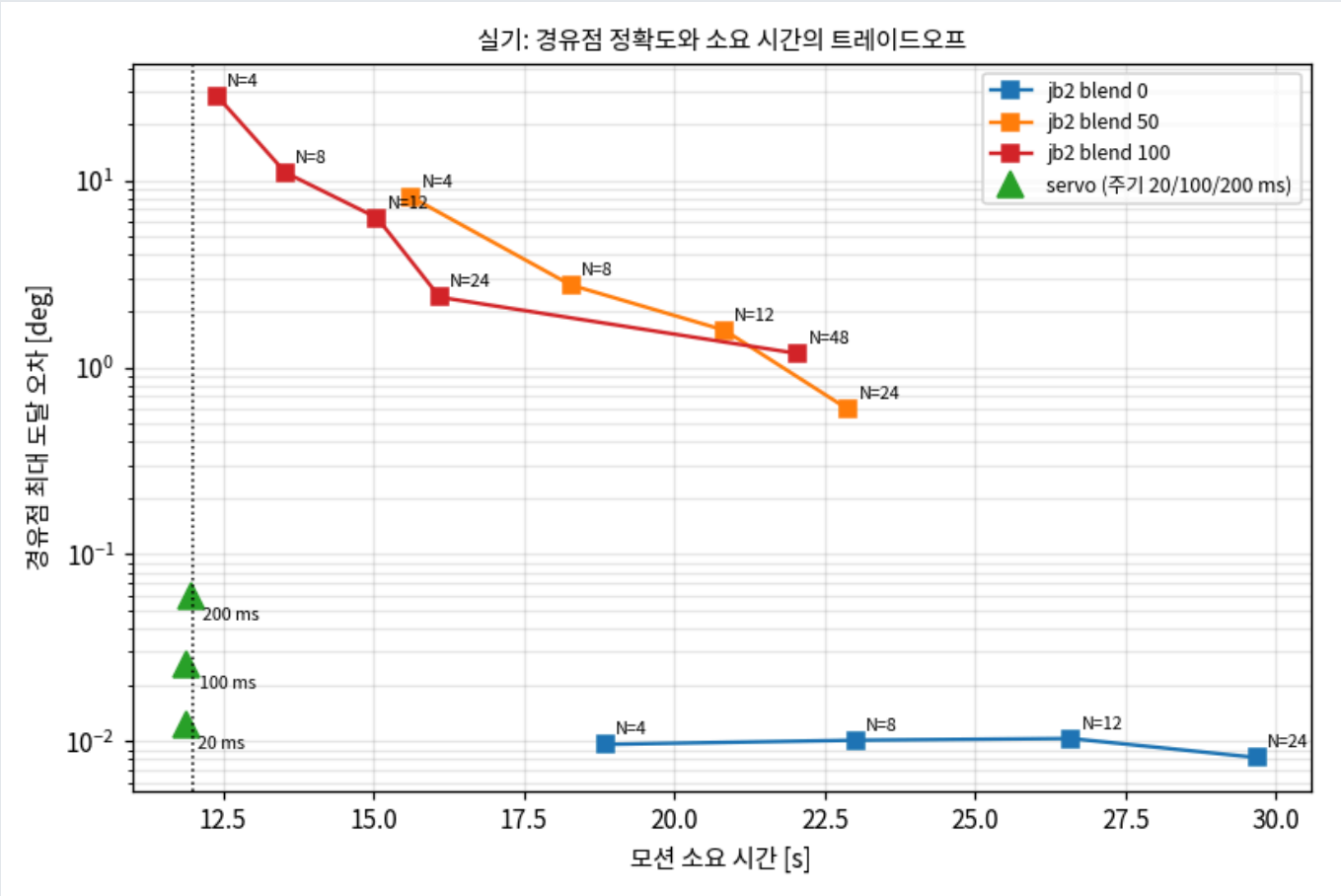
- **blend 100**: 이상 시간(12 s)에 가장 가까움 (N=4 12.4 s) 하지만 분할 수를 늘릴수록 길어짐 (N=48 22.1 s)

3.3 servo: 전송 주기와 경유점 도달 오차

설정 주기	실제 전송 간격 평균 ± 표준편차 [ms] (최소 ~ 최대)	계획 대비 늦음 평균 / 최대 [ms]	경유점 최대 오차 [deg]	via1 / via2 / via3 [deg]	측정 최고 속도	모션 시간 [s]
20 ms	20.00 ± 0.04 (19.5 ~ 20.5)	0.16 / 0.70	0.012	0.012 / 0.010 / 0.012	10.7	11.88
100 ms	100.00 ± 0.03 (99.9 ~ 100.1)	0.16 / 0.20	0.026	0.026 / 0.023 / 0.024	10.7	11.90
200 ms	193.54 ± 1.05 (192.2 ~ 194.5)	0.16 / 0.20	0.060	0.055 / 0.060 / 0.059	10.5	11.96

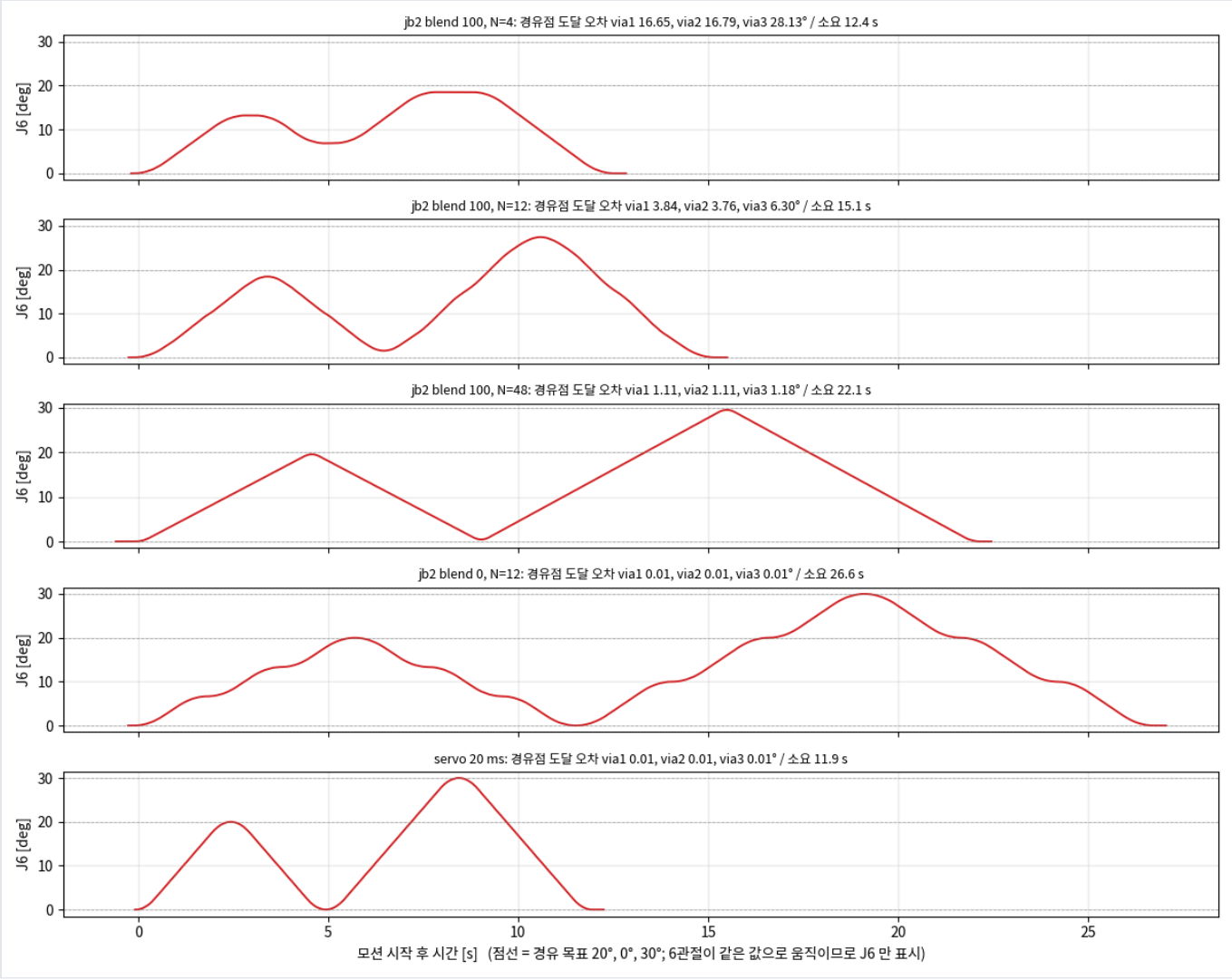
- **전송 주기의 정확도**: PC 에서 `move_servo_j` 를 보낸 시각 기준으로 계획 간격과 거의 같음 (20 ms, 100 ms 는 표준편차 0.03 ~ 0.04 ms, 계획 대비 늦음 최대 0.7 ms)
- **설정 200 ms 는 실제로 평균 193.5 ms**: 모션 시간 12 s 를 설정 주기 이하의 같은 간격으로 나누므로 (`ceil(12 / 0.2) = 62` 개) 설정값은 주기의 상한임. 이 표의 "200 ms"는 실제 193.5 ms
- 측정한 것은 **PC 가 보낸 시각**임. 제어기가 명령을 받아 처리한 시각(네트워크·버퍼 지연)은 측정하지 못했음
- 지터 최대 0.7 ms 는 20 ms 주기의 3.5 %, t2 (0.1 s) 에 비해 작음. 다만 일반 Linux 라 드물게 큰 지연이 생길 수 있고, 본 실험 600회에서는 관측되지 않음
- 모두 0.06° 이내로 경유점에 도달하고 모션 시간은 이상값 12 s 와 거의 같음 (11.9 ~ 12.0 s)
- 전송 주기가 길수록 오차가 커짐 (20 ms 0.012° → 설정 200 ms(실제 193.5 ms) 0.060°, 5배)
- 측정 최고 속도는 10.5 ~ 10.7 로 설정의 1.05 ~ 1.07배
- servo 는 시뮬레이션 실행이 없고, gain 0.1 실행은 중단되어 gain 의 영향은 확인하지 못함
- servo 는 경로 전체를 하나의 시간 프로파일로 보내므로 jb2 와 달리 분할 수·blend 개념이 없음

3.4 트레이드오프



정확도와 시간의 트레이드오프

- jb2 는 **blend 0 (정확하지만 느림) ↔ blend 100 (빠르지만 코너를 깎음)** 사이에서 선택하고, 분할 수로 정확도를 사 오면 시간이 늘어남
- servo 는 0.06° 이하의 도달 오차와 이상 시간에 가까운 시간을 동시에 만족
- 각 실행의 궤적 예 (J6):


jb2 blend 100, N=4: 경유점 도달 오차 via1 16.65, via2 16.79, via3 28.13° / 소요 12.4 s
jb2 blend 100, N=12: 경유점 도달 오차 via1 3.84, via2 3.76, via3 6.30° / 소요 15.1 s
jb2 blend 100, N=48: 경유점 도달 오차 via1 1.11, via2 1.11, via3 1.18° / 소요 22.1 s
jb2 blend 0, N=12: 경유점 도달 오차 via1 0.01, via2 0.01, via3 0.01° / 소요 26.6 s
servo 20 ms: 경유점 도달 오차 via1 0.01, via2 0.01, via3 0.01° / 소요 11.9 s
J6 [deg]
0
10
20
30
0
5
10
15
20
25
모션 시작 후 시간 [s] (점선 = 경유 목표 20°, 0°, 30°; 6관절이 같은 값으로 움직이므로 J6 만 표시)

예시 궤적

4. 결론과 권장

1. **경유점을 정확히 지나야 하면 jb2 blend 0 또는 servo**: 0.01 ~ 0.06° 이내 (jb2 blend 0 은 느림, servo 는 주기 20 ms 권장)
2. **jb2 blend 50·100 은 코너 절삭이 있음**: 오차 ≈ 구간 길이 × 계수 (blend 50 약 0.06, blend 100 약 0.23, 관절당)
   - 허용 오차 e [deg/관절] 를 정하면 구간 길이 ≈ e / 계수 이하로 분할 (예: blend 100 에서 관절당 0.5° 이내이면 구간 약 2° 이하)
   - 분할 수가 늘수록 시간이 늘고 속도가 낮아지므로, 정확도와 시간 사이에서 정함
3. **jb2 blend 100 은 일부 분할 수에서 설정 속도를 초과**할 수 있음 (N=8 에서 1.2배). 속도 제한이 중요하면 실측으로 확인
4. **시뮬레이션으로 사전 선정 가능**: 경유점 도달 오차 차이 최대 0.007° (jb2)

5. 한계

- 경로 1종 (6관절 같은 값으로 움직이는 왕복, 이동량 20°·30°), 속도 10 deg/s, 가속도 20 deg/s² 한 조건. 속도·가속도·관절 구성이 다르면 계수와 속도 거동이 달라질 수 있음
- 조건마다 1회 실행 (반복 오차 미확인). 단, 시뮬레이션과 실기가 같은 값을 줘 재현성은 높아 보임
- servo 는 실기 3건만 (시뮬레이션 없음, gain·alpha 변화 없음, 분할 개념 없음)
- 오차 계수(blend 별 비율)는 이 경로에서 얻은 경험값이며 제어기 내부 blending 식은 확인하지 못함
- 이상 시간 12 s 는 경유점마다 정지하는 프로파일의 합으로, blend 가 있는 제어기 동작의 예측값이 아님
- 6관절이 같은 값으로 움직이는 경로라 관절마다 속도·가속도 한계가 다른 효과(이전 단일 관절 실험)는 이 경로에서 분리되지 않음

부록. 파일

파일	내용
`runs_summary.csv`	실행별 조건, 경유점 오차, 속도, 시간 (분석 29건 중 완료 28건)
`out/20261016_152212_* ~ 153709_*/`	원본 실행 폴더 (`waypoint_proximity_0.csv` , `command_points_proximity_0.csv` , `velocity_0.csv` , `summary.csv`)
`report/make_report.py` , `collect.py`	이 보고서의 표·그림 생성

동일 조건 반복 기록 확인 — 2026-10-20

검사 범위: `/home/dev/rb_servo_gui_runs`의 config 보유 실행 145개 및 `rb_servo_test/jb2_gui/out`의 201개. 데이터셋 복제 폴더는 중복 실행으로 세지 않았다. 완료 마크/manifest가 있는 실행만 비교. 저장된 유효 설정에서 경로 위치·표시/평가 관련 필드를 제외한 설정 일치 및 저장 전송점 파일 SHA-256 일치로 묶었다. 실제 코드 버전·외부 부하·온도까지 동일하다는 보장은 없다. servo 실제 파라미터는 follow 메타데이터도 확인했다. 전체 목록은 matching_runs.json.

실기 반복

- servo, waypoints_sample.csv: 20261008_111008_REAL_servo_step10 / 20261008_111351_REAL_servo_step10.

75개 지령점, 0.2초 간격, 실제 t1=t2=0.2초, gain=alpha=1. config의 t2=0.4는 실행 시 0.2로 제한됨. speed=100, accel=200은 당시 servo 자체의 속도 프로파일을 의미하지 않는다.

- jb2: 20261016_141046_862354 / 20261016_141107_694066.

0 자세 → [0,0,0,20,20,20]°, speed=30, accel=60, blend=100, 1분할.

- jb2: 20261016_141054_813994 / 20261016_141137_793703.

0 자세 → [20,20,20,20,20,20]°, 같은 speed/accel/blend/분할 조건.

10/14 보고서 실기 프로파일 시험은 A·B·C 각 1회이므로 위 반복과 구분해야 한다.

방향 예비 분석

10/08 servo 두 실행의 엔코더각을 원본 6관절 꺾은선의 최근접 선분에 투영하고, `오차 = 측정각 - 투영점`의 부호를 비교했다. 경로 누적 길이 5~95%를 1% 폭 90구간으로 나눠 각 실행에서 최소 2개 측정점이 있는 구간의 평균 오차를 비교했다. 시간 오차를 제거한 관절공간 경로 잔차이므로 TCP/작업공간 오차 방향과는 다르다. 전역 최근접 투영은 자기 교차·가까운 반복 구간에서 대응이 모호할 수 있으며 시간 순서는 보장하지 않는다.

두 실행의 평균 오차 벡터 크기가 모두 0.02°를 초과한 76구간:

- 방향 코사인 중앙값 0.98494 (1에 가까울수록 같은 방향)
- 내적이 양수인 구간 72/76 = 94.7%

관절별로는 두 실행 모두 |평균 오차| >0.02°인 구간에서 부호 일치: J1 100%(44구간), J2 98.1%(54), J3 100%(48), J4 95.3%(43), J5 100%(45), J6 100%(40). 0.02°는 미소 오차의 불안정한 부호를 제외하기 위한 분석 기준이며 센서 정확도 인증값이 아니다. 동일 방향으로 반복되는 경향은 있으나 2회만으로 통계적 반복 정밀도나 보정 가능성을 확정할 수 없다.

jb2 두 쌍은 같은 기준을 넘는 공통 구간이 없어 이 기준으로 방향성을 판단하지 않았다. 그래프 real_servo_signed_errors.png 및 원자료 집계 CSV/JSON 참조. 원래 실행 폴더의 분석 결과는 수정하지 않았다.

재현 (프로젝트 루트):

```
python3 rb_servo_gui/docs/report/repeatability/audit_repeats.py
venv/bin/python rb_servo_gui/docs/report/repeatability/analyze_direction.py
```

평가 지표 해설 (산출식 + 그래프)

> 매칭 방식 업데이트: 현재 분석은 극값 기준점 + 구간 DTW를 사용합니다. 아래 기존 DTW 설명과 그림은 이전 방식입니다. 새 매칭과 파일 형식은 README를 참고하세요.

이 프로그램이 계산하는 **모든 평가 지표**의 정의, 산출식, 읽는 법을 정리했습니다. 수식은 그림(PNG)으로 조판했고, 지표마다 실제 실행 기록으로 만든 설명 그래프를 붙였습니다. (마크다운 뷰어의 수식 지원 여부와 상관없이 그대로 보입니다. 그림을 다시 만드는 방법은 맨 아래에 있습니다.)

- 대상 코드: `analysis.py` (`analyze`, `motion_window`, `ordered_match`, `refine_pairs`, `coverage`, `velocity`, `smoothness`)
- 설정값(합격선): `settings.py`
- 결과 파일의 이름: 실행 폴더의 `compare_*.json`(지표와 판정), `summary.csv`(실행마다 한 줄)

0. 표기와 예제 실행

기호	뜻
`q_i`	모션 구간에서 측정한 6관절 각도 벡터 (i = 1..N), 단위 deg
`r_k`	원본(기준) 궤적의 점, `r_N` 은 마지막 점, `r_0` 은 첫 점
`p_i`	측정 샘플 `i` 와 DTW 로 짝지은 원본 경로 위의 점 (선분 위의 연속 위치)
`p_i` (진행도)	그 점이 원본 경로 길이의 몇 % 지점인지 (0~1). 문맥으로 구분
`L`	원본 경로 전체 길이 (6관절 공간의 L2 호길이, deg)
`t_cmd`	추종 지령(jb2 는 `move_jb2_run`, servo 는 첫 `move_servo_j`)을 보낸 시각

설명 그래프는 아래 두 실행으로 그렸습니다.

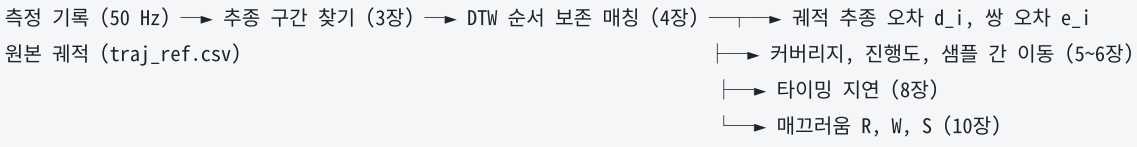
용도	실행	내용
주 예제	`20261008_162955_servo_step10`	`waypoints_robot2.csv` , servo, 시뮬레이션, 지령 간격 0.2 s
리스트 구성 시간	`20261008_165824_jb2_step10`	jb2 (리스트 구성 시간 `t_build` 가 있는 실행)
종료 오버슈트	`20261008_132802_servo_step10`	마지막 점을 반복하지 않아 종료 오차가 큰 실행

1. 지표 한눈에

분류	지표	기호	단위	판정에 사용	합격선 (기본값)
시간	추종 시간	`T_track`	s	아니오	-
시간	지령→모션 시작	`t_latency`	s	아니오	-
시간	지령→완료	`t_total`	s	아니오	-
시간	리스트 구성 시간 (jb2)	`t_build`	s	아니오	-
시간	원본 대비 배율	`ratio`	-	아니오	-
궤적 추종 오차	평균, rms, p95, 최대	`d_mean` , `d_rms` , `d_p95` , `d_max`	deg	p95, 최대	p95 ≤ 1.0, 최대 ≤ 2.0
궤적 추종 오차	각도 오차 RMSE (매칭 후 / 전)	`RMSE_after` , `RMSE_before`	deg	아니오	-
커버리지	평균, p95, 최대	`c_mean` , `c_p95` , `c_max`	deg	최대	≤ 2.0
진행도	경로 진행도	`progress`	0~1	예	≥ 0.99
샘플 간 이동	최대 이동	`Δ_max`	deg	예	≤ 5.0
시작/종료	시작 각 오차, 종료 각 오차	`e_start` , `e_end`	deg	예 (둘 중 큰 값)	≤ 0.5
타이밍	지연 범위, 늦음/빠름 비율	`Δτ` , `late` , `early`	비율	아니오 (참고)	표시 기준 0.03
속도	관절 최대 속도, 속력 최대/평균	`v_max,joint` , `v_max,norm` , `v_mean,norm`	deg/s	아니오	-
링크 표면 (URDF 지정 시)	최악 점 변위, Chamfer, Hausdorff	`D` , `CD` , `HD`	mm	아니오 (참고)	-
매끄러움	속도 리플	`R`	비율	예	≤ 0.15
매끄러움	횡방향 흔들림	`W`	deg	예	≤ 0.30
매끄러움	정체 비율	`S`	비율	예	≤ 0.02
매끄러움	거친 구간 비율, 최대 점수	`rough_ratio` , `rough_score_max`	비율, -	아니오 (참고)	점수 1 = 기준

판정은 **9개 항목**(표에서 "예"인 것: 추종 오차 p95, 최대, 커버리지, 진행도, 샘플 간 이동, 시작/종료 오차, 리플, 흔들림, 정체)이 모두 통과해야 PASS 입니다. 합격선은 근거 없이 잡은 **초기값**이라 응용에 맞게 조정해야 합니다.

2. 처리 흐름


측정 기록 (50 Hz)
추종 구간 찾기 (3장)
DTW 순서 보존 매칭 (4장)
원본 궤적 (traj_ref.csv)
궤적 추종 오차 d_i, 쌍 오차 e_i
커버리지, 진행도, 샘플 간 이동 (5~6장)
타이밍 지연 (8장)
매끄러움 R, W, S (10장)
측정 기록
속도 (9장), 마지막/직전 안정 구간
시작/종료 각 오차 (7장)

3. 시간 지표 (추종 구간과 추종 시간)

추종 구간은 모션이 실제로 일어난 부분입니다. 시작과 끝을 다음과 같이 정합니다.

$$i_s = \min\{\, i : \max_j |q_{ij} - q_{0j}| > \theta_{move} \} - 1$$

$$i_e = \min\{\, i : \max_j \left|q_{kj} - q_j^{end}\right| < \varepsilon_{reach} \ \text{ for all } k \geq i \,\}, \quad q^{end} = \text{median (last max}(t_{hold},\, 0.3\ s))$$

- 시작: 시작 자세 `q_0` 에서 `θ_move` (기본 0.2°) 넘게 벗어나기 **직전** 샘플.
- 종료: 이후 기록 끝까지 **로봇이 멈춘 자세**(마지막 `max(t_hold, 0.3)` 초의 중앙값 `q^end`)의 `ε_reach` (기본 0.5°) 이내에 머무는 첫 샘플.

목표가 아니라 **로봇이 실제로 멈춘 자세**를 기준으로 하므로, 종료 오차가 커도(서보 오버슈트 등) 추종 시간이 기록 끝까지 늘어나지 않습니다.

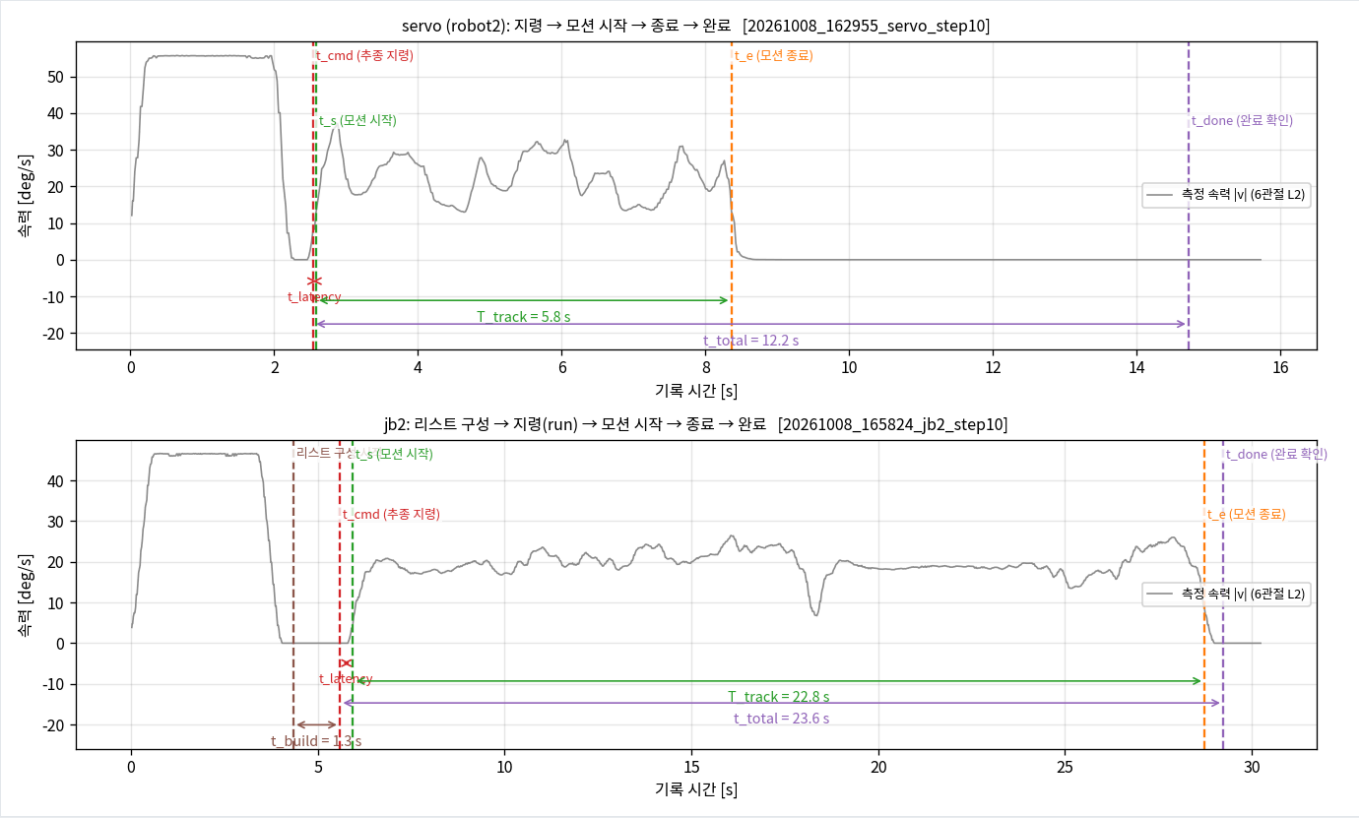
$$T_{track} = t_e - t_s$$

$$t_{latency} = \max(t_s - t_{cmd},\ 0)$$

$$t_{total} = t_{done} - t_{cmd} \qquad (\text{if no } t_{done} :\ t_e \text{ is used})$$

$$t_{build} = t_{cmd} - t_{list\ start} \quad (\text{jb2})$$

$$\text{ratio} = \frac{T_{track}}{T_{ref}}$$



시간 지표 그래프

지표	읽는 법
`T_track`	모션이 일어난 시간. 예제 servo 는 5.78 s (원본 5.78 s, 배율 1.0), jb2 는 22.80 s (원본 14.67 s)
`t_latency`	지령을 보낸 뒤 움직이기까지의 지연. 예제 servo 0.04 s, jb2 0.35 s. 음수가 되지 않게 0 으로 자릅니다
`t_total`	지령부터 **완료를 확인한** 시각까지. 모션이 끝난 뒤 도달을 확인하는 시간과, servo 는 마지막 점을 반복 전송하는 시간(`final_hold` × 명령 주기)이 포함됩니다. 예제 servo 는 `final_hold = 30` 이라 12.18 s 입니다
`t_build`	jb2 에서 `move_jb2_add` 로 포인트를 쌓는 데 걸린 시간. 예제 1.25 s
`ratio`	`T_track / T_ref` . 1 보다 크면 원본보다 느리게 실행됨 (예제 jb2 는 약 1.55)

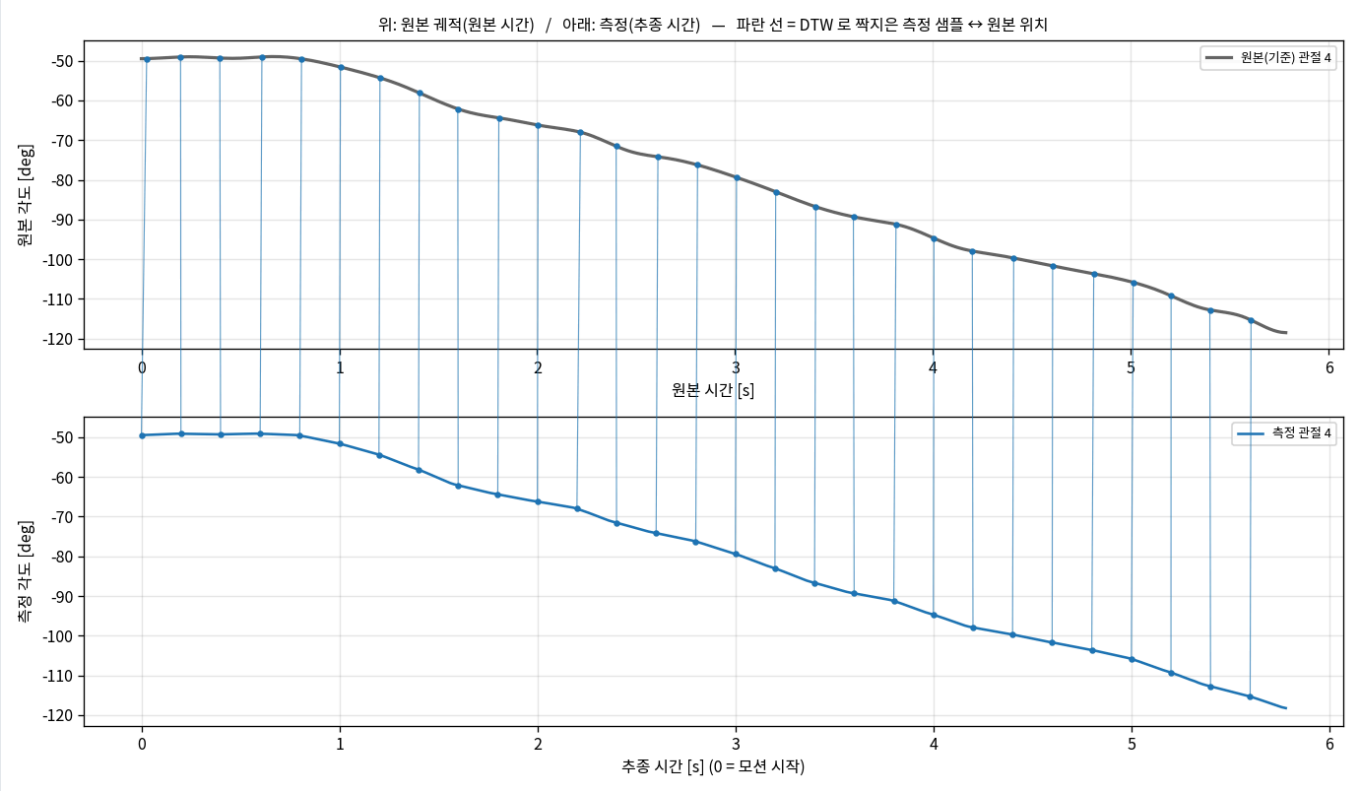
4. 궤적 추종 오차 (DTW 매칭쌍 기준)

측정과 원본을 같은 시각끼리 비교하면 **시간이 어긋난 것까지 오차**가 됩니다. 그래서 먼저 측정 샘플마다 원본 경로 위의 짝 `p_i` 를 정합니다.

- 짝을 정하는 방법은 **순서를 보존하는 DTW**입니다 (측정 샘플이 시간 순서로 진행하면 짝도 같은 순서로 진행).

다른 시각에 우연히 같은 각도를 지나가더라도 거기에 짝지어지지 않고, 시간 지연이나 속도 차이는 허용합니다.

- DTW 가 정한 원본 정점의 양옆 선분 위에서 가장 가까운 점을 짝으로 삼아 정점 단위 양자화 오차를 없앱니다.



DTW 매칭 그림

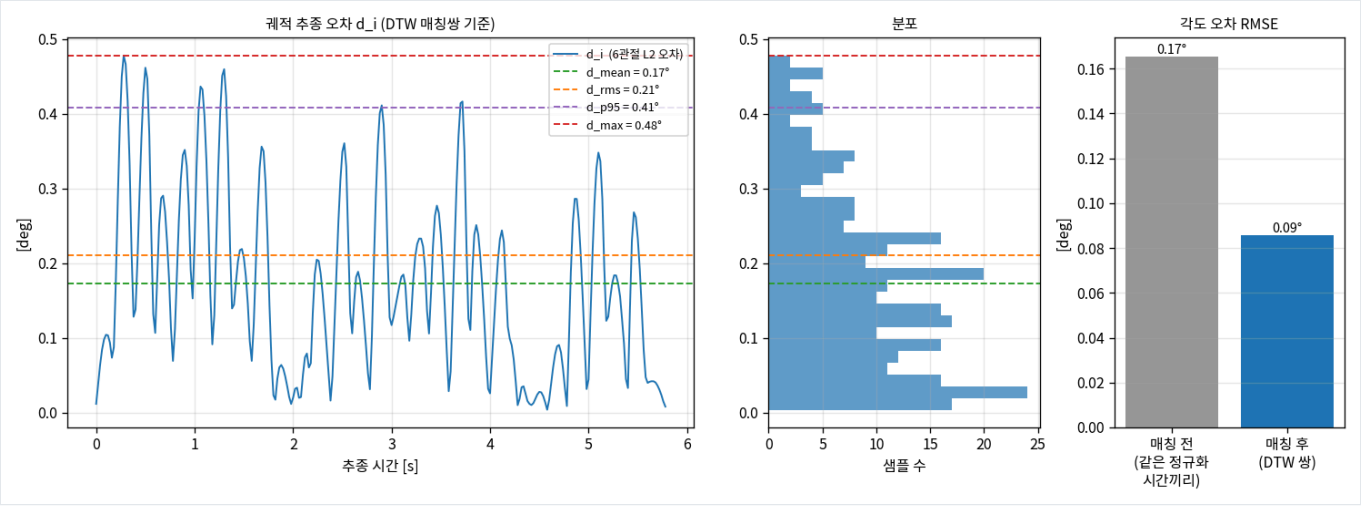
짝이 정해지면 오차 벡터와 크기를 다음과 같이 정의합니다.

$$\mathbf{e}_i = \mathbf{q}_i - \mathbf{p}_i, \qquad d_i = \|\mathbf{e}_i\|_2 = \sqrt{\sum_{j=1}^{6} e_{ij}^2}$$

$$d_{mean} = \frac{1}{N}\sum_i d_i, \quad d_{rms} = \sqrt{\frac{1}{N}\sum_i d_i^2}, \quad d_{p95} = P_{95}(d_i), \quad d_{max} = \max_i d_i$$

$$\mathrm{RMSE}_{after} = \sqrt{\frac{1}{6N}\sum_{i=1}^{N}\sum_{j=1}^{6} e_{ij}^2}$$

$$\mathrm{RMSE}_{before} = \sqrt{\frac{1}{6M}\sum_{k=1}^{M}\sum_{j=1}^{6} (q_{kj}^{norm} - r_{kj}^{norm})^2}$$



궤적 추종 오차 그래프

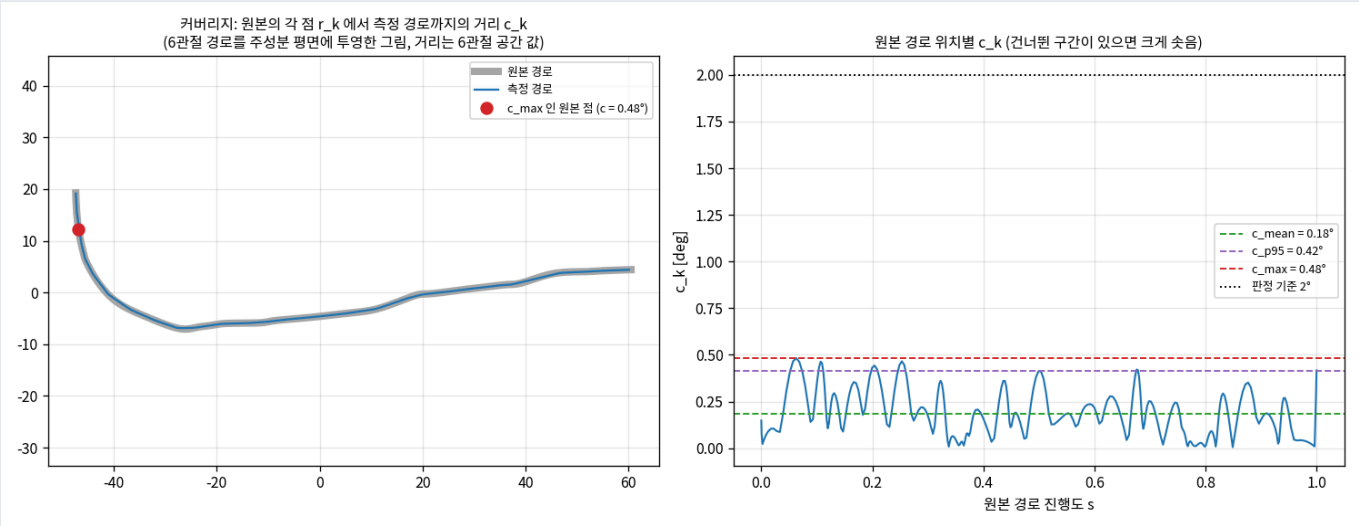
- **판정에는 `d_p95` 와 `d_max` 를 씁니다.** 예제 servo: `d_mean` 0.17°, `d_rms` 0.21°, `d_p95` 0.41°, `d_max` 0.48°.
- 오차 `e_i` 는 경로에 **수직인 방향**입니다(실기록에서 경로 접선과의 |cos| 중앙값이 0.000). 경로 위에서 앞서거나 뒤처진 것은 DTW 가 흡수합니다.
- `RMSE_before` 는 시작과 종료만 맞춘 정규화 시간(M = 500점)에서 **같은 시각끼리** 비교한 값, `RMSE_after` 는 DTW 쌍 기준입니다. 예제는 0.17° 에서 0.09° 로 줄었습니다.

두 값의 차이가 클수록 오차의 상당 부분이 경로 이탈이 아니라 시간 어긋남이었다는 뜻입니다.

5. 커버리지 (건너뛴 구간 검출)

`d_i` 는 "측정 → 원본" 방향의 거리라서, 로봇이 경로의 일부를 **건너뛰어도** 작게 나올 수 있습니다. 반대로 "원본의 각 점 → 측정 경로" 거리를 재면 건너뛴 구간이 드러납니다.

$$c_k = \min_{\ell \in W(k)} \operatorname{dist}(\mathbf{r}_k,\ [\mathbf{q}_\ell, \mathbf{q}_{\ell+1}]), \quad c_{mean},\ c_{p95},\ c_{max}$$



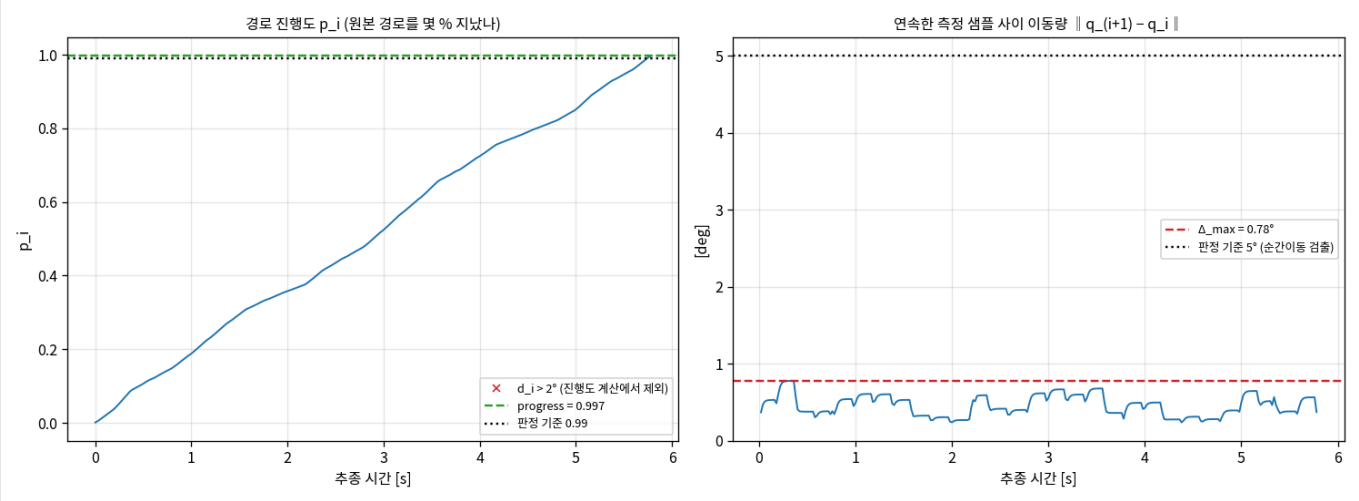
커버리지 그래프

- `W(k)` 는 원본 점 `r_k` 에 DTW 로 짝지어진 측정 구간 근처의 선분들입니다. **전체 측정 경로에서 가장 가까운 점을 찾지 않는** 이유는, 다른 시각에 같은 각도를 지나간 우연한 일치를 인정하지 않기 위해서입니다.
- 한 번도 가지 않은 구간이 있으면 그 구간의 `c_k` 가 크게 솟고 `c_max` 가 커집니다. 예제는 `c_mean` 0.18°, `c_p95` 0.42°, `c_max` 0.48° 입니다.

6. 경로 진행도와 샘플 간 이동

$$\text{progress} = \max\{\, p_i : \; d_i \le d_{th} \,\}, \qquad p_i = \frac{\text{arc length up to matched point}}{L}$$

$$\Delta_{max} = \max_i \|\mathbf{q}_{i+1} - \mathbf{q}_i\|_2$$



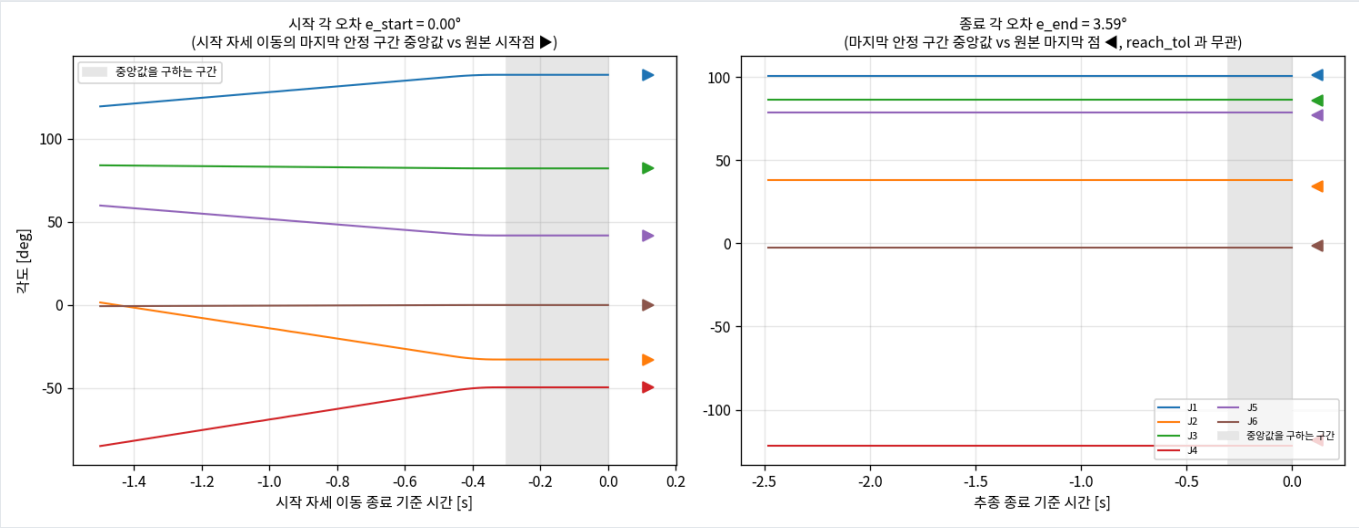
진행도와 샘플 간 이동

- **진행도**: 원본 경로를 어디까지 갔는지. 경로 편차가 `d_th`(= 판정 기준 2°)를 넘는 샘플은 짝이 의심스러워 **계산에서 제외**합니다(왼쪽 그림의 ×). 예제 0.997.
- **샘플 간 이동**: 연속한 두 측정 샘플 사이 이동량. 갑자기 큰 값이 나오면 순간이동이나 건너뜀입니다. 예제 최대 0.78°, 기준 5°.

7. 시작/종료 각 오차

$$e_{end} = \max_j \left| \operatorname{median}_{t \geq t_{last} - \max(t_{hold},\, 0.3)} q_j(t) - r_{N,j} \right|$$

$$e_{start} = \max_j \left| \operatorname{median}\, q_j^{pre} - r_{0,j} \right| \quad (\text{pre} = \text{move_to_start phase})$$



시작/종료 각 오차 그래프

- **종료 각 오차**는 추종 구간 끝의 **마지막 안정 구간 중앙값**과 원본 마지막 점의 관절별 차이 중 최댓값입니다. 모션 종료 판정(`ε_reach`)과는 **무관하게** 따로 잽니다.
- **시작 각 오차**는 추종 직전 단계(시작 자세 이동)의 마지막 안정 구간으로 잽니다. 추종 구간의 첫 샘플은 이미 움직이기 시작한 뒤일 수 있기 때문입니다.
- 위 그림은 마지막 점을 반복하지 않았던 실행(`132802`)이라 종료 오차가 3.59° 로 큽니다. 같은 궤적을 `final_hold` 로 돌린 예제 실행은 0.00° 였습니다.

(servo 는 마지막 지령 뒤에 마지막 속도 × `t2` 만큼 더 나아가 멈추는 것이 관측되었습니다. 이 동작은 공식 문서에 없습니다.)

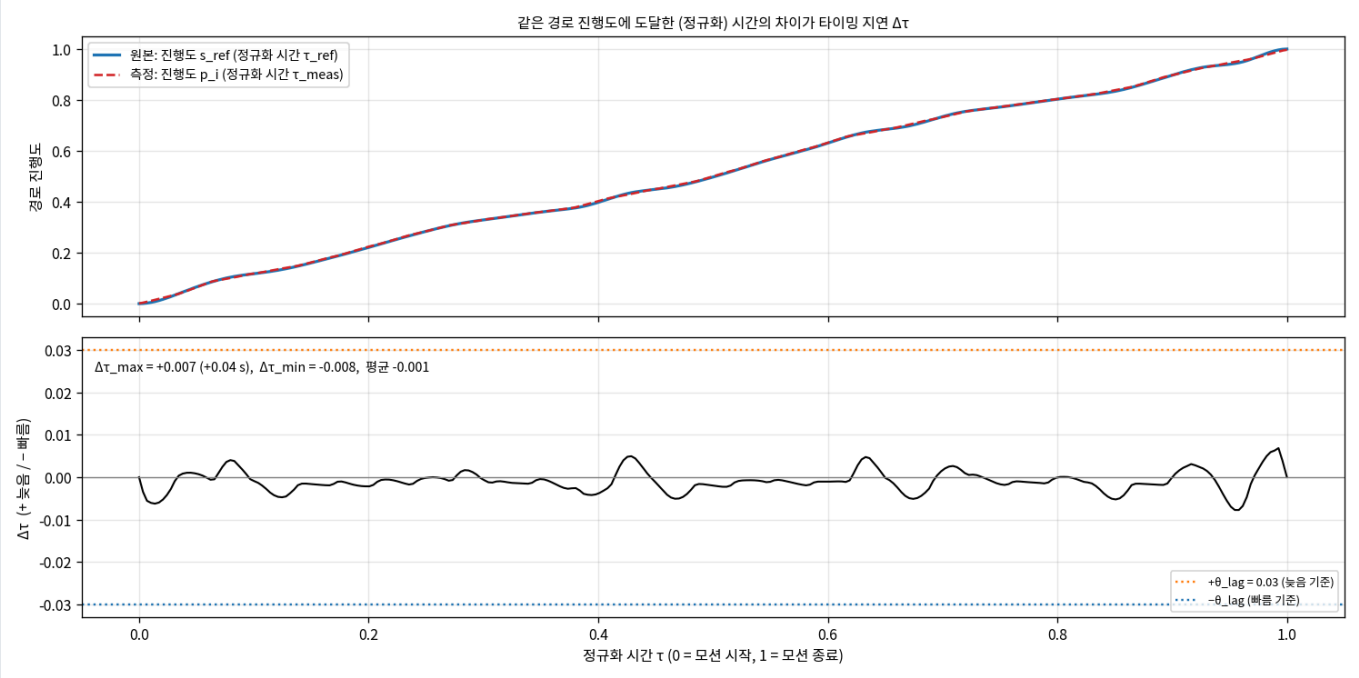
- 판정은 `max(e_start, e_end) ≤ 0.5°` 입니다.

8. 타이밍 지연 (참고 지표, 판정에 쓰지 않음)

같은 **경로 진행도**에 도달한 (정규화) 시간이 원본보다 늦었는지 빨랐는지를 봅니다.

$$\Delta\tau(s) = \tau_{meas}(s) - \tau_{ref}(s) \qquad (+:\ \text{late},\ \ -:\ \text{early})$$

$$\Delta t_{max} = \Delta\tau_{max} \cdot T_{track}, \qquad \text{late} = \frac{\#\{\Delta\tau > \theta_{lag}\}}{M}, \quad \text{early} = \frac{\#\{\Delta\tau < -\theta_{lag}\}}{M}$$



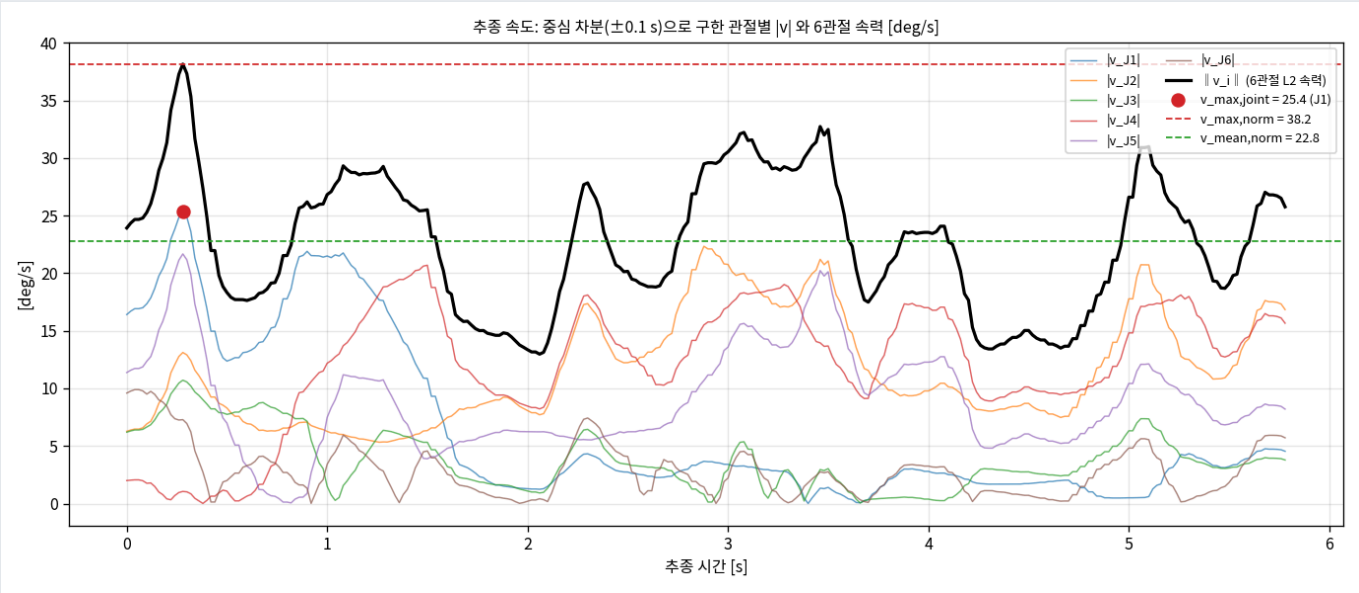
타이밍 지연 그래프

- 정규화 시간은 모션 시작을 0, 종료를 1 로 한 시간입니다. `+` 는 측정이 **늦음**, `−` 는 **빠름**입니다. `Δt_max` 는 비율을 추종 시간으로 환산한 초 값입니다.
- 예제 servo: `Δτ` 범위 −0.008 ~ +0.007, 평균 −0.001 (최대 +0.04 s). 거의 일치합니다.
- 실행 시간이 크게 다른 jb2 에서는 비율로 보는 것이 의미가 있습니다(원본 14.7 s 를 22.8 s 에 걸쳐 실행해도 진행 방식이 같으면 `Δτ` 가 작게 나옵니다).
- 지연을 합격/불합격으로 쓰지 않는 이유는, 시간이 어긋나도 경로를 정확히 따라갔는지를 먼저 평가하려는 것입니다.

9. 추종 속도

$$v_{ij} = \frac{q_{hj} - q_{lj}}{t_h - t_l}, \quad l = \text{first } (t \geq t_i - 0.1), \quad h = \text{last } (t \leq t_i + 0.1)$$

$$v_{max,\,joint} = \max_{i,j} |v_{ij}|, \quad \mathbf{v}_i \text{ norm} = \|\mathbf{v}_i\|_2, \quad v_{max,\,norm} = \max_i \|\mathbf{v}_i\|_2, \quad v_{mean,\,norm} = \frac{1}{N}\sum_i \|\mathbf{v}_i\|_2$$



추종 속도 그래프

- 속도는 측정 각도를 `t ± 0.1 s` 구간의 양 끝으로 **중심 차분**해 구합니다 (샘플 시각 지터에 강하게 하려는 것).

이 차분은 저역통과(sinc) 특성이라 5 Hz 이상의 빠른 요동은 거의 걸러집니다.

- 예제 servo: 관절 최대 25.4 deg/s, 6관절 속력 최대 38.2 / 평균 22.8 deg/s.
- 결과 화면과 저장 그래프에서는 방향이 아닌 크기를 보려고 **절대값**으로 그립니다. `velocity_*.csv` 는 부호 있는 값입니다.

10. 매끄러움 (속도 리플, 횡방향 흔들림, 정체 비율)

경로 자체가 꺾였거나 매끄럽지 않은 것은 감점하지 않도록, 경로 모양과 무관한 양만 봅니다. jerk 는 측정 노이즈에 지나치게 민감해서 쓰지 않습니다.

10.1 공통 준비: 경로를 따라간 속력과 추세

$$s_i = p_i \cdot L, \qquad v_i^s = \frac{s_h - s_l}{t_h - t_l} \quad (\text{same } \pm 0.1 \text{ s window})$$

$$w = \text{clip}\left(\frac{T_{track}}{10},\ 0.3,\ 2.0\right)\ [s], \qquad n = \max\left(\text{round}\left(\frac{w}{\Delta t}\right),\ 3\right)$$

$$hp_i = v_i^s - \text{MA}_n(v^s)_i, \qquad \mathbf{e}_i^{hp} = \mathbf{e}_i - \text{MA}_n(\mathbf{e})_i$$

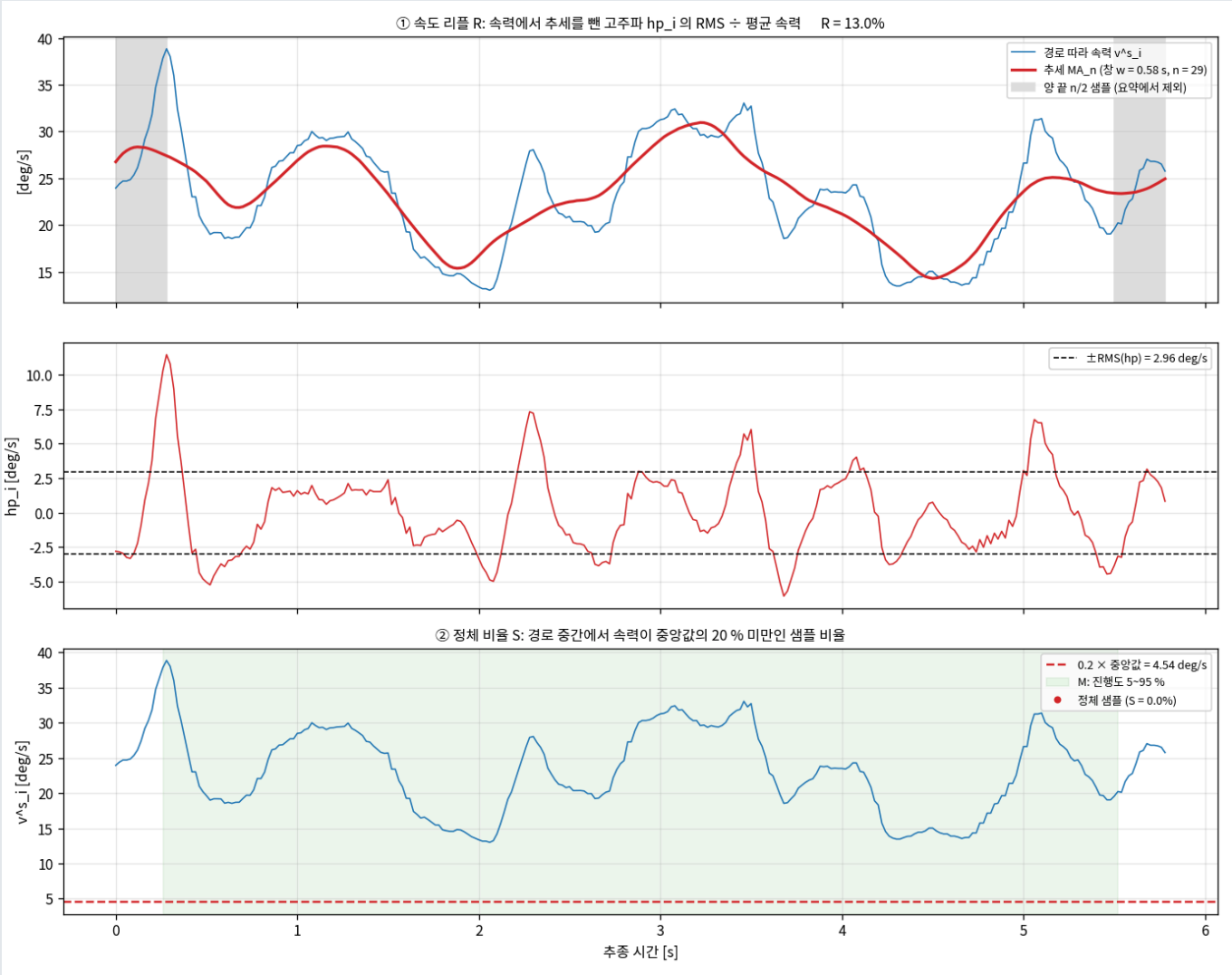
$$core = \{\, i :\ n/2 \le i < N - n/2 \,\} \quad (N \le 2n :\ \text{all})$$

- `v^s_i` 는 **경로 위에서 간 거리(호길이)의 변화율**입니다. 경로가 꺾여도 이 값은 그대로라서 경로 모양의 영향을 받지 않습니다. 진행도가 줄지 않게 처리하므로 $v^s \ge 0$ 입니다.
- `MA_n` 은 길이 `n` 샘플의 중심 이동평균입니다(양 끝은 끝값으로 연장). 창 `w` 는 추종 시간의 1/10 이고 0.3~2.0 s 로 제한됩니다. 예제는 `w` = 0.58 s, `n` = 29.
- **고주파 = 원래 값 − 추세**. 추세보다 느린 변화는 "정상적인 가감속"으로 보고 리플에 넣지 않습니다.
- 양 끝 `n/2` 샘플(`core` 밖)은 이동평균의 가장자리 효과 때문에 요약 값에서 제외합니다.

10.2 속도 리플 `R` 과 정체 비율 `S`

$$R = \frac{\sqrt{\text{mean}_{i \in core}\ hp_i^2}}{\text{mean}_{i \in core}\ v_i^s}$$

$$S = \frac{\#\{\, i \in \mathcal{M} :\ v_i^s < 0.2 \cdot \text{median}(v_{\mathcal{M}}^s) \,\}}{|\mathcal{M}|}, \qquad \mathcal{M} = \{\, i :\ 0.05 < p_i < 0.95 \,\}$$

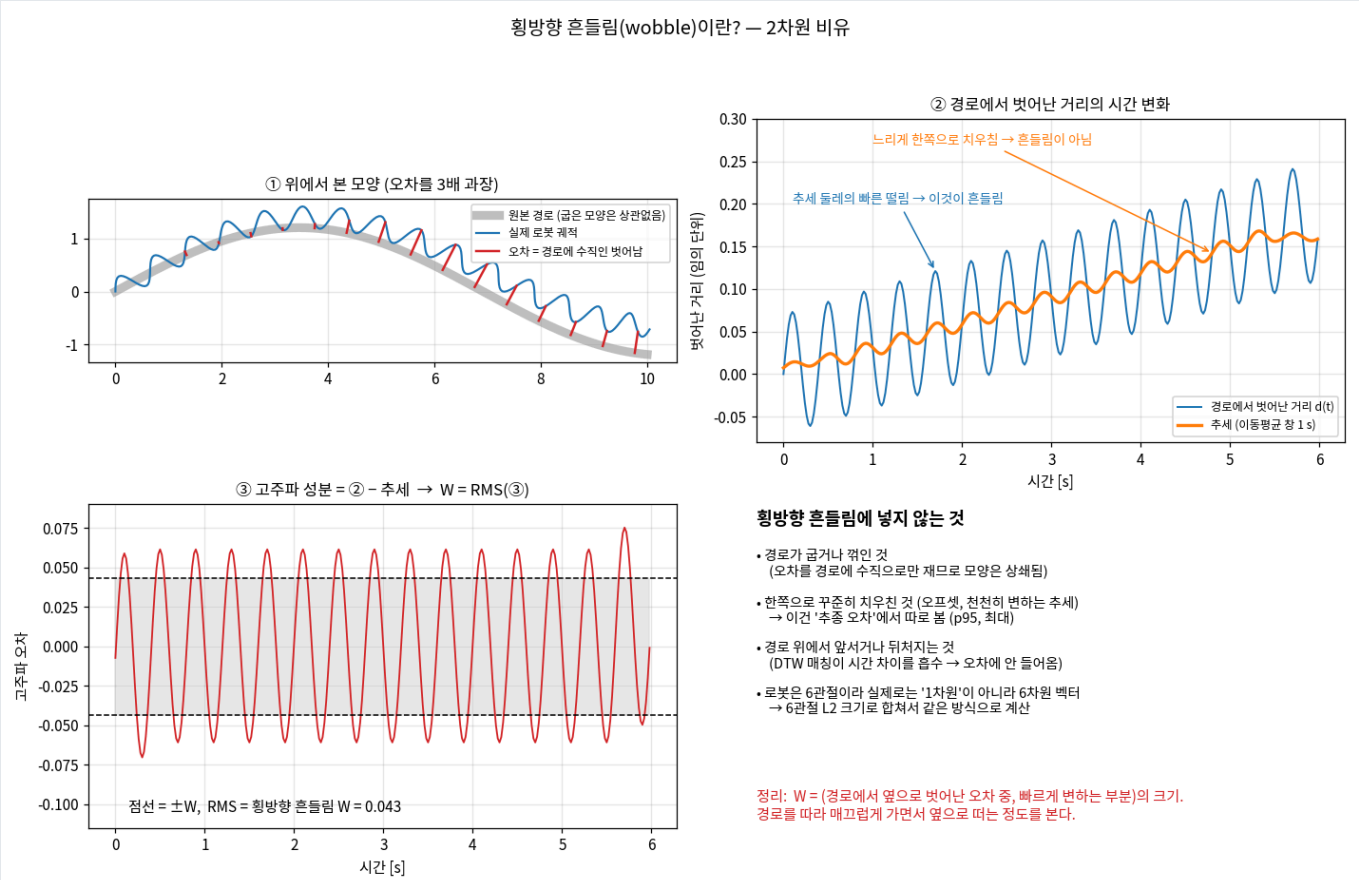


속도 리플 그래프

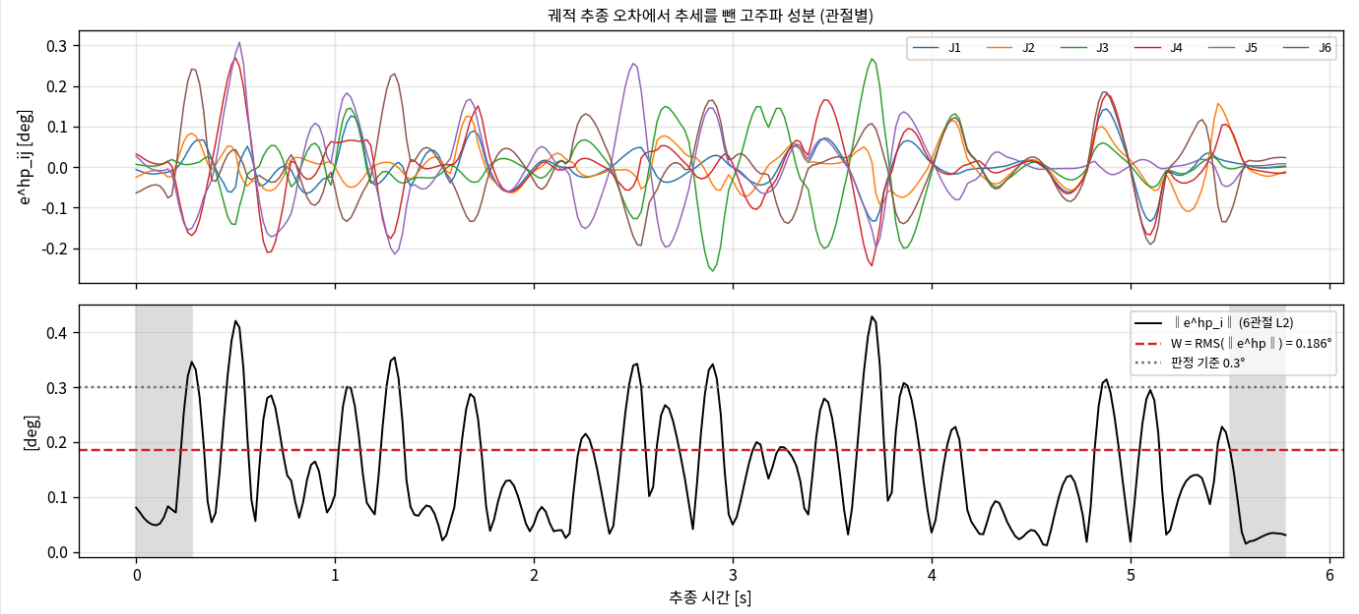
- `R` 은 속력이 추세 둘레에서 얼마나 요동하는지를 평균 속력에 대한 **비율**로 나타냅니다. `R = 15 %` 는 평균 속력의 15 % 정도 요동한다는 뜻입니다. 예제 `R` = 13.0 %.
- `S` 는 경로 중간(진행도 5~95 %, 시작과 끝의 정상적인 가감속 제외)에서 속력이 **중앙값의 20 % 미만**으로 떨어진 샘플의 비율입니다. 멈칫거림을 잡습니다. 예제 `S` = 0 %.

10.3 횡방향 흔들림 `W`

$$W = \sqrt{\mathrm{mean}_{i \in core} \left\| \mathbf{e}_i^{hp} \right\|_2^2} \ \ [\mathrm{deg}]$$



횡방향 흔들림 개념



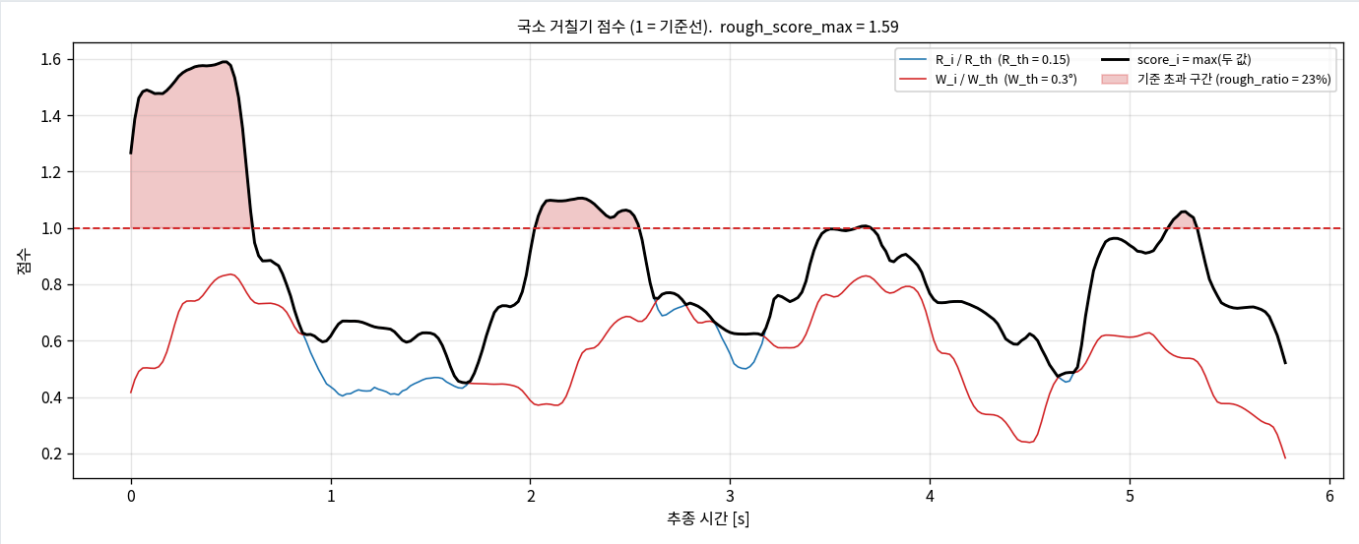
횡방향 흔들림 실측

- "횡방향"은 **경로에 수직인 방향**입니다(6관절 공간에서의 방향).
- 경로에서 옆으로 벗어난 정도를 느린 변화(추세)와 빠른 변화(떨림)로 나눠, 빠른 변화의 크기가 `W` 입니다. 한쪽으로 꾸준히 치우친 오프셋은 추세라서 `W` 가 아니라 4장의 추종 오차로 잡힙니다.
- 읽는 법: `W = 0.2°` 이면 경로 옆 벗어남이 추세 둘레에서 RMS 0.2° 정도 떤다는 뜻입니다. 예제 `W` = 0.186°.

10.4 국소 거칠기 점수와 거친 구간

$$\text{score}_i = \max\left(\frac{R_i}{R_{th}},\ \frac{W_i}{W_{th}}\right), \quad R_i = \frac{\sqrt{\text{MA}_n(hp^2)_i}}{\text{mean } v^s}, \quad W_i = \sqrt{\text{MA}_n(\|\mathbf{e}^{hp}\|^2)_i}$$

$$\text{rough_ratio} = \frac{\#\{\, i :\ \text{score}_i > 1 \,\}}{N}, \qquad \text{rough_score_max} = \max_i \text{score}_i$$



거칠기 점수 그래프

- 위 지표를 **샘플마다 이동 창 RMS** 로 바꿔서 경로의 어디가 거친지 봅니다. 점수 1 이 기준선이고, 결과 표의 "거친구간" 열과 `smooth_map_*.png` 의 색이 이 값입니다.
- 예제는 거친 구간 비율 23.4 %, 최대 점수 1.59 입니다. 판정에는 쓰지 않는 참고 정보입니다.

11. 링크 표면 오차 (URDF 지정 시, 참고 지표)

관절각 오차는 **로봇 형상에서 표면이 실제로 얼마나 떨어지는지(mm)** 로 바꿔 봐야 의미가 분명합니다. 같은 1° 오차도 어깨 관절이면 20 mm 가까이, 손목 끝 관절이면 1 mm 미만입니다.

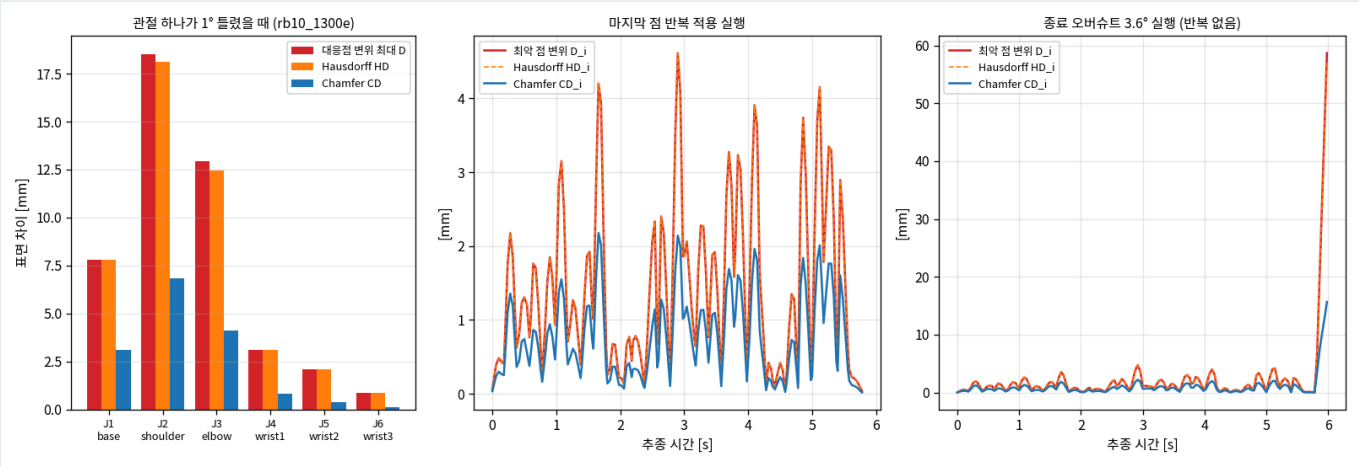
방법. URDF 의 링크 메시(충돌 STL)에서 표면 점을 링크당 `surface_points` 개(기본 600) 뽑습니다. 추종 구간에서 매칭쌍 `surface_samples` 개(기본 200, 균등 추출)를 골라, 각 쌍마다

- 측정 자세 `q_i` 와 DTW 로 짝지은 원본 자세 `p_i` 를 **각각 FK 로 배치**해 두 표면 점집합 `A`(측정), `B`(원본)를 만들고 비교합니다.
- 전체 로봇 지표(Chamfer, Hausdorff, 대응점 변위의 전체 최대)는 **팔 관절과 함께 움직이는 링크의 점집합**으로 계산하고, 링크별 최대 변위도 구합니다. 레일, 캐리지 같은 박스 구조물과 팔 베이스(link0)는 팔 오차로 움직이지 않아서, 넣으면 평균을 희석하므로 제외합니다.
- 외부축 등 앞쪽 관절은 0 으로 두는데, 두 자세에 똑같이 적용되므로 상대 차이에는 영향이 없습니다.
- 길이는 URDF 의 m 를 mm 로 환산해 보고합니다.

$$D_i = \max_v \| \mathbf{x}_v(\mathbf{q}_i) - \mathbf{x}_v(\mathbf{p}_i) \|_2, \qquad D_{max} = \max_i D_i\,, \quad D_{p95} = P_{95}(D_i), \quad D_{mean} = \frac{1}{N}\sum_i D_i$$

$$\mathrm{CD}_i = \frac{1}{2}\left(\frac{1}{|A|}\sum_{a\in A} \min_{b\in B} \|a-b\| + \frac{1}{|B|}\sum_{b\in B} \min_{a\in A} \|a-b\| \right)$$

$$\mathrm{HD}_i = \max\left(\max_{a\in A} \min_{b\in B} \|a-b\|,\ \max_{b\in B} \min_{a\in A} \|a-b\| \right)$$



링크 표면 오차 그래프

셋 중 무엇을 봐야 하나

지표	무엇을 재나	장점	주의
대응점 변위 `D`	**같은 메시 점**이 두 자세에서 떨어진 거리의 최댓값	최악의 점을 정확히 보여줌. 링크별 최대도 구함	대응점이 있어야 함 (여기서는 같은 URDF 를 쓰므로 항상 있음)
Hausdorff `HD`	두 표면 사이 최근접 거리의 **최댓값**	대응점이 필요 없음, 최악의 경우를 보여줌	표면이 자기 위를 미끄러지면 `D` 보다 작아질 수 있음
Chamfer `CD`	두 표면 사이 최근접 거리의 **평균**	전체적인 표면 일치 정도	**최악 위치를 크게 낮춰 봄**

- 이 문서의 예시에서 `D` 와 `HD` 는 거의 같고 `CD` 는 훨씬 작습니다. 어깨 관절 1° 오차는 `D` 18.5 mm, `HD` 18.1 mm, `CD` 6.8 mm 입니다.

종료 오버슈트가 3.6° 인 실행은 `D` 최대 58.7 mm 인데 `CD` 최대는 15.7 mm 라서, **`CD` 만 보면 최악 위치의 오차를 약 3.7 배 낮게 평가합니다.**

- 그래서 **주 지표는 `D`(최악 점 변위)** 이고 Chamfer, Hausdorff 는 함께 보고합니다. 충돌이나 간극 관점에서는 평균이 아닌 최악이 중요합니다.

- 결과는 시간에 대해 `D_i` 의 최대, p95, 평균, Chamfer 의 최대와 평균, Hausdorff 최대, 링크별 최대 변위, 최악 링크로 정리합니다.

예제 servo(마지막 점 반복 적용)는 `D` 최대 4.6 mm, p95 3.6 mm, 평균 1.5 mm, Chamfer 최대 2.19 mm 였습니다.

계산이 맞는지 확인한 것. 한 관절만 1° 돌렸을 때의 `D` 가 기하학적 예측값(회전축에서 가장 먼 점까지의 거리 × 각도, `2·r·sin(θ/2)`)과 6관절 모두 소수 둘째 자리까지 일치했습니다 (rb10_1300e, robot1 시작 자세).

화면으로 보기. 결과 탭의 ★ **메시 차이** 항목에서, 시간 그래프를 클릭해 고른 시점의 두 자세를 메시로 겹쳐 볼 수 있습니다 (색 = 같은 점의 실제 변위, 형상은 실제 크기 ×1). 자세한 사용법은 README 참고.

쓰는 법. 설정의 "URDF" 칸에 URDF 파일을 지정합니다(비우면 계산하지 않음). 결과는 `compare_*.json` 의 `surf_*` 항목과 `surface_*.csv` (시간별 `D` , `CD` , `HD`), 결과 탭의 상세 패널에 나옵니다. 링크별 최대 변위는 JSON 의 `surface_links_max_mm` 에 있습니다. 합격선은 정하지 않았고 참고용입니다.

모델에 따른 차이. 같은 측정값이라도 팔이 클수록 mm 값이 큽니다. 사용자의 RT 궤적(`waypoints_sample`) 실행에서 `rb20_1900es_RT` 는 `rb10_1300e` 보다 약 1.6배 큰 값을 줍니다(REAL servo 실행: 대응점 변위 최대 15.4 mm 대 9.5 mm). 어느 URDF 를 쓰는지가 결과에 직접 영향을 줍니다.

한계

- **URDF 가 실제 로봇과 같아야** 합니다. 제어기 설정(`controller.cfg`)에는 두 로봇이 모두 rb10_1300e 이고, `rb10_1300e.urdf` , `rb10_1300e_RT.urdf` , `rb10_1300e_DDA.urdf` 의 팔 기구학은 같습니다(링크 원점 간 거리 차이 0.000 mm). `rb20_1900es_RT.urdf` 는 다른 팔입니다(차이 약 777 mm). 관절 영점, 부호, 링크 치수가 다르면 값이 달라집니다. 이 도구는 이를 확인하지 못합니다 (`reqdata` 의 `tcp_pos` 와 FK 를 비교하면 확인할 수 있지만 아직 하지 않았습니다).
- 메시는 충돌 STL 을 샘플링한 점입니다. 점이 성기면 Chamfer, Hausdorff 가 조금 흔들립니다. `D` 는 같은 점끼리 비교하므로 이 영향이 작습니다.
- 도구와 케이블, 가공물 등 URDF 에 없는 형상은 포함되지 않습니다.

12. 판정 요약

번호	판정 항목	조건	설정키
1	궤적 추종 오차 p95	$d_{p95} \le 1.0$ deg	th_path_p95
2	궤적 추종 오차 최대	$d_{max} \le 2.0$ deg	th_path_max
3	커버리지 최대	$c_{max} \le 2.0$ deg	th_cover
4	경로 진행도	$progress \ge 0.99$	th_progress
5	샘플 간 이동	$\Delta_{max} \le 5.0$ deg	th_step
6	속도 리플	$R \le 0.15$	th_ripple
7	횡방향 흔들림	$W \le 0.30$ deg	th_wobble
8	정체 비율	$S \le 0.02$	th_stall
9	시작/종료 각 오차	$\max(e_{start}, e_{end}) \le 0.5$ deg	th_end

그 밖의 설정: th_lag (타이밍 지연 표시 기준, 판정 미사용), reach_tol (모션 종료 판정 허용 오차), reach_hold, move_thr (모션 시작 판정).

13. 한계와 주의

- **창이 추종 시간에 비례**합니다(매끄러움). 추종 시간이 크게 다른 실행끼리 `R`, `W` 를 직접 비교하기는 어렵습니다.
- 속력은 DTW 로 구한 진행도의 미분이라 **매칭 오차가 리플에 섞입니다**. 실행 간 비교용으로는 괜찮지만 절대값에는 이 오차가 포함됩니다.
- 이상적으로 추종해도 `R` 은 0 이 아닙니다(정상적인 가감속이 창보다 빨리 변할 때, 합성 데이터에서 약 3~6 %).
- 진행도는 감소하지 않게 처리해서 **되돌아가는 움직임**은 매끄러움에서 잡히지 않습니다. 그런 경우는 추종 오차나 진행도에서 드러납니다.
- 시뮬레이션에서는 측정각으로 `jnt_ref` 를 쓰므로(`jnt_ang` 는 0 으로 고정됨) 이 값이 실제 하드웨어를 반영하는지는 확인되지 않았습니다.
- 합격선은 실제 허용 수준에서 정한 값이 아닙니다.

14. 그림과 수식 다시 만들기

```
cd rb_servo_gui
PYTHONPATH=src python docs/make_metric_figures.py \
    --servo   <servo 실행 폴더> \
    --jb2     <jb2 실행 폴더> \
    --overshoot <마지막 점 반복 없이 종료 오버슈트가 있던 servo 실행 폴더>
```

수식은 `make_metric_figures.py` 안의 `FORMULAS` 사전에 있고, 설명 그래프는 현재 `analysis.py` 의 함수로 다시 계산해서 그립니다.

RB 협동로봇 외부 스크립트 인터페이스 정의서 (간이)

- 대상: Rainbow Robotics 협동로봇 제어기, 외부 스크립트 방식 (TCP)
- 근거: 공식 문서 외부 스크립트 API, UI Script 와 이 저장소 도구(`rb_servo_gui`, `jb2_gui`)의 구현·실험
- 표기: **[문서]** 공식 문서에 있는 내용, **[관측]** 실험에서 확인했지만 문서에는 없는 내용 (제어기·펌웨어에 따라 다를 수 있음)
- 예시 값(IP 10.0.2.7, 관절값)은 이 프로젝트의 환경이며, **모션 명령은 반드시 시뮬레이션 모드에서 먼저 시험**

1. 연결

포트	용도	방향	형식
5000	명령 포트	외부 → 제어기 (명령), 제어기 → 외부 (응답)	텍스트 한 줄 = 명령 하나
5001	데이터 포트	`reqdata` 요청 → 이진 상태 패킷 응답	580바이트 고정 길이 [문서: 포트 역할], [관측: 길이·오프셋]

- 두 포트는 **별도 TCP 연결**. 명령 전송과 상태 읽기를 서로 다른 연결(스레드)에서 동시에 해도 됨 [관측]
- 명령은 **받는 즉시 실행**되며 별도 확인 절차가 없음. 모드와 목표값을 보내기 전에 확인
- 제어기 IP 는 제어기 화면(LCD)에 표시된 값

2. 명령 포트 (5000)

2.1 전송 규칙

- 한 줄에 명령 하나, 끝에 개행(`\n`)
- 응답 [문서]: 유효한 명령이면 `The command was executed` , 허용되지 않는 명령이면 `The command is not allowed`
- **여러 줄을 연속 전송할 때**
  - 응답을 기다리지 않고 몰아 보내면 TCP 가 줄을 합치거나 쪼개 구문 오류가 남 [관측]. 증상: 태블릿에 `syntax error at ' ' in this line: move_jb2_add(j`
  - 대응: Nagle 끄기(`TCP_NODELAY`) + 한 줄마다 응답을 받은 뒤 다음 줄 전송 (특히 `move_jb2_add` 반복 구간)
- 관절값: 임시 변수 `jnt[j0, j1, j2, j3, j4, j5]` (deg, 별도 선언 불필요) [문서]

2.2 모드 전환

명령	설명
`pgmode simulation`	시뮬레이션 모드: 로봇은 움직이지 않고 지령값(`jnt_ref`)만 갱신
`pgmode real`	실기 모드: 실제 구동

- 시뮬레이션에서 엔코더값 `jnt_ang` 는 갱신되지 않고 0 으로 고정 [관측] → 시뮬레이션은 `jnt_ref` , 실기는 `jnt_ang` 로 위치를 읽음

2.3 모션 명령

명령	인자	설명
`move_j(J, v, a [, wait])`	J: 관절 목표(deg), v: 속도(deg/s), a: 가속도(deg/s²), wait: 0 대기 안 함 / 1 완료까지 대기 (기본 1)	단일 관절 이동
`move_jb2_clear()`	없음	jb2 점 리스트 초기화
`move_jb2_add(J, v, a, 0, blend)`	4번째 인자는 더미(0), blend: 블렌드율(%)	jb2 리스트에 점 추가
`move_jb2_run([wait])`	wait: 0 이면 시작만 하고 반환, 1 대기 (기본 1)	리스트의 점을 이어서 실행
`move_servo_j(J, t1, t2, gain, alpha)`	아래 표	실시간 서보: 목표 관절각을 계속 갱신

`move_servo_j` 인자 [문서]

인자	의미	범위
J	목표 관절각 (deg)	-360 ~ 360
t1	도달 시간 (s)	≥ 0.002
t2	유지 시간 (s)	0.02 < t2 < 0.2
gain	속도 추종 비율	> 0
alpha	저역 통과 필터	0 < alpha < 1

2.4 작업(task) 제어

명령	설명
`task load <파일명>`	작업 파일(.wsl) 불러오기
`task repeat <횟수>`	반복 횟수
`task play once`	불러온 작업 실행
`task pause` / `task resume_a`	일시 정지 / 재개

명령	설명
`task stop`	작업 정지 요청 (**비상정지가 아님**)

(`resume_b` 도 문서에 있음)

3. 명령 예시

3.1 시퀀스 (한 줄씩, 시뮬레이션 모드)

```
pgmode simulation
move_j(jnt[66.757,-42.499,49.902,-6.537,113.220,0.363], 50, 120)
move_jb2_clear()
move_jb2_add(jnt[66.9632,-42.2424,51.4177,-9.6759,112.9050,0.2800], 50, 120, 0, 100)
move_jb2_add(jnt[67.0438,-41.5110,53.5442,-14.9398,111.9343,0.1912], 50, 120, 0, 100)
move_jb2_run(0)
```

```
pgmode simulation
move_servo_j(jnt[66.9632,-42.2424,51.4177,-9.6759,112.9050,0.2800], 0.02, 0.04, 1, 1)
move_servo_j(jnt[67.0438,-41.5110,53.5442,-14.9398,111.9343,0.1912], 0.02, 0.04, 1, 1)
```

- servo 는 **주기적으로 계속** 보내야 함 (예: 20 ms 마다 다음 목표, `t1` = 주기). 한꺼번에 보내면 앞 목표가 덮어써질 것으로 보임 [추정, 직접 시험하지 않음]

3.2 터미널(nc)

```
nc 10.0.2.7 5000                          # 명령 포트에 접속 후 한 줄씩 입력
pgmode simulation
move_j(jnt[0,0,0,0,0,0], 20, 40)
```

3.3 Python (명령 + 상태 읽기)

```
import socket, struct

IP = "10.0.2.7"
cmd = socket.create_connection((IP, 5000)); cmd.setsockopt(socket.IPPROTO_TCP, socket.TCP_NODELAY, 1)
dat = socket.create_connection((IP, 5001))

def send(line):                       # 한 줄 전송 (개행 포함)
    cmd.sendall((line.strip() + "\n").encode())

def read_state():                     # reqdata → 580바이트
    dat.sendall(b"reqdata\n")
    buf = b""
    while len(buf) < 580:
        buf += dat.recv(580 - len(buf))
    assert buf[0] == 0x24 and buf[3] == 0x03, "헤더 불일치"
    return dict(jnt_ref=struct.unpack_from("<6f", buf, 8), jnt_ang=struct.unpack_from("<6f", buf, 32),
                robot_state=struct.unpack_from("<i", buf, 340)[0], sim=struct.unpack_from("<i", buf, 404)[0])

send("pgmode simulation")
print(read_state())
```

4. 데이터 포트 (5001): `reqdata`

- 요청: 텍스트 `reqdata\n` → 응답: 580바이트 이진, **리틀엔디안** [관측]
- 헤더 확인: 첫 바이트 `0x24`, 네 번째 바이트 `0x03`

이 프로젝트에서 읽는 필드 (오프셋은 바이트) [관측]

오프셋	형식	필드	설명
4	float	제어기 시간	
8	float × 6	`jnt_ref`	관절 **지령**값 (deg). 시뮬레이션의 위치
32	float × 6	`jnt_ang`	관절 **엔코더**값 (deg). 실기의 위치, 시뮬레이션에서는 0
104	float × 6	`tcp_pos`	TCP 위치 (이 프로젝트에서는 사용하지 않음)
340	int	`robot_state`	1 = 정지(idle), 3 = 실행 중
404	int	`real_vs_simulation_mode`	1 = 시뮬레이션, 0 = 실기

- 위 외의 필드(속도 배율 등)는 공식 문서의 "Request 데이터 구조 매뉴얼" 참조
- 모드를 확인하는 습관: 모션 명령 전후에 `real_vs_simulation_mode` 를 읽어 의도한 모드인지 검사

5. 사용상 주의

1. **안전**: 명령은 확인 없이 즉시 실행. `pgmode real` 상태의 모션은 실제 로봇이 움직임. `task stop` 은 비상정지가 아니므로 물리 E-Stop 을 손이 닿는 곳에 둘 것
2. **완료 확인**: 모션 완료는 측정각이 목표에 도달 + `robot_state` = 1 로 판단. 시뮬레이션에서 `jnt_ang`(0)로 판단하면 도달하지 않았는데 도달로 오인함
3. **`move_servo_j` 범위**: `t2` 와 `alpha` 는 문서상 열린 구간. 경계값(`t2` = 0.2, `alpha` = 1.0)은 제어기가 거부하지 않았지만 문서 범위 밖일 수 있음
4. **servo 종료 시 오버슈트** [관측]: 마지막 지령 후 로봇이 (마지막 지령 속도 × `t2`) 만큼 더 나아가 멈춤. 마지막 목표를 여러 번 반복해 보내 속도를 0 으로 만들면 해소
5. **`move_jb2_*` 속도** [관측, 문서에 없음]
   - 설정 속도는 상한. 점이 촘촘하면(분할 수가 많으면) 도달 가능한 속도가 `√(accel × 구간 길이)` 정도로 낮아짐
   - 관절마다 속도·가속도 배율이 달라 어떤 관절이 포함되었는지에 따라 같은 설정에서도 속도가 달라짐 (예: J1·J2 단독은 설정의 약 0.68배, J3 약 0.93배, J4~J6 약 1.0배). 조합 규칙은 확정하지 못함
   - blend 가 클수록 경유점을 지나지 않고 코너를 깎음 (blend 0 은 점마다 정지하며 정확히 통과)
6. **명령 간격**: `move_servo_j` 의 `t1` 보다 짧은 간격으로 명령이 겹칠 때의 동작은 문서에 없음. `t1` 을 전송 주기와 같게 두는 방식으로 사용했음
7. 연속 전송 시 응답 대기(2.1절), 관절값 범위(-360 ~ 360)와 충돌 여부는 제어기가 검증해 주지 않으므로 호출 측에서 확인

6. 변경 이력

날짜	내용
2026-10-19	최초 작성 (간이판)

결과 데이터 명세서 (결과 CSV · 원본 궤적 CSV)

- 대상 보고서: 「RB 협동로봇 외부 스크립트 기반 궤적 추종 성능 평가 보고서」
- 수록 범위: 보고서 표의 원본 수치 (`results/` , CSV 7개), 실험에 쓴 궤적 (`trajectories/` , CSV 6개). 원시 측정 데이터는 미수록
- 로봇: rb20_1900es (RT), 실험일 2026-10-08, 비교 대상 24회 (jb2 blend 0 / 50 / 100, servo × 궤적 A·B·C × 시뮬레이션·실기)
- 형식: 구분자 쉼표. `results/` 는 UTF-8 (BOM 포함, 엑셀에서 바로 열림), `trajectories/` 는 UTF-8

`results/`

파일	내용	보고서 위치
`runs_index.csv`	실행 목록과 명령 파라미터	3.1절
`run_metrics.csv`	실행별 지표와 판정	3.3 ~ 3.4절
`method_summary.csv`	방식별 요약	3.5절
`link_displacement_rb20.csv`	궤적·방식·모드별 링크 변위	3.7절
`sim_real_pairs.csv`	시뮬레이션 대 실기	3.6절
`trajectory_chord_error.csv`	궤적별 지령 간격 현 오차	1.3절
`joint_sensitivity_rb20.csv`	관절별 1° 당 표면 변위	3.7절

`trajectories/`

파일	내용	보고서 위치
`waypoints_sample.csv` , `waypoints_robot1.csv` , `waypoints_robot2.csv`	궤적 A / B / C 의 입력 웨이포인트 (경로 계획기 출력)	1.2절, 3.1절
`reference_A_50Hz.csv` , `reference_B_50Hz.csv` , `reference_C_50Hz.csv`	입력 웨이포인트를 내삽한 원본 궤적 (평가 기준)	1.2 ~ 1.3절

0. 공통 정의

- 궤적: A `waypoints_sample` (14.67 s), B `waypoints_robot1` (8.16 s), C `waypoints_robot2` (5.78 s)
- 방식: jb2 = `move_jb2_add(J, 50, 120, 0, blend)` (blend 0 / 50 / 100), servo = `move_servo_j(J, 0.2, 0.2, 1, 1)` 0.2 s 주기, 마지막 점 30회 반복
- 궤적 추종 오차 `d_i` : 측정 관절각과 DTW 로 짝지은 원본 경로 위 점의 6관절 L2 거리 [deg]
- 표면 편차 (최악 점 변위) `D_i` : 측정 자세와 짝지은 원본 자세를 rb20_1900es_RT URDF 로 배치했을 때 같은 메시 점끼리 거리의 최댓값 [mm], 추종 구간의 매칭쌍 전체로 계산
- 통계: `_mean` 평균, `_p95` 95 백분위수, `_max` 최댓값 (모두 추종 구간 전체에 대한 값)
- 판정 (`verdict`): 속도 리플을 제외한 8개 항목 (추종 오차 p95 $\leq$ 1°, 최대 $\leq$ 2°, 커버리지 $\leq$ 2°, 진행도 $\geq$ 0.99, 샘플 간 이동 $\leq$ 5°, 시작/종료 각 오차 $\leq$ 0.5°, 흔들림 $\leq$ 0.3°, 정체 $\leq$ 2 %)

1. 파일별 명세

1.1 `runs_index.csv` — 실행 목록

열	내용
`run`	실행 폴더 이름
`role`	`비교 대상` 또는 `보조: <용도>`
`method_group`	`jb2 blend 0` / `jb2 blend 50` / `jb2 blend 100` / `servo`
`trajectory`, `trajectory_file`	A / B / C, 입력 파일
`mode`	시뮬레이션 / 실기
`command`	`move_jb2_add` / `move_servo_j`
`speed`, `accel`, `blend`	jb2 파라미터 [deg/s, deg/s², %]
`t1`, `t2`, `gain`, `alpha`, `final_hold`	servo 파라미터 [s, s, -, -, 회]
`subsample_step`	서브샘플링 비율

1.2 `run_metrics.csv` — 실행별 지표 (비교 대상 24회)

열	단위	내용
`run`, `method_group`, `trajectory`, `mode`	-	실행 식별
`t_track`	s	추종 시간 (참고)
`path_dev_mean`, `path_dev_p95`, `path_dev_max`	deg	궤적 추종 오차
`cover_max`, `progress_max`, `max_step`	deg, 0 ~ 1, deg	커버리지 최대, 진행도, 샘플 간 최대 이동
`start_err`, `end_err`	deg	시작 / 종료 각 오차
`smooth_ripple`, `smooth_wobble`, `smooth_stall`	비율, deg, 비율	속도 리플(참고), 횡방향 흔들림, 정체 비율
`t_latency`	s	지령 → 모션 시작
`surf_disp_mean`, `surf_disp_p95`, `surf_disp_max`	mm	최악 점 변위 `D_i` 의 추종 구간 평균, 95 백분위수, 최댓값 (rb20)
`surf_chamfer_max`, `surf_hausdorff_max`	mm	Chamfer, Hausdorff 의 추종 구간 최댓값
`surf_worst_link`	-	추종 구간 중 최대 변위가 나온 링크
`verdict`, `failed_items`	-	**보고서 판정** (리플 제외 8개 항목), 불합격 항목

1.3 `method_summary.csv` — 방식별 요약 (보고서 3.5절 종합 표)

- 행: `jb2 blend 0`, `jb2 blend 50`, `servo`, `jb2 blend 100` (각 6회)
- 열: `n_runs`, 그리고 `path_dev_p95`, `path_dev_max`, `smooth_ripple`, `smooth_wobble`, `surf_disp_mean`, `surf_disp_p95`, `surf_disp_max` 각각의 `_median`, `_min`, `_max`

1.4 `sim_real_pairs.csv` — 시뮬레이션 대 실기 (보고서 3.6절)

열	단위	내용
`method_group`, `trajectory`	-	조건
`sim_run`, `real_run`	-	짝지은 실행
`path_dist_p95_deg`, `path_dist_max_deg`	deg	실기 측정 경로의 각 점에서 시뮬레이션 측정 경로까지 거리 (시간 무관)
`sim_path_dev_p95`, `real_path_dev_p95`	deg	각각의 궤적 추종 오차 p95
`sim_surf_disp_p95`, `real_surf_disp_p95`	mm	각각의 표면 변위 p95

1.5 `trajectory_chord_error.csv` — 지령 간격에 따른 현 오차 (보고서 1.3절)

열	단위	내용
`trajectory`, `file`	-	궤적
`duration_s`	s	원본 길이
`n_ref_points_50Hz`, `n_command_points`	개	원본 점 수, 지령점 수
`path_length_deg`	deg	경로 길이 (6관절 L2 호길이)
`chord_err_p95_deg`, `chord_err_max_deg`	deg	원본 각 점에서 지령 꺾은선까지 거리의 95 백분위수, 최댓값

1.6 `joint_sensitivity_rb20.csv` — 관절별 1° 감도 (보고서 3.7절)

열	단위	내용
`joint`	-	J1 ~ J6
`surface_disp_max_mm_per_1deg`	mm	해당 관절만 1° 돌렸을 때 최악 점 변위 (궤적 A 시작 자세 기준)

1.7 `link_displacement_rb20.csv` — 궤적별 링크 변위 (보고서 3.7절)

- 행 = 실행 1개 × 링크 1개 (24 × 7 = 168행)
- 링크마다 "그 순간 해당 링크의 최악 점 변위" 시계열을 매칭쌍 전체로 구해 요약

열	단위	내용
`trajectory`, `method_group`, `mode`, `run`	-	실행 식별
`link`	-	`link1` ~ `link6`, `link_end` (URDF 의 `rt_link1` ~ `rt_link_end`, 베이스 제외)
`disp_max_mm`	mm	추종 구간 중 해당 링크 변위의 최댓값
`disp_p95_mm`	mm	95 백분위수 (추종 시간의 95 % 는 이 값 이하)
`disp_mean_mm`	mm	평균

2. 궤적 파일 명세 (`trajectories/`)

2.1 입력 웨이포인트 `waypoints_*.csv`

파일	궤적	열	단위	비고
`waypoints_sample.csv`	A	`waypoint, rt_joint_linear_track, rt_joint_carriage, rt_base, rt_shoulder, rt_elbow, rt_wrist1, rt_wrist2, rt_wrist3`	rad (팔 6관절)	앞 2열(외부축: 리니어 트랙, 캐리지)은 사용하지 않음
`waypoints_robot1.csv`	B	`Order, J1_rad .. J6_rad, Speed, Acceleration`	rad	`Speed`, `Acceleration` 열은 사용하지 않음
`waypoints_robot2.csv`	C	`Order, J1_rad .. J6_rad, Speed, Acceleration`	rad	위와 같음

- 행 = 웨이포인트 1개, 시간 정보 없음 (순서만 의미)

2.2 원본 궤적 `reference_*_50Hz.csv`

- 생성: 연속 중복 웨이포인트 제거 → 구간 시간 부여 (최대 이동 관절 기준 20 deg/s, 하한 0.2 s) → 3차 스플라인 (시작·끝 속도 0) → 50 Hz 샘플링
- 같은 궤적을 쓴 모든 실행에서 동일 (실행 폴더의 `traj_ref.csv` 와 같은 내용)
- 로봇 지령점은 이 파일의 10번째 점마다 (0.2 s 간격, 마지막 점 포함)

열	단위	내용
`t`	s	원본 시간 (0 시작, 0.02 s 간격)
`j0` ~ `j5`	**rad**	관절각 J1(base) ~ J6(wrist3)

파일	길이	점 수
`reference_A_50Hz.csv`	14.67 s	735
`reference_B_50Hz.csv`	8.16 s	410
`reference_C_50Hz.csv`	5.78 s	291

3. 주의사항

- 시뮬레이션 측정각은 제어기 지령 추종값 `jnt_ref`, 실기는 관절 엔코더 `jnt_ang` → 링크 처짐, 백래시, 공구 끝 위치 미포함
- 표면 편차는 URDF 와 실제 로봇이 같다는 가정에 기반 (일치 여부 미확인), 공구·케이블 미포함
- 각도 단위: 궤적 파일은 rad, 결과 CSV 는 deg
- `runs_index.csv` 의 보조 실행 2개는 보고서 설명 그림용이며 다른 표에는 포함되지 않음

실험 데이터 명세서

- 대상 보고서: 「RB 협동로봇 외부 스크립트 기반 궤적 추종 성능 평가 보고서」 (`report.md` / `report.pdf`)
- 수록 범위: **보고서에 사용된 데이터만** 수록
- 로봇: Rainbow Robotics rb20_1900es (RT), 제어기 IP 10.0.2.7, 외부 스크립트 API (TCP 5000/5001)
- 실험일: 2026-10-08
- 생성 도구: `rb_servo_gui/docs/report/make_report_dataset.py` (원본 실행 폴더에서 복사·계산, 원본은 수정하지 않음)

1. 구성

```
report_dataset_20261008/
├── DATA_SPEC.md                이 문서
├── runs/                       비교 대상 실행 24개 (실행마다 폴더 1개)
│   └── <실행 이름>/
│       ├── config.json          실행 설정
│       ├── traj_ref.csv         원본 궤적 (50 Hz 내삽)
│       ├── traj_sub10.csv       로봇 지령점 (1/10 서브샘플링)
│       ├── follow_*.csv         측정 기록 (원시 데이터)
│       ├── compare_*_pairs.csv  DTW 매칭쌍과 쌍별 오차
│       ├── compare_*.json       실행별 지표와 판정
│       ├── velocity_*.csv       추종 속도
│       ├── smooth_*.csv         매끄러움 시계열
│       └── surface_rb20_*.csv   링크 표면 편차 시계열 (rb20 URDF, 이 묶음에서 새로 계산)
├── aux_runs/                   설명 그림용 보조 실행 2개 (비교에는 미사용)
├── inputs/                     입력 웨이포인트 3종
├── urdf/                       rb20_1900es_RT.urdf + 충돌 메시
└── tables/                     보고서 표의 원본 수치 (CSV)
```

- 모든 CSV: UTF-8. 이 묶음에서 새로 만든 CSV(`tables/*.csv`, `surface_rb20_*.csv`)는 엑셀 호환을 위해 BOM 포함 (UTF-8-SIG)
- 그림(PNG)은 수록하지 않음 (보고서에 포함)

2. 실행 목록

2.1 비교 대상 (24회)

- 조건: 궤적 3종을 시뮬레이션과 실기로 모두 수행한 조건만
- 실행 이름 규칙: `<날짜>_<시각>[_REAL]_<방식>_step<서브샘플링 비율>` (`REAL` 이 있으면 실기, 없으면 시뮬레이션)

방식	궤적	시뮬레이션	실기
jb2 blend 0	A	`20261008_165637_jb2_step10`	`20261008_164607_REAL_jb2_step10`
jb2 blend 0	B	`20261008_165307_jb2_step10`	`20261008_164116_REAL_jb2_step10`
jb2 blend 0	C	`20261008_165514_jb2_step10`	`20261008_165012_REAL_jb2_step10`
jb2 blend 50	A	`20261008_165736_jb2_step10`	`20261008_164714_REAL_jb2_step10`
jb2 blend 50	B	`20261008_165339_jb2_step10`	`20261008_164156_REAL_jb2_step10`
jb2 blend 50	C	`20261008_165540_jb2_step10`	`20261008_165048_REAL_jb2_step10`
jb2 blend 100	A	`20261008_165824_jb2_step10`	`20261008_164805_REAL_jb2_step10`
jb2 blend 100	B	`20261008_165406_jb2_step10`	`20261008_164230_REAL_jb2_step10`
jb2 blend 100	C	`20261008_165602_jb2_step10`	`20261008_165117_REAL_jb2_step10`
servo	A	`20261008_163900_servo_step10`	`20261008_164846_REAL_servo_step10`
servo	B	`20261008_163425_servo_step10`	`20261008_164302_REAL_servo_step10`
servo	C	`20261008_163634_servo_step10`	`20261008_165140_REAL_servo_step10`

2.2 보조 실행 (2회, 비교에는 미사용)

실행	조건	용도
`20261008_132802_servo_step10`	servo, 궤적 C, 시뮬레이션, gain 1, 마지막 점 반복 없음	종료 오버슈트 사례 (보고서 3.2절 그림)
`20261008_162955_servo_step10`	servo, 궤적 C, 시뮬레이션, gain 0.2, 마지막 점 30회 반복	지표 설명 그림의 원본 (보고서 1.5절·1.7절의 DTW 매칭, 추종 오차, 리플, 흔들림 그림)

2.3 공통 조건

항목	값
내삽	시간 부여 `v_max = 20 deg/s` (구간 하한 0.2 s), 3차 스플라인(시작·끝 속도 0), 50 Hz
서브샘플링	10 (지령 간격 0.2 s, 마지막 점 포함)
측정	`reqdata` 50 Hz
jb2	`move_jb2_add(J, 50, 120, 0, blend)`, `move_jb2_run(0)` — speed 50 deg/s, accel 120 deg/s², blend 0 / 50 / 100 %
servo	`move_servo_j(J, 0.2, 0.2, 1, 1)` 0.2 s 주기 — t1 = t2 = 0.2 s, gain 1, alpha 1, 마지막 점 30회 반복
시작 자세 이동	`move_j(첫 점, 50, 120)`

3. 입력 데이터 (`inputs/`)

파일	궤적	형식	단위	비고
`waypoints_sample.csv`	A	`waypoint, rt_joint_linear_track, rt_joint_carriage, rt_base, rt_shoulder, rt_elbow, rt_wrist1, rt_wrist2, rt_wrist3`	rad (팔 6관절)	경로 계획기 출력. 앞 2열 (외부축)은 사용하지 않음
`waypoints_robot1.csv`	B	`Order, J1_rad .. J6_rad, Speed, Acceleration`	rad	`Speed` , `Acceleration` 열은 사용하지 않음
`waypoints_robot2.csv`	C	`Order, J1_rad .. J6_rad, Speed, Acceleration`	rad	위와 같음

4. 실행 폴더 파일 명세 (`runs/<실행>/` , `aux_runs/<실행>/`)

4.1 공통 규칙

- 관절 순서: 0 ~ 5 = J1(base) ~ J6(wrist3)
- 각도 단위: **`traj_ref.csv` , `traj_sub10.csv` 는 rad, 나머지는 deg**
- 첫 줄이 `#` 로 시작하는 CSV: 첫 줄은 JSON 메타데이터, 둘째 줄부터 일반 CSV
- 시간 기준
  - `t` (follow): 기록 시작 시각 기준 [s]
  - `t_track` : 모션 시작 시각 기준 [s] (모션 시작 = 시작 자세에서 0.2° 넘게 벗어나기 직전 샘플)

4.2 `config.json` — 실행 설정

키	내용
`requested`	사용자가 입력한 설정
`effective`	실제 적용된 설정 (주요 키 아래)

effective 키	단위	내용
`ip`	-	제어기 IP
`robot_mode`	-	`simulation` / `real`
`traj_path`	-	입력 웨이포인트 경로
`vmax` , `interp_rate`	deg/s, Hz	내삽 시간 부여 속도, 내삽 주기
`mode` , `step`	-	`jb2` / `servo` , 서브샘플링 비율
`speed` , `accel` , `blend`	deg/s, deg/s², %	jb2 및 시작 자세 이동 파라미터
`t1` , `t2` , `gain` , `alpha`	s, s, -, -	servo 파라미터 (`t1` , `t2` 가 0 이면 지령 간격 사용)
`final_t2` , `final_hold`	s, 회	servo 마지막 지령의 t2 (0 = t2 와 동일), 마지막 점 반복 횟수
`reach_tol` , `reach_hold`	deg, s	모션 종료 판정 허용 오차, 유지 시간
`record_hz`	Hz	측정 주기
`th_*`	-	판정 기준값 (프로그램 기본값, `th_ripple` 포함)

4.3 `traj_ref.csv` — 원본 궤적

열	단위	내용
`t`	s	원본 시간 (0 시작, 0.02 s 간격)
`j0` ~ `j5`	**rad**	3차 스플라인 내삽 관절각

4.4 `traj_sub10.csv` — 로봇 지령점

열	단위	내용
`t`	s	원본 시간 (0.2 s 간격, 마지막 점은 원본 끝 시각)
`j0` ~ `j5`	**rad**	지령 관절각 (로봇에는 deg 로 변환해 전송)

4.5 `follow_*.csv` — 측정 기록 (원시 데이터)

- 첫 줄 메타데이터 (JSON)

키	내용
`timestamp`, `mode`, `robot_mode`	실행 시각, 방식, 모드
`step`, `n_points`, `dt`	서브샘플링 비율, 지령점 수, 지령 간격 [s]
`traj`, `record_hz`	입력 파일 이름, 측정 주기
`measured_source`	측정각 출처 (시뮬레이션 `jnt_ref`, 실기 `jnt_ang`)
`methods`	명령 파라미터
`marks`	이벤트 시각 [s, `t` 기준]: `servo` / `jb2` = 추종 지령 시각, `servo_sent` = 마지막 servo 지령 전송, `*_done` = 완료 확인, `jb2_build` = 리스트 구성 시작

- 열

열	단위	내용
`t`	s	기록 시작 기준 시각
`phase`	-	`<방식>_move_to_start` (시작 자세 이동), `<방식>` (추종), `<방식>_end` (종료 후)
`robot_state`	-	`reqdata` robot_state (1 = idle, 3 = 실행 중)
`meas0` ~ `meas5`	deg	측정 관절각 (시뮬레이션: `jnt_ref`, 실기: `jnt_ang` 엔코더)

4.6 `compare_*_pairs.csv` — DTW 매칭쌍

- 첫 줄 메타데이터: 명령 파라미터, `T_ref` (원본 길이 [s]), `T_track` (추종 시간 [s]), `reached`
- 행 = 추종 구간의 측정 샘플 1개

열	단위	내용
`t_track`	s	모션 시작 기준 측정 시각
`t_ref_matched`	s	짝지은 원본 위 점의 원본 시각
`ref_idx`	-	DTW 가 고른 원본 정점 번호 (`traj_ref.csv` 행 번호, 0 시작)
`progress`	0 ~ 1	짝지은 점의 원본 경로 진행도 (호길이 비율)
`meas_q0` ~ `meas_q5`	deg	측정 관절각 `q_i`
`ref_q0` ~ `ref_q5`	deg	짝지은 원본 경로 위 점 `p_i` (정점 양옆 선분에 투영한 연속 위치)
`err_q0` ~ `err_q5`	deg	쌍 오차 $e_i = q_i - p_i$
`pair_dist`	deg	궤적 추종 오차 $d_i = \|e_i\|_2$

4.7 `compare_*.json` — 실행별 지표와 판정

키	내용
`source`, `mode`, `params`, `step`	측정 파일, 방식, 명령 파라미터, 서브샘플링 비율
`metrics`	지표 (아래)
`verdict`	항목별 판정 (**속도 리플 항목 포함**, 보고서는 이 항목을 제외)
`all_pass`	전 항목 통과 여부 (**리플 포함 기준**, 보고서 판정과 다를 수 있음 → `tables/run_metrics.csv` 의 `verdict` 사용)

`metrics` 키	단위	내용
`t_track`, `t_ref`, `track_ratio`	s, s, -	추종 시간, 원본 길이, 비율 (참고값)
`t_latency`, `t_total`, `t_build`	s	지령 → 모션 시작, 지령 → 완료 확인, jb2 리스트 구성(고정 대기 0.5 s 포함)
`path_dev_mean`, `_rms`, `_p95`, `_max`	deg	궤적 추종 오차 `d_i` 통계
`pair_rmse_after`, `time_rmse_before`	deg	관절별 RMSE (DTW 쌍 기준 / 같은 정규화 시각 기준)
`cover_mean`, `_p95`, `_max`	deg	커버리지 (원본 각 점 → 측정 경로 거리)

metrics 키	단위	내용
progress_max	0 ~ 1	경로 진행도
max_step	deg	연속 측정 샘플 간 최대 이동
start_err, end_err	deg	시작 / 종료 각 오차 (관절별 최대)
smooth_ripple	비율	속도 리플 R (참고 지표)
smooth_wobble	deg	횡방향 흔들림 W
smooth_stall	비율	정체 비율 S
rough_ratio, rough_score_max	비율, -	국소 거칠기 점수 기준 초과 비율, 최대 점수
v_max_joint, v_max_norm, v_mean_norm	deg/s	관절 최대 속도, 6관절 속력 최대 / 평균
lag_max, lag_min, lag_mean, lag_abs_max	비율	정규화 시간 기준 타이밍 지연 (+ 늦음 / − 빠름, 참고)
lag_max_s, late_ratio, early_ratio	s, 비율, 비율	지연 최대의 초 환산, 늦음 / 빠름 구간 비율
reached	bool	최종 목표 도달 여부

- 링크 표면 지표(surf_*)는 실험 당시 계산하지 않아 JSON 에 없음 → surface_rb20_*.csv , tables/run_metrics.csv 사용

4.8 velocity_*.csv — 추종 속도

열	단위	내용
t_track	s	모션 시작 기준 시각
v0 ~ v5	deg/s	관절 속도 (부호 있음, ±0.1 s 중심 차분)
speed_norm	deg/s	6관절 속력 $\|v\|_2$

4.9 smooth_*.csv — 매끄러움 시계열

열	단위	내용
t_track	s	모션 시작 기준 시각
progress	0 ~ 1	경로 진행도
speed_along_path	deg/s	경로 속력 v^s (진행 호길이의 시간 변화율)
trend	deg/s	경로 속력의 이동평균 추세
local_ripple	비율	국소 리플 R_i
local_wobble_deg	deg	국소 흔들림 W_i
score	-	국소 거칠기 점수 (1 = 기준선)

4.10 surface_rb20_*.csv — 링크 표면 편차 시계열 (이 묶음에서 계산)

- 계산 조건
  - URDF: urdf/rb20_1900es_RT.urdf (충돌 메시), 링크당 표면 점 600개
  - 대상 링크: rt_link1 ~ rt_link_end (팔과 함께 움직이는 링크, 레일·캐리지 제외), 외부축은 두 자세 모두 0
  - **매칭쌍 전체** 사용 (compare_*_pairs.csv 의 모든 행)
  - 측정 자세 meas_q* 와 원본 자세 ref_q* 를 각각 순기구학으로 배치해 비교

열	단위	내용
t_track	s	모션 시작 기준 시각 (compare_*_pairs.csv 와 같은 행)
disp_max_mm	mm	그 순간 최악 점 변위 D_i (같은 메시 점끼리 거리의 최댓값)
disp_p95_mm, disp_mean_mm	mm	그 순간 메시 점 변위의 95 백분위수, 평균

열	단위	내용
`chamfer_mm`	mm	Chamfer 거리 (최근접 거리 평균)
`hausdorff_mm`	mm	Hausdorff 거리 (최근접 거리 최댓값)

5. 요약 표 명세 (`tables/`)

5.1 `runs_index.csv` — 실행 목록

열	내용
`run`	실행 폴더 이름
`role`	`비교 대상` 또는 `보조: <용도>`
`method_group`	`jb2 blend 0` / `jb2 blend 50` / `jb2 blend 100` / `servo`
`trajectory` , `trajectory_file`	A / B / C, 입력 파일
`mode`	시뮬레이션 / 실기
`command`	`move_jb2_add` / `move_servo_j`
`speed` , `accel` , `blend`	jb2 파라미터 [deg/s, deg/s², %]
`t1` , `t2` , `gain` , `alpha` , `final_hold`	servo 파라미터 [s, s, -, -, 회]
`subsample_step`	서브샘플링 비율

5.2 `run_metrics.csv` — 실행별 지표 (비교 대상 24회)

열	단위	내용
`run` , `method_group` , `trajectory` , `mode`	-	실행 식별
`t_track`	s	추종 시간 (참고)
`path_dev_mean` , `path_dev_p95` , `path_dev_max`	deg	궤적 추종 오차
`cover_max` , `progress_max` , `max_step`	deg, 0 ~ 1, deg	커버리지 최대, 진행도, 샘플 간 최대 이동
`start_err` , `end_err`	deg	시작 / 종료 각 오차
`smooth_ripple` , `smooth_wobble` , `smooth_stall`	비율, deg, 비율	속도 리플(참고), 횡방향 흔들림, 정체 비율
`t_latency`	s	지령 → 모션 시작
`surf_disp_mean` , `surf_disp_p95` , `surf_disp_max`	mm	최악 점 변위 `D_i` 의 추종 구간 평균, 95 백분위수, 최댓값 (rb20)
`surf_chamfer_max` , `surf_hausdorff_max`	mm	Chamfer, Hausdorff 의 추종 구간 최댓값
`surf_worst_link`	-	추종 구간 중 최대 변위가 나온 링크
`verdict` , `failed_items`	-	**보고서 판정** (리플 제외 8개 항목), 불합격 항목

5.3 `method_summary.csv` — 방식별 요약 (보고서 3.5절 종합 표)

- 행: `jb2 blend 0` , `jb2 blend 50` , `servo` , `jb2 blend 100` (각 6회)
- 열: `n_runs` , 그리고 `path_dev_p95` , `path_dev_max` , `smooth_ripple` , `smooth_wobble` , `surf_disp_mean` , `surf_disp_p95` , `surf_disp_max` 각각의 `_median` , `_min` , `_max`

5.4 `sim_real_pairs.csv` — 시뮬레이션 대 실기 (보고서 3.6절)

열	단위	내용
`method_group` , `trajectory`	-	조건
`sim_run` , `real_run`	-	짝지은 실행
`path_dist_p95_deg` , `path_dist_max_deg`	deg	실기 측정 경로의 각 점에서 시뮬레이션 측정 경로까지 거리 (시간 무관)
`sim_path_dev_p95` , `real_path_dev_p95`	deg	각각의 궤적 추종 오차 p95
`sim_surf_disp_p95` , `real_surf_disp_p95`	mm	각각의 표면 변위 p95

5.5 `trajectory_chord_error.csv` — 지령 간격에 따른 현 오차 (보고서 1.3절)

열	단위	내용
`trajectory`, `file`	-	궤적
`duration_s`	s	원본 길이
`n_ref_points_50Hz`, `n_command_points`	개	원본 점 수, 지령점 수
`path_length_deg`	deg	경로 길이 (6관절 L2 호길이)
`chord_err_p95_deg`, `chord_err_max_deg`	deg	원본 각 점에서 지령 꺾은선까지 거리의 95 백분위수, 최댓값

5.6 `joint_sensitivity_rb20.csv` — 관절별 1° 감도 (보고서 3.7절)

열	단위	내용
`joint`	-	J1 ~ J6
`surface_disp_max_mm_per_1deg`	mm	해당 관절만 1° 돌렸을 때 최악 점 변위 (궤적 A 시작 자세 기준)

5.7 `link_displacement_rb20.csv` — 궤적별 링크 변위 (보고서 3.7절)

- 행 = 실행 1개 × 링크 1개 (24 × 7 = 168행)
- 링크마다 "그 순간 해당 링크의 최악 점 변위" 시계열을 매칭쌍 전체로 구해 요약

열	단위	내용
`trajectory`, `method_group`, `mode`, `run`	-	실행 식별
`link`	-	`link1` ~ `link6`, `link_end` (URDF 의 `rt_link1` ~ `rt_link_end`, 베이스 제외)
`disp_max_mm`	mm	추종 구간 중 해당 링크 변위의 최댓값
`disp_p95_mm`	mm	95 백분위수 (추종 시간의 95 % 는 이 값 이하)
`disp_mean_mm`	mm	평균

6. 주의사항

- 시뮬레이션의 측정각은 `jnt_ref` (제어기 지령 추종값)이며, 시뮬레이션에서 `jnt_ang` 는 0 으로 고정되어 기록하지 않음
- 실기 측정각 `jnt_ang` 는 관절 엔코더 값 → 링크 처짐, 감속기 백래시, 공구 끝 위치 미포함
- JSON 의 `verdict` , `all_pass` 는 속도 리플을 포함한 프로그램 판정 → 보고서 판정은 `tables/run_metrics.csv` 의 `verdict`
- 표면 편차는 URDF 와 실제 로봇이 같다는 가정에 기반 (일치 여부 미확인), URDF 에 없는 공구·케이블 미포함
- `urdf/` 에는 계산에 쓰는 충돌 메시만 수록 (URDF 가 참조하는 시각화 메시 `.dae` 는 미수록 → 시각화 용도로는 원본 저장소의 `urdf/` 사용)
- 보조 실행 `162955` 는 servo gain 0.2 조건으로, 비교 대상에서 제외한 조건임 (지표 설명 그림에만 사용)

7. 재생성

```
cd rb_servo_gui
# 이 데이터 묶음
PYTHONPATH=src python docs/report/make_report_dataset.py --out <출력 폴더>
# 보고서 그림과 표 수치
PYTHONPATH=src python docs/report/make_report_figures.py
```